



31 Days of SSIS

Version 1.0

Jason Strate
[www.twitter.com/stratesql](https://twitter.com/stratesql)
www.jasonstrate.com

CONTENTS

Forward.....	8
Downloads	8
Biography	8
31 Days of SSIS – The Introduction.....	9
The 31 Days of SSIS Posts	9
Raw Files Are Awesome.....	11
Raw File Format	11
Raw File Format Destination.....	12
Raw File Format Source	14
Raw File Wrap-Up	15
References	15
Using Breakpoints.....	16
Breakpoint Demonstration	16
Breakpoint Wrap-Up	19
References	19
Data Flow Breakpoints?!.....	20
Data Viewers Definition.....	20
Data Viewer Setup.....	20
Data Viewer In Use.....	22
Data Viewers Wrap-Up.....	23
Folder Structures	25
The Folder Structure.....	25
The Folder Purpose	25
Folder Structure Wrap-Up.....	26
One Variable to Rule Them All	27
Variable Expressions.....	27
The One Variable.....	28
Variable Wrap-Up.....	29
References	30
Recursive SSIS Package	31
Package Requirements	31
Recursive Solution	31
Package Control Flow	31

Package Variables.....	32
Package Data Flows.....	33
DF – Build Hierarchy Root	33
DF – Build Hierarchy Levels.....	33
Package Output.....	34
Recursive Wrap-up	34
What The Sorts?!.....	35
Sort Configuration	35
String Comparisons	35
Advanced Setting	36
Sort Considerations	36
Sort Wrap-Up	37
Merge Join Is My Friend	38
Package Requirements	38
Merge Join Solution.....	38
Package Control Flow	38
Package Variables.....	39
Package Data Flows.....	39
DF – Data Setup	39
DF – Match Rows	40
Configuring Source Sort Order.....	40
Configure Merge Join Transformation	42
Word of Caution	42
Merge Join Wrap-up.....	42
Override Variables	44
Variable Expressions.....	44
Override Expressions	45
Another Consideration	47
Wrap-Up	47
Pivot Transformation.....	48
Package Control Flow	48
Package Data Flows.....	48
DFT – Pivot Transformation	48
Pivot Transformation Configuration	49

Component Properties	49
Input Columns	49
Input and Output Properties	49
Pivot Transformation Wrap-Up	52
UnPivot Transformation	53
Package Control Flow	53
Package Data Flows.....	53
DFT – Unpivot Transformation	53
Unpivot Transformation Configuration	54
Unpivot Transformation Wrap-Up	55
UnPivot Alternative	57
Package Requirements	57
Unpivot Alternative Solution	57
Package Control Flow	57
Package Variables.....	59
Package Data Flows.....	60
DFT_Demo_File	60
DFT_Unpivot.....	60
SCR_Unpivot Configuration	60
Unpivot Wrap-up	63
Create XML Document.....	64
Package Requirements	64
Script Task Solution	64
Package Control Flow	64
Package Variables.....	64
Package Data Flows.....	65
DFT_Demo_File	65
DFT_XMLDocument	65
SCR_Fill_XMLData Configuration	66
Input0_ProcessInputRow Function	67
Formatting Functions	68
XML Script Task Wrap-Up	69
Instant Excel Spreadsheet.....	71
Trouble In Excel Land	71

Instant Spreadsheet	71
Excel Wrap-Up	72
Every Which Way Outputs	74
Multiple Outputs Requirements	74
Multiple Outputs Solution	74
Package Control Flow	75
Package Variables.....	75
Package Data Flows.....	75
DFT_MultiOutputs.....	75
SCR_MultiOutput Configuration	76
Multiple Outputs Wrap-up	78
10 Things About Expressions	79
1. Expression Evaluation	79
2. Use SystemStartDate for Timestamps.....	79
3. Conditional Logic	79
4. AND and OR	79
5. Backslash.....	80
6. Too Many Parenthesis	80
7. Too Few Parenthesis.....	80
8. Date Functions	81
9. Missing Functions Suck	81
10. Concatenation	81
Expression Wrap-Up.....	82
Environmental Variable	83
Why Environmental Variable?	83
Environmental Variable Setup	83
Environmental Variable Wrap-Up.....	84
SQL Server Configuration.....	86
Why SQL Server Configuration?	86
SQL Server Configuration Setup.....	86
SQL Server Configuration Wrap-Up	87
DTSTConfig Configuration.....	89
Heading.....	89
Configuration	89

Why XML Configuration?.....	90
XML Configuration Setup.....	90
Setup Complete.....	91
XML Configuration Wrap-Up	91
No More Procedures	93
Best Practice Roots.....	93
Output Contract	93
Best Practice Wrap-Up	94
Controlling Your Flow In The Control Flow	95
Guest Blogger.....	95
What The Flow?	95
Keeping With the Flow	95
In Sequence.....	96
Flow Wrap-up	96
Data Flow Expressions	97
SSIS Configuration Problem	97
Solution Through Data Flow Expressions	97
Lookup Transformation Setup	97
Data Flow Expressions Wrap-Up.....	99
Generating Row Numbers	100
Package Requirements	100
Row Number Solution	100
Package Control Flow	100
Package Data Flow	100
DF_RowNumbers	101
Script Component Transformation Setup	101
SCR_RowNumberAll.....	101
Row Number Wrap-up	102
One Package, Unlimited Databases	104
Package Requirements	104
One Package Solution.....	104
Package Control Flow	104
Package Variables.....	105
Connection Manager.....	105

Package Data Flows	106
DFT_InsertUpdateStateProvince	106
DFT_DeleteStateProvince	106
Last Tidbits	107
Package Results.....	107
One Package Wrap-Up	108
One Package, Unlimited Tables.....	109
Package Requirements	109
Unlimited Table Solution	109
Package Control Flow	110
Package Variables.....	110
Variable Expressions.....	111
Package Data Flows.....	111
Unlimited Tables Wrap-Up	111
Small Package, Big Win	113
Practice Impact	113
A Two-Part Story	114
Small Package.....	114
Big win	114
Wrap-up.....	114
Adding Some Delta Force.....	115
Delta Force Requirements	115
Delta Force Solution	115
Package Control Flow	115
Package Data Flow	116
DFT_InsertUpdateStateProvince	116
LKUP_Subscriber	116
SCR_DeltaPower	117
Wrap-Up	119
Jim Croce and the Foreach (File) Loop Container	120
Guest Blogger.....	120
Exporting Execution Plans.....	125
File Export Requirements	125
File Export Solution	125

Package Control Flow	125
Package Variables.....	125
Package Data Flow	126
DFT_ExportPlans	126
DS_MSS_ParallelPlans Query	126
EC_QueryPlan Configuration.....	127
Wrap-Up	127
Importing Execution Plans	129
File Import Requirements.....	129
File Export Solution	129
Package Control Flow	129
Package Variables.....	129
Package Data Flow	130
DFT_ExportPlans	130
DS_SCR_SQLPlan Configuration.....	130
IC_SQLPlan Configuration	132
Wrap-Up	132
SSIS Naming Conventions	134
Practice Impact	134
Convincing Me, Convincing You.....	134
Looking At My Package.....	134
Looking At Your Package	135
Naming Convention Wrap-Up	135

FORWARD

This document is a compilation of a 31 day blog series by Jason Strate. All of the posts were written by either Jason Strate with three guest posts by Mark Vaillancourt, Chris Fish, and Aaron Drinkwine. The content contained in this document is copyrighted by Jason Strate and may not be reused or redistributed without permission and attribution.

For more information on this and other posts, check out my blog at www.jasonstrate.com.

DOWNLOADS

The SSIS project and packages listed in the document can be downloaded.

BIOGRAPHY

JASON STRATE

Jason Strate, Digiener Inc, is a database architect and administrator with over fifteen years of experience. He is a recipient of the Microsoft Most Valuable Professional (MVP) for SQL Server since July, 2009. His experience includes design and implementation of both OLTP and OLAP solutions as well as assessment and implementation of SQL Server environments for best practices, performance, and high availability solutions. Jason is a SQL Server MCITP and participated in the development of Microsoft Certification exams for SQL Server 2008.

Jason enjoys helping others in the SQL Server community and does this by presenting at technical conferences and user group meetings. Most recently, Jason has presented at the SSWUG Virtual Conferences, TechFuse, numerous SQL Saturdays, and at PASSMN user group meetings. Jason is also a contributing author for the Microsoft white paper "Empowering Enterprise Solutions with SQL Server 2008 Enterprise Edition." Jason is an active blogger with a focus on SQL Server and related technologies.

CHRIS FISH

Chris Fish, a consultant with Digiener Inc., is a database developer with over 20 years of experience in IT. As a Microsoft Certified Solution Developer, he has done everything from business analysis to database administration with lots of development in between on various platforms and tools. In the past 10 years his focus has been on SQL Server database design and ETL development for both OLTP and OLAP implementations. Chris has most recently developed solutions using SSIS, SSRS and SSAS in SQL Server 2005 & 2008.

31 DAYS OF SSIS – THE INTRODUCTION

Last year, Glenn Berry ([Blog](#) | [@GlennAlanBerry](#)) wrote a series of posts that he called [A DMV A Day](#). Since then there have been two other series that followed a similar concept – Paul Randal ([Blog](#) | [@PaulRandal](#)) with [A SQL Server DBA Myth A Day](#) and Jonathan Kehayias ([Blog](#) | [@SQLSarg](#)) with [An XEvent A Day: 31 days of Extended Events](#).

Since I've been working on an SQL Server Integration Services (SSIS) project over the last six months, I thought I would blog a little about some of the tips and tricks and solutions that I've worked through with this project and other SSIS projects I've worked on. This isn't intended as training for learning how to use SSIS. The intent is to help prevent some of the same difficulties that I've run into and to provide some ideas on how to use SSIS creatively.

I've figured I'd follow suit with a series on SQL Server Integration Services (SSIS). Unlike many DBAs, I've spent quite a bit of time using SSIS. Enough time that I figure I probably have 31 days' worth of posts that I can write based on some of my experiences.

THE 31 DAYS OF SSIS POSTS

Below are all of the posts in this series and it'll be updated each day as the new posts come out.

1. [31 Days of SSIS – Raw Files Are Awesome \(1/31\)](#)
2. [31 Days of SSIS – Using Breakpoints \(2/31\)](#)
3. [31 Days of SSIS – Data Flow Breakpoints?! \(3/31\)](#)
4. [31 Days of SSIS – Folder Structures \(4/31\)](#)
5. [31 Days of SSIS – One Variable to Rule Them All \(5/31\)](#)
6. [31 Days of SSIS – Recursive SSIS Package \(6/31\)](#)
7. [31 Days of SSIS – What The Sorts?! \(7/31\)](#)
8. [31 Days of SSIS – Merge Join Is My Friend \(8/31\)](#)
9. [31 Days of SSIS – Override Variables \(9/31\)](#)
10. [31 Days of SSIS – Pivot Transformation \(10/31\)](#)
11. [31 Days of SSIS – UnPivot Transformation \(11/31\)](#)
12. [31 Days of SSIS – UnPivot Alternative \(12/31\)](#)
13. [31 Days of SSIS – Create XML Document \(13/31\)](#)
14. [31 Days of SSIS – Instant Excel Spreadsheet \(14/31\)](#)
15. [31 Days of SSIS – Every Which Way Outputs \(15/31\)](#)
16. [31 Days of SSIS – 10 Things About Expressions \(16/31\)](#)
17. [31 Days of SSIS – Environmental Variable\(17/31\)](#)
18. [31 Days of SSIS – SQL Server Configuration \(18/31\)](#)
19. [31 Days of SSIS – DTSTConfig Configuration \(19/31\)](#)
20. [31 Days of SSIS – No More Procedures \(20/31\)](#)
21. [31 Days of SSIS – Controlling Your Flow In The Control Flow \(21/31\)](#)
22. [31 Days of SSIS – Data Flow Expressions \(22/31\)](#)
23. [31 Days of SSIS – Generating Row Numbers \(23/31\)](#)
24. [31 Days of SSIS – One Package, Unlimited Databases \(24/31\)](#)
25. [31 Days of SSIS – One Package, Unlimited Tables \(25/31\)](#)
26. [31 Days of SSIS – Small Package, Big Win \(26/31\)](#)

- 27. [31 Days of SSIS – Adding Some Delta Force \(27/31\)](#)
- 28. [31 Days of SSIS – Jim Croce and the Foreach \(File\) Loop Container \(28/31\)](#)
- 29. [31 Days of SSIS – Exporting Execution Plans \(29/31\)](#)
- 30. [31 Days of SSIS – Importing Execution Plans \(30/31\)](#)
- 31. [31 Days of SSIS – SSIS Naming Conventions \(31/31\)](#)

RAW FILES ARE AWESOME

As I mentioned in the [introductory post](#), I'm writing [31 Days of SSIS](#). Let's start off this series with a SSIS file format that I was unaware of until just over a year ago. As the title already gives away – this file format is the raw file format. And it is awesome!

You've heard about this file format before, you say? Unfortunately, most people that I talk about SSIS with haven't heard of it or know when to apply it to their solutions. So the purpose for today is to define, describe, and demonstrate.

RAW FILE FORMAT

Starting with the definition, the raw file format is the data format that is native to SSIS. When data is being consumed by the data flows of an SSIS package it has a format. This format is the same that appears in raw files.

Since these are the same file format, SSIS doesn't have to parse or translate the data between the data streams or the raw file format. This results in the raw file format being faster than most other data source and destination formats. For some statistics on the speed benefit see [this post](#) by John Welch ([blog](#) | [@John_Welch](#)).

The raw file format can be incredibly useful. It can be used for either a [data source](#) or a [data destination](#). Once you've gotten your hands wet with them, you'll find a great many uses. Some of which may solve issues that you've run into with previous development projects.

For instance if you want to build an SSIS package that processes data in multiple steps, temporary versions of the data can be stored in raw files and passed between data flows. This will allow packages to be built such that they can be restarted at different points in the data processing.

Similar to the previous example, raw files can be used to exchange data between SSIS packages. You can develop one package to convert a client's file to a standardized structure in the raw file format. And then use a couple other packages to read that format in the next packages that process the data in a standard format. All of this can be done without hitting the database.

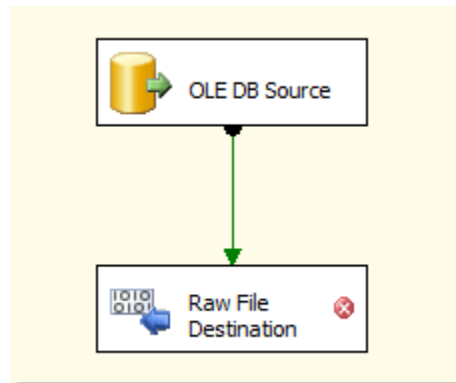
Another thing you can do is export data from a database to a raw file to use as a reference file for multiple SSIS packages. If you need to cycle through a group of IDs or states multiple times, store the data in a raw file and use it as a data cache. Sure you can use a Lookup Cache instead – but that doesn't fit all scenarios and not everyone is using SQL Server 2008.

Let's move onto a couple demonstrations

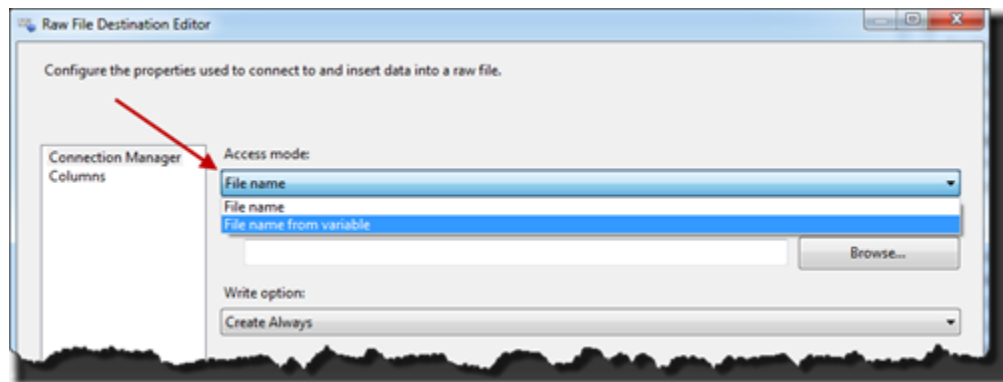
RAW FILE FORMAT DESTINATION

First off, let's build an SSIS package that uses a Raw File as destination. We'll fast forward through adding the Data Flow and the OLE DB Source. The query for the source doesn't matter for this demonstration so we'll skip over that too.

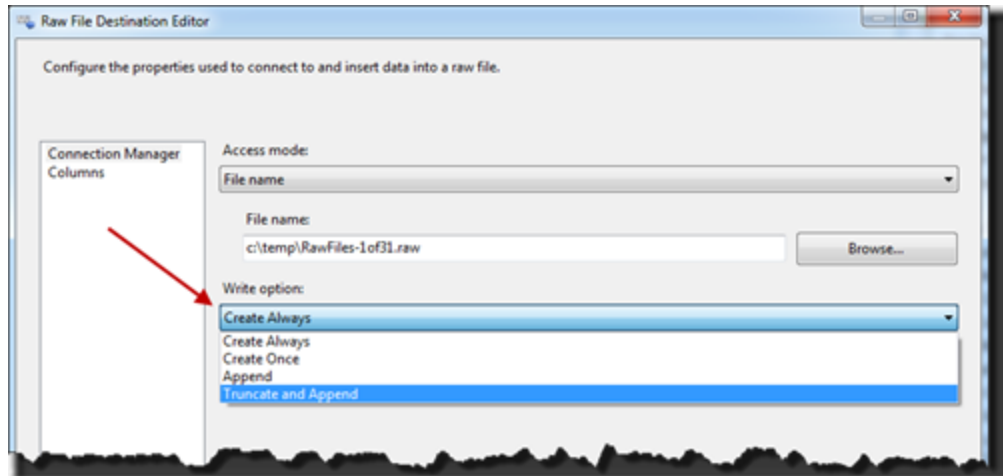
Let's begin. Start by dragging a Raw File Destination onto the the Data Flow and connecting the OLE DB Source to it. The package should look like this:



From here, open the Raw File Destination so that we can configure it. The first thing you will need to configure is the location for the raw file. This can either be a static value or tied to a variable. In most cases I use variables to improve configurability between environments



Next you'll configure how you want the Raw File Destination to behave when it writes. The write option selection is important since it will have significant impact on how the destination is populated

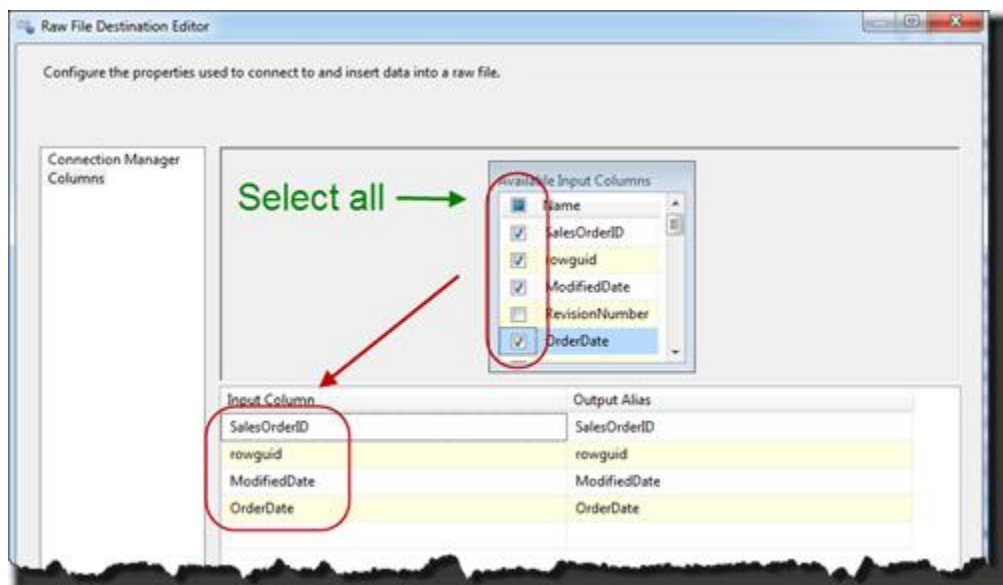


There are a few options for this, they are (definitions from Books Online):

- **Append:** Appends data to an existing file. The metadata of the appended data must match the file format.
- **Create always:** Always creates a new file.
- **Create once:** Creates a new file. If the file exists, the component fails.
- **Truncate and append:** Truncates an existing file and then writes the data to the file. The metadata of the appended data must match the file format.

Missing from this list is a Create First Time or Create Metadata option. As you begin to use the file format more, you'll realize that this is an important option to have. Please, help out by [voting for a metadata option for raw files](#).

At this point, you can move to the next screen and select the columns you need. There is a select all checkbox for SQL Server 2008 and forward, which is quite handy. For SQL Server 2005, you need to select all of the columns individually.

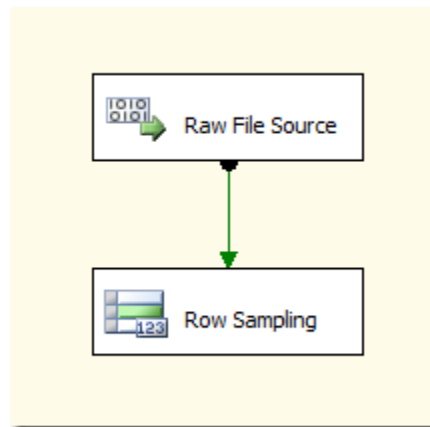


One extremely nice thing about the Raw File Destination is that the order of the columns in it don't matter. Whether you add a column now or in six months it's position in the raw data file doesn't matter. Or if you forget to add a column that data source expects and it is setup for the source as the first column.

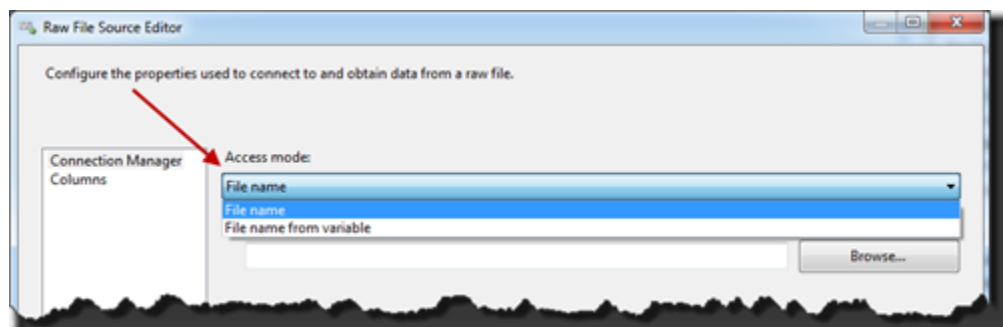
Execute the package and you will have a Raw File created.

RAW FILE FORMAT SOURCE

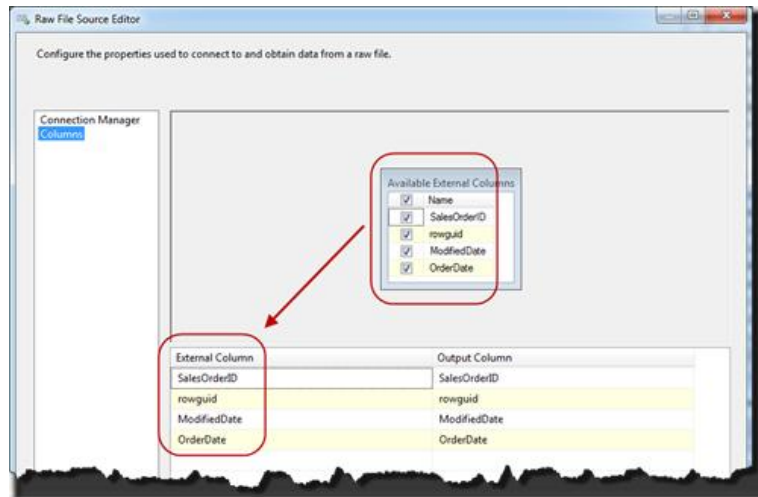
Now that we have a file to work with, let's create a quick package that can import some data from a raw file. As before, we'll skip going through adding the Data Flow and then Raw File Source and Row Sampling task. The Row Sampling task is just there to provide a place to add a data viewer if you desire to see the data that is returned.



Configuration for the the Raw File Source is similar to the Raw File Destination. Open up the data source and first you determine the location for the raw file. Same options as above with either be a static value or tied to a variable.



Next you'll select the columns that you want returned. They can be selected through the checkboxes or by typing their names in the grid below. The column names can also be changed at this point in the Output Column column.

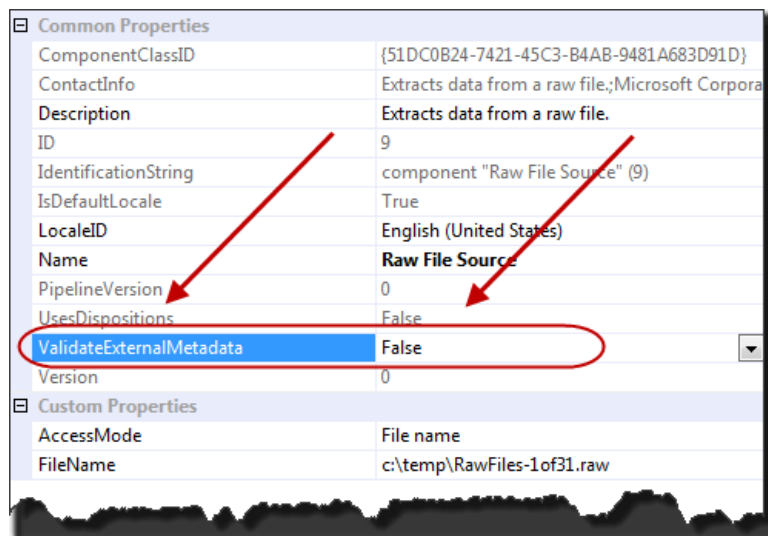


At this point the Raw File Data Source is configured. It knows the underlying data so there isn't any need to call up and discuss this with the person that created the raw file, if it isn't you. The steps are simple and not likely to get you bogged down in details – since there really aren't any.

RAW FILE WRAP-UP

Now that we've got through this far, I hope that you see some of the value is using the raw files. I've got a couple upcoming posts in this series that will go deeper into their actual use, but for now this should get you started.

One bit of advice that I'll leave you with. After the data types for your data source or data destination are set it is wise to change the `ValidateExternalMetadata` property to `False`. This prevents some pesky data type and file availability issues that can come up during validation and helps prevent accidentally changing data types.



REFERENCES

[Using RAW Files for Staging Data in SSIS](#)

[How to Read Raw Files in SSIS](#)

USING BREAKPOINTS

As I mentioned in the [introductory post](#), I'm writing [31 Days of SSIS](#). Today's post is going to deal with one of the pieces within SSIS that is often overlooked with troubleshooting SSIS packages. This is the ability to set [Breakpoints](#) in SSIS packages.

Much like running projects in other languages through Visual Studio, you can also stop SSIS packages mid-execution when specific events occur in the Control Flow. This ability allows you to stop the execution of a package to investigate and troubleshoot the state of the SSIS package.

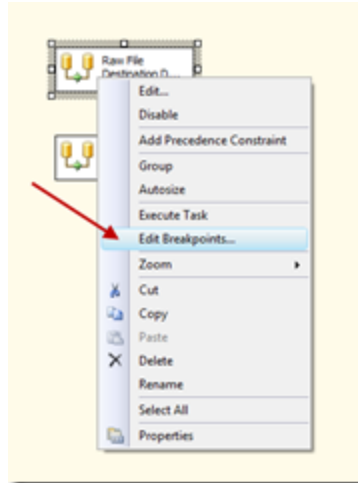
There are ten events in the Control Flow (definitions from Books Online):

- **OnPreExecute:** Called when a task is about to execute. This event is raised by a task or a container immediately before it runs.
- **OnPostExecute:** Called immediately after the execution logic of the task finishes. This event is raised by a task or container immediately after it runs.
- **OnError:** Called by a task or container when an error occurs.
- **OnWarning:** Called when the task is in a state that does not justify an error, but does warrant a warning.
- **OnInformation:** Called when the task is required to provide information.
- **OnTaskFailed:** Called by the task host when it fails.
- **OnProgress:** Called to update progress about task execution.
- **OnQueryCancel:** Called at any time in task processing when you can cancel execution.
- **OnVariableValueChanged:** Called by the Integration Services runtime when the value of a variable changes. The RaiseChangeEvent of the variable must be set to true to raise this event.
- **OnCustomEvent:** Called by tasks to raise custom task-defined events.

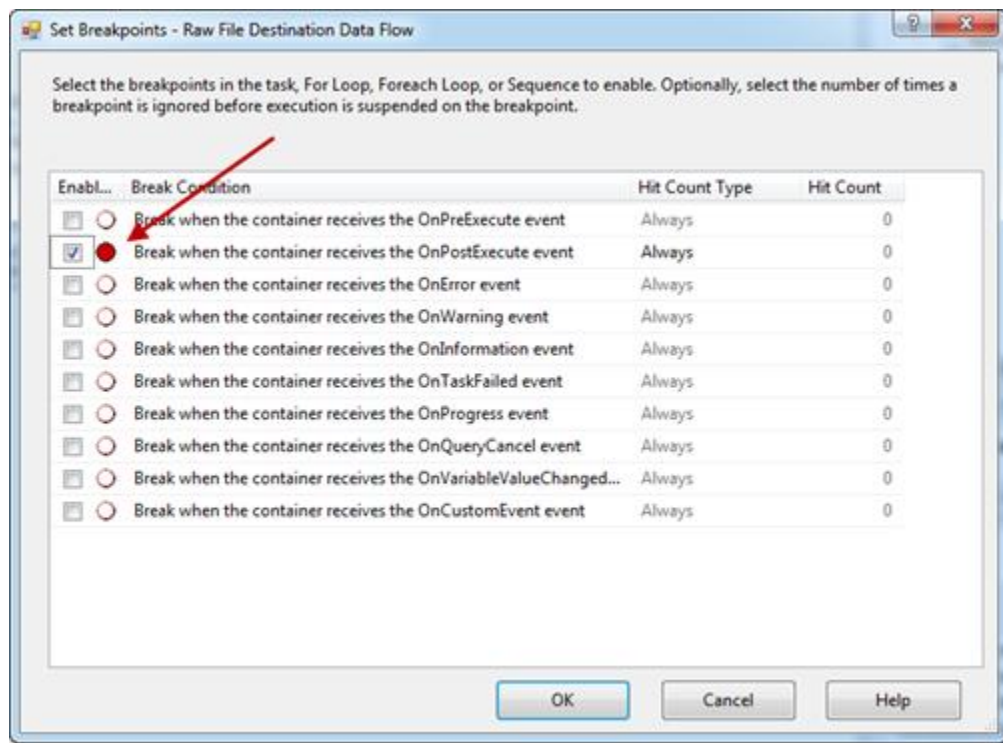
I've mentioned this twice now, and thought it should be called out yet again. Breakpoints function only at the Control Flow. If you need to break the action in a Data Flow, this isn't the solution for that. ~~This is called foreshadowing.~~

BREAKPOINT DEMONSTRATION

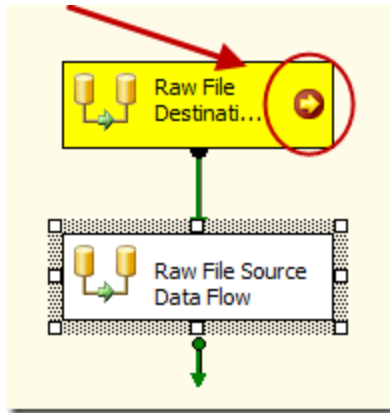
To implement a breakpoint in the Control Flow, start by right-clicking on a task in the Control Flow. In the menu, the option to Edit Breakpoints will be about half way down this list.



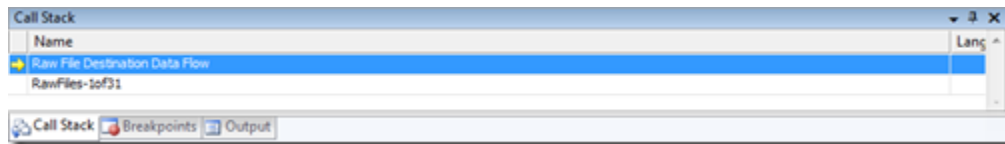
Select this menu item and you can choose between any of the breakpoints described above. For this example, we'll select the OnPostExecute breakpoint. As mentioned above, this event will fire after the task's execution has completed.



Let's run the package now to see what happens. First you'll notice that the the task has a red rot added to it, this indicates that there is a breakpoint on the task. Next when the breakpoint is reached, a little yellow arrow is added to the breakpoint icon.



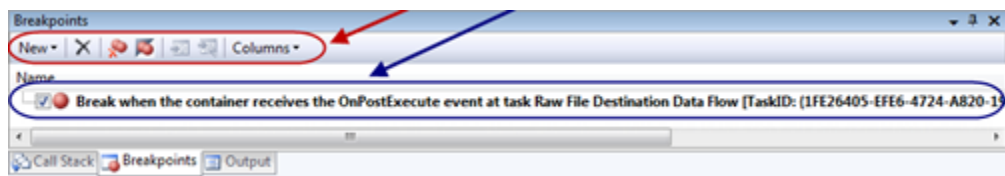
Once a breakpoint has been used there are a number of things that can be used to get some information on the breakpoint. To start with there is the Call Stack window. This window provides information on where the breakpoint is located.



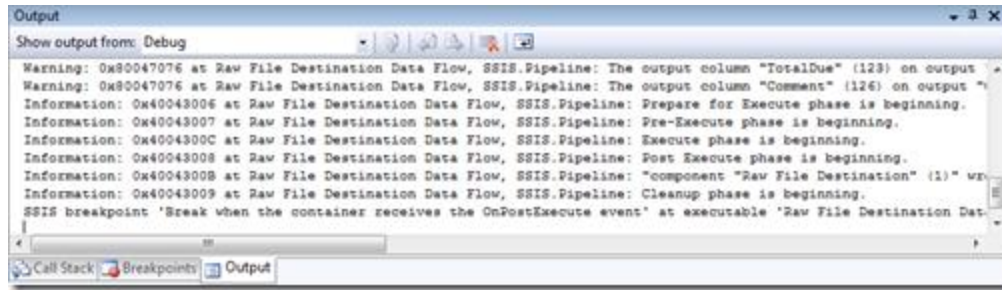
Next there is the Breakpoints window. Guess what it has? Yes, all of the breakpoints in your project will be listed here. There are couple points of interest and points of usefulness with this window.

First, the area in blue is all of the breakpoints in your package. From this point you can enable and disable them. A very useful feature in case you have many breakpoints in your package.

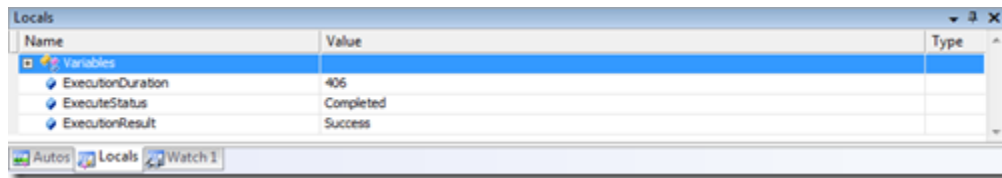
Second, the area in red is a control panel for your breakpoints. There are options to delete single breakpoints, delete all breakpoints in a project, disable all breakpoints in the project, and change the columns returned. There is additional functionality that can be used as well in advanced scenarios.



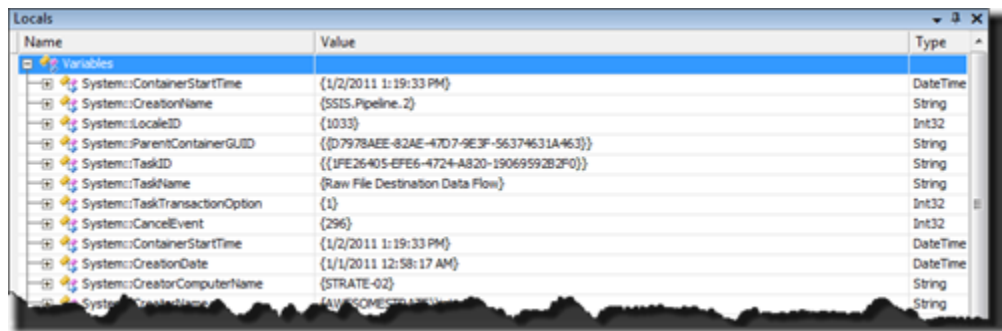
There is also an Output window that displays messaged on the progress of the package and breakpoint. This information will help determine what has been run in the package when the breaks occur.



Finally, the most important piece that breakpoints bring to you is the Locals window. This window will contain information on the current status of the package execution and the amount of time in which the package has been executing.



Most importantly, it contains all of the package variables. Often when I need to troubleshoot some odd package behavior it is related to variables not being populated as I thought they should. This is the golden ticket to finding out what the package is doing during execution.



BREAKPOINT WRAP-UP

Breakpoints are an essential tool in troubleshooting SSIS packages. Take time to understand how they work and practice using them a few times so that when the time comes you are ready to pull them out. They've gotten me through a few package executions and will probably save the day for you some time in the future.

This post scratches the surface on using breakpoints. There is a lot more that can be covered and other scenarios for their use. But those will have to wait for another day. Maybe one of the other 29 more posts in this series.

REFERENCES

[Breakpoints in SSIS](#)

[Breakpoints in SQL Server 2005 Integration Services SSIS](#)

DATA FLOW BREAKPOINTS?!

As I mentioned in the [introductory post](#), I'm writing [31 Days of SSIS](#). In yesterday's post I talked about [Using Breakpoints](#). One of the points I stressed was that breakpoints only exist in the Control Flow and not in the Data Flow. Today, I'm going to show you that I'm a liar. ~~But not really.~~

A specific breakpoint feature does not exist in the Data Flow. A feature that behaves very much like a breakpoint, though, does exist. It isn't called a breakpoint. Instead it is called Data Viewers.

DATA VIEWERS DEFINITION

Data Viewers are containers that can contain a single SSIS buffer's work. As the data passes through the Data Path you are able to view what is in each buffer. As each buffer passes into the Data Viewer, the processing of the SSIS package on that Data Path is paused. In essence, you have a breakpoint. ~~But not really.~~

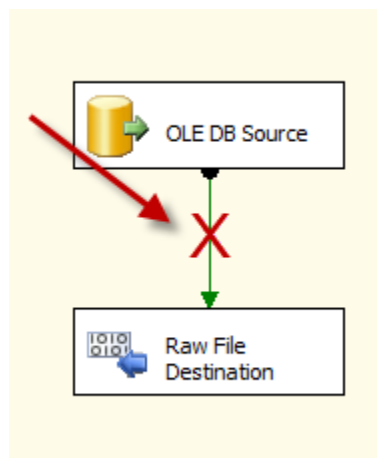
There are four types of data viewers. These are:

- **Grid:** Displays selected columns display in a tabular format.
- **Histogram:** Displays distribution of numeric values and works only with numeric data.
- **Scatter plot:** Displays scatter plot graph works only with numeric data. You are able to set the x-axis and y-axis.
- **Chart format:** Displays a column chart of the count of discrete values in the selected column.

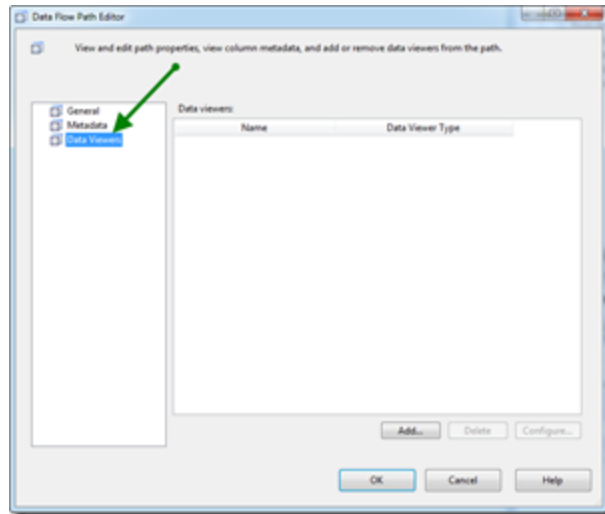
To be perfectly honest, I've only ever found a use for the Grid Data Viewer. There may be situations that I'm unaware of that the others are of great benefit. Unfortunately, I've never come across those.

DATA VIEWER SETUP

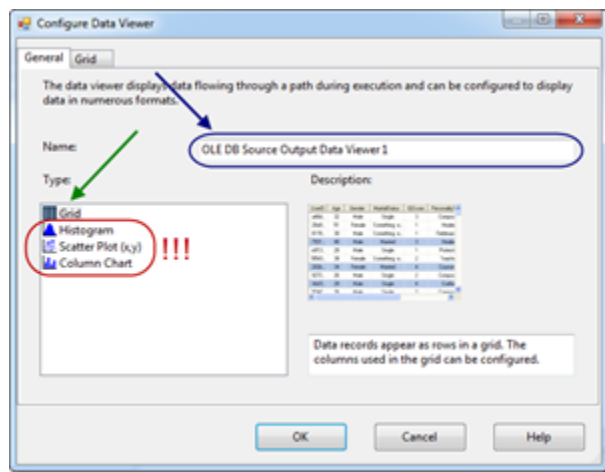
Now setting up a Data Viewer on a Data Path is fairly simple. First, go into the Data Flow and select the Data Path that you want to pause the data processing. You can right-click to open the properties or double click on the Data Path. Just like make sure your cursor is on the red "X" below.



Once you have that, the Data Flow Path Editor will open. From this screen you can tab into the Data Viewers tab to get the options for Data Viewers.

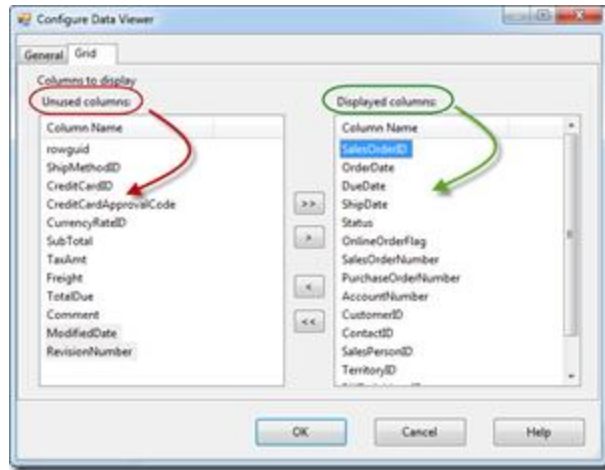


The types of Data Viewers will appear on the left-hand side of the screen. It's shown below at the green arrow. All four of the types from above will be listed. As I mentioned, the only one I use is the Grid. The other three have never been of use to me. I've looked at them a few times and just thought [they were pretty](#).

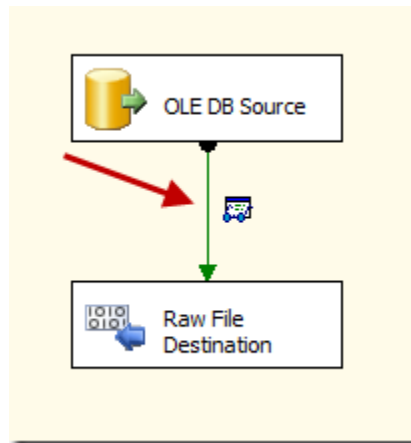


Before moving on, it's advisable if you are putting these into your SSIS package to name them with something meaningful. The blue arrow and bubble above indicate the place where the name can be typed

Now this next step is optional but I do recommend that you consider this. When you create a Data Viewer Grid you have the option to remove some columns from the grid. This can be extremely handy when the Data Path has more than a handful of columns. Since you are likely adding the Data Viewer to look for specific values, you may as well only return the values that are of use.



When you are finished, select OK and you'll return to the Data Flow. From here you will notice that a little icon has been added to the Data Flow Path to indicate that there is one or more Data Viewers on the path.



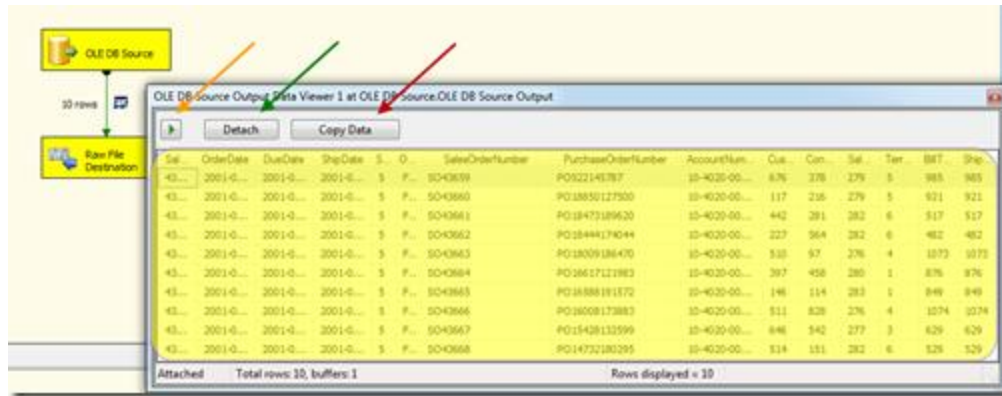
Yes, that's correct. You can have more than one Data Viewer on the same Data Flow Path. Depending on what you are troubleshooting it may be worthwhile have two or three Data Viewers that return different columns.

DATA VIEWER IN USE

Once you have a Data Viewer in place on a package, go ahead and fire it up. The execution of the package will change when there are Data Viewers.

When the package begins, a container will open for each Data Viewer. By default, these will float over the window. If you choose, you can dock these, that is up to you.

On the container there are a few things to take notice of. First, in the image below the yellow area is all of the data currently contained in the Data Viewer. Within it you can highlight one row or multiple rows.

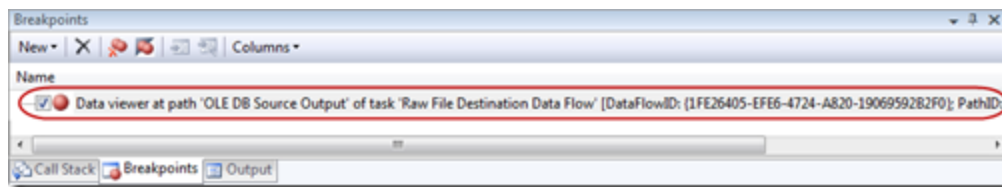


If you want to copy out the data, you need to use the Copy Data button, indicated at the red arrow. The use of Ctrl-C to copy doesn't do a dang thing. ~~And that sucks.~~

If the data you are looking for isn't in the current buffer, you can move the package forward to the next buffer. You do this with the Play button located at the orange arrow. Selecting this will clear the Data Viewer and it will populate with the next buffer of data.

Lastly, if you are done with the Data Viewer and want to stop using it you can detach it from the Data Flow Path. This is done at the green arrow. By detaching, the Data Viewer retains it's current data while the package continues with processing the rest of the Data Flow.

As I mentioned, Data Viewers are a lot like Breakpoints. Due to this, you can use the Breakpoint window to disable, enable, and delete data viewers in your project. This can be extremely handy when you have a half dozen or more data viewers you just want to get rid of.



DATA VIEWERS WRAP-UP

Now that we've gone through this, I hope you can see the power and benefit of using Data Viewers. When you are troubleshooting an SSIS package trying to figure out where some of the data is becoming out of wack.

Above, I mentioned a couple bits where I compared Breakpoints to Data Viewers. In a lot of regards, they function similar to one another. There is one way, though, in which they are very different. When a breakpoint is encountered, the entire package execution is paused until the break is cleared. When a Data Viewer is encountered only the data in that Data Flow Path is paused. While these can be in effect the same thing, in more complicated packages this might cause unexpected results.

Finally, you may have trouble using Data Viewers when you are processing large volumes of data. If the row that is causing the issue is a few thousand or million rows into the data set you will likely want to do a little bit more than just add a Data Viewer. You may need filter the data some to make the Data Viewer useful for troubleshooting.

Thanks for reading and if you have any questions, feel free to leave a comment. Also, since I still have 29 posts to go, feel free to ask for items on specific topics.

FOLDER STRUCTURES

Hopefully by now you've read the [introductory post](#) for the [31 Days of SSIS series](#) that I am writing. Today we will talk about something completely fuzzy and outside of the development environment. We're talking about folder structures! Wahoo!

I have often needed to consult with clients on how to setup the SSIS environment. For some clients, they are just getting into SSIS and haven't had the opportunity to consider how to setup their environment. Others have been at it a while and haven't stopped to think out how they want the environment to function.

Today we will be going through some of the things that I do to setup the SSIS environment. These will all have a focus outside of SSIS packages and the service. They will all deal with folder structures. These folders are where the SSIS packages, configuration files, and files that are imported and exported are stored.

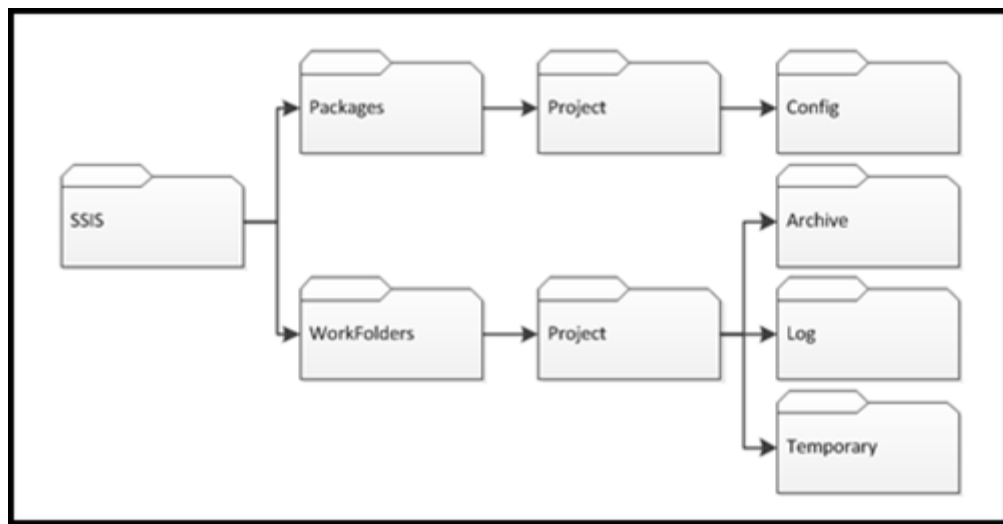
THE FOLDER STRUCTURE

The SSIS folder structure is usually contained under a single root directory. For simplicity, I often name this folder **SSIS**. Yeah, I'm creative. Off of the root folder, I'll add folders for **Packages** and **WorkFiles**.

The **Packages** folder often contains many folders underneath it. These folders are for all of the SSIS projects that have been deployed. For every project there will be a Project folder. The **Project** folder may have other folder beneath them, such as a **Configurations** folder.

The **WorkFiles** folder will be similar to the **Packages** folder in the sense that there will be **Project** folders below it. The **Project** folders will have other folders such as **Archive**, **Logs**, **Response**, or **Temporary**.

If you are good at visualizing, the folder structure will look like this diagram:



THE FOLDER PURPOSE

Now that this has been described, less put some definition and understanding behind these folders.

- **Packages:** Everything under the **Packages** folder is considered “code”. Because of this status these folders are under change management control. People who are not the gatekeepers should only be able to browse these folders, if that. These folders should be backed up on a regular basis.
- **Project (under Packages):** These folders contain all of the SSIS package files (dtsx) and configuration files (dtsconfig) for each deployed project. A project could be a single SSIS project or a collections of projects based on a client’s business. Typically, I store SSIS packages outside of the MSDB database in the file system. This has worked best for change management and with security in the environments.
- **WorkFiles:** Everything under the **WorkFiles** folder is considered temporary. These folders will store files to be imported, exported, or log files. Business users are often granted access to these folders depending on their projects and whether they need to pick-up, drop-off, or review files.
- **Project (under WorkFiles):** These folders contain project, same project definition from above, related files. They can be logs, data received for import, or data exported.
- **Archive (Workfiles -> Project):** After a file has been imported by an SSIS package, the imported file is dated and placed in this folder.
- **Logs (Workfiles -> Project):** Log files generated from the SSIS package and the [SQL Agent job steps](#) are placed here. These need to be in a place where more than those running the jobs can find them.
- **Response (Workfiles -> Project):** When the SSIS package has data that cannot be imported and is rejected for some reason – that data will go in this folder.
- **Temporary (Workfiles -> Project):** This folder stores the [raw files](#) that are generated during an SSIS package execution. This keeps them out of the way and is an easy place to just delete everything in the unlikely scenario that there are space constraints.

FOLDER STRUCTURE WRAP-UP

There are countless other folders that you could add, but this should cover a lot of the basics. These have been most of the basics that I try to use.

This has been about how I do things. The tips and tricks I’ve learned to keep things going and to reduce headaches. These aren’t necessarily the right answers, but they are my answers.

My question to you is – do you agree? What do you do differently?

ONE VARIABLE TO RULE THEM ALL

Hopefully by now you've read the [introductory post](#) for the [31 Days of SSIS series](#) that I am writing. Today we are going to look at [variables in SSIS packages](#).

This post assumes that you already know a bit about variables in SSIS packages. If there is a groundswell of requests, I can do a post on variables themselves. So ask if you want to see it.

Time to begin...

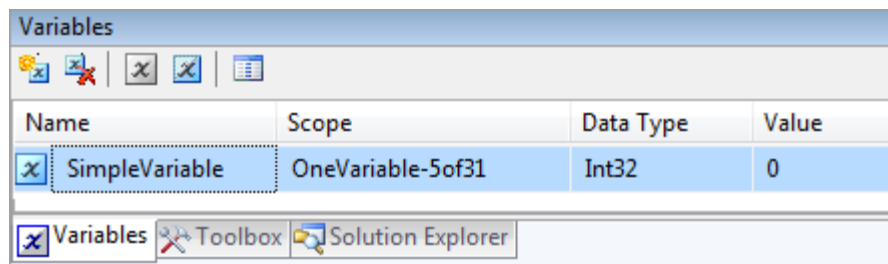
VARIABLE EXPRESSIONS

Back in the DTS days, in order to change the value of a variable we need to write VBScripts. These scripts were at best irritating to write and were wrought with opportunities for misspellings and unrunnable code.

Enter stage right with SSIS and its variables, which can be evaluated as expressions. This was a vast improvement over DTS and all of the VBScript that we had in the past was chuckled.

But how exactly do we evaluate variables as expressions. First, what does it mean to do this? When a variable is evaluated as an expression, the value for the variable is calculated at run-time based on SSIS package properties, system properties, and other variables to determine a value for the variable.

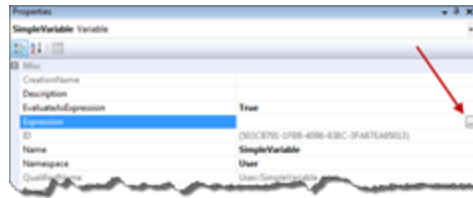
From here though, let's see how this works with our own variable. To start create a package and open up the Variables window. In this window create a new variable named **SimpleVariable**.



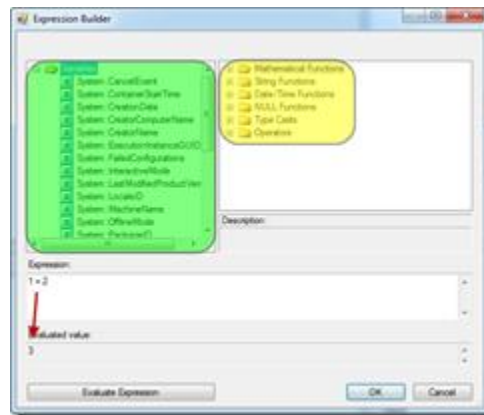
Once the variable is created bring up the Properties window for the variable. There will be a number of things to choose from in the properties. The item we are concerned with is **EvaluateAsExpression**. Follow the red arrow below if cannot find the property. Change this value to True.



When you change the **EvaluateAsExpression** property to True, an ellipse will be enabled (at the red arrow below) that will allow you to open the expression builder for the variable.



Click on the ellipse and the Expression Builder window will appear. This is pictured below. There are a couple areas to pay attention to when building expressions. First, the area in green below contains all of the system and package variables that are available. If you are unsure on the name of a variable just browse through this list to retrieve one. Next, in the yellow area, has a list of all of the operations that are possible in expressions. These can be a bit tricky to get used to at first and I'll dedicate a later post to some of their nuances.



For our expression at the moment, type in "1 + 2". This is written in the Expression box above. Selecting the Evaluate Expression button will return the result of 3.

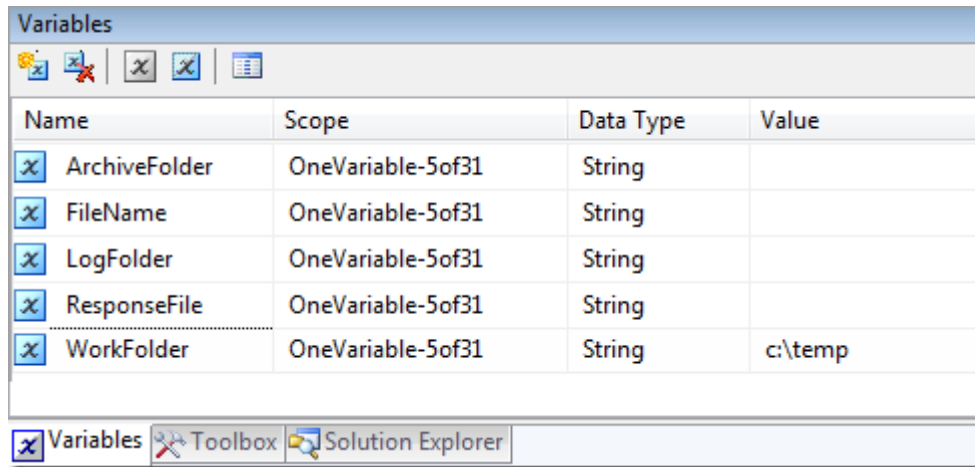
This was a simple explanation of how to build an expression for a variable. Now onto something much more useful.

THE ONE VARIABLE

Yesterday's post focused on the [configuration of the SSIS environment from a folder perspective](#). The reason for that post was because I wanted to talk today about how I will often use one variable in a package as a fulcrum for setting many other variables and configuring other portions of the SSIS package.

In the [Folder Structure post](#), I talked about the WorkFiles folder. Often I include a variable that has this value which I use to set the values for files and folders that the package must interact with. The rest of this post will demonstrate this practice.

In an SSIS package, create the following variables; WorkFolder, ArchiveFolder, FileName, LogFolder, and ResponseFile. Set the value for the WorkFolder variable to "c:\temp". At this point your Variables window should look like this:

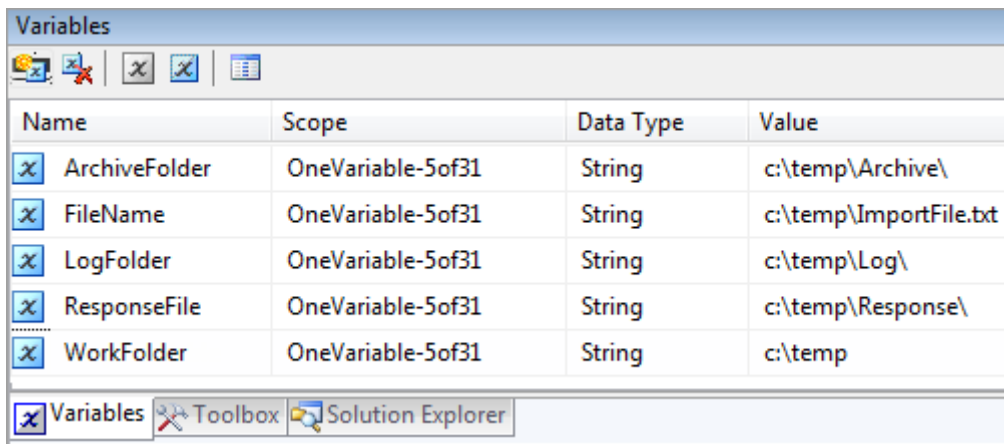


Name	Scope	Data Type	Value
ArchiveFolder	OneVariable-5of31	String	
FileName	OneVariable-5of31	String	
LogFolder	OneVariable-5of31	String	
ResponseFile	OneVariable-5of31	String	
WorkFolder	OneVariable-5of31	String	c:\temp

Now change the **EvaluateAsExpression** property for ArchiveFolder, FileName, LogFolder, and ResponseFile like we did in the previous example. What we want to do now is create expressions for these variables based on the value in WorkFolder. Type in the following expressions for each of the variables:

- **ArchiveFolder:** @[User::WorkFolder] + "\\Archive\\"
- **FileName:** @[User::WorkFolder] + "\\ImportFile.txt"
- **LogFolder:** @[User::WorkFolder] + "\\Log\\"
- **ResponseFile:** @[User::WorkFolder] + "\\Response\\"

Once these are all set, the values for each of the variables should change to the following:



Name	Scope	Data Type	Value
ArchiveFolder	OneVariable-5of31	String	c:\temp\Archive\
FileName	OneVariable-5of31	String	c:\temp\ImportFile.txt
LogFolder	OneVariable-5of31	String	c:\temp\Log\
ResponseFile	OneVariable-5of31	String	c:\temp\Response\
WorkFolder	OneVariable-5of31	String	c:\temp

As you can see, the **WorkFolder** variable now controls the values for each of the other variables. By just changing the value of **WorkFolder** all of the locations for all of the files can be changed.

VARIABLE WRAP-UP

Using a variable or two to control the configuration of your SSIS packages can be relatively simple to do. In the demo above, the one variable reduced the amount of configuration points needed for the package. Instead of setting all of the values individually, they are set by the one variable.

This can help reduce configuration changes when you are moving your SSIS packages from environment to environment. It can also reduce the amount of work required when troubleshooting a package since only a few variable will **vary** between environments.

Now that I've laid out some ground work, be prepared for a few packages in the next few days that I've used to solve business problems.

REFERENCES

- [Getting Started with Variables in SSIS](#)

RECURSIVE SSIS PACKAGE

As I mentioned in the [introductory post](#), I'm writing [31 Days of SSIS](#). In the first post I talked about the [raw file format for sources and destinations](#).

Today, I am going to discuss a solution that I implemented that use raw files for performing a recursive operation within an SSIS package to build an employee hierarchy. Before I get to that solution, we will review the requirements that I faced with this SSIS package.

PACKAGE REQUIREMENTS

The package needed to import a number of people into an employee table. Like most employee tables, this table had references on each employee to their manager. Since the managers weren't already in the database I was going to be loading them in the same import that their employees were being imported. Since I needed to reference each employee's manager, the managers needed to be loaded before their employees.

One of the restrictions for the package was that I couldn't write to the destination database or server prior to completing the data ordering. This might seem like a silly requirement. The goal though was to have the data completely prepared for loading prior to hitting the production server.

RECURSIVE SOLUTION

While designing the package it occurred to me that this was a classic example of recursion. There was a root level with some people related to it. And then people related to those people and so on. If I loaded each level of the hierarchy before the lower level, the employees could be associated with their managers as they were loaded into the database.

In envisioning the solution, I considered the same types of operations that would occur when doing recursion through common tables expressions. In this scenario, the root level items are identified within dataset. We'll call this Level 0.

After Level 0 is selected, it is joined back to the data to find out what data is related to it. This new group of people would be those that report to people in Level 0. This new group will be Level 1.

Then Level 1 is given the same treatment as Level 0 to find the data that relates to Level 1. The new set of data is then Level 2. This continues until there is no more data to join together and the process ends.

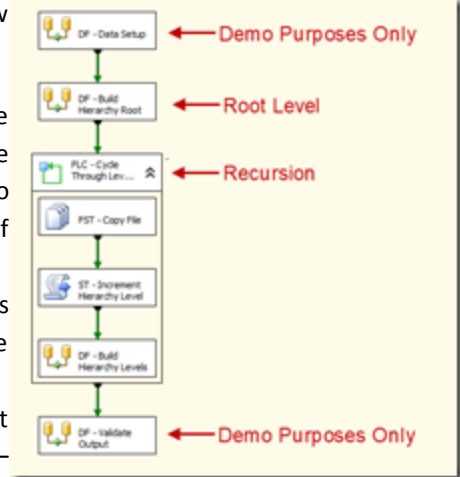
This is what we need to do in as SSIS package. But we are going to do it within the flow without accessing a database. And we'll be using something awesome... [Raw Files](#).

PACKAGE CONTROL FLOW

Now that we've defined the solution that is needed let's consider what is needed from a Control Flow perspective. First, we'll need to identify the rows that comprise the root level. From there we need to compare that level and subsequent levels to the source data to build the new levels. When there is no more data to process the package should complete.

This logic can be implemented with the following Control Flow components:

- **DF – Data Setup:** This task is included to create a flat file that can be used as the source for the process. Since the original package didn't touch any databases, I wanted to maintain that spirit as well as possible. This would be deleted if you opt to use this package.
- **DF – Build Hierarchy Root:** Data Flow task that selects the first level from the data to create the root level of the employee hierarchy.
- **FLC – Cycle Through Levels:** A For Loop container that loops across the next tasks as long as the row count from DF – Build Hierarchy Levels does not return 0.
- **FST – Copy File:** File System Copy task that copies the file created by the data flow tasks to create a source raw file for the DF – Validate Output.
- **ST – Increment Hierarchy Level:** Script task that increments the HierarchyLevel variable by 1 for every cycle in the loop.
- **DF – Build Hierarchy Levels:** Data Flow task that adds each of the non-root levels the employee hierarchy.
- **DF – Validate Output:** This step is only included so that we can see the final output without having to open up any files. This would be deleted if you opt to use this package.



PACKAGE VARIABLES

There are a number of variables that I added to the package to help control how everything processed.

Name	Scope	Data Type	Value
HierarchyLevel	Hierarchy-6of31	Int32	0
RawFileHierarchy	Hierarchy-6of31	String	C:\temp\Employee.raw
RawFileSource	Hierarchy-6of31	String	C:\temp\Source.raw
RowCount	Hierarchy-6of31	Int32	-1
WorkFolder	Hierarchy-6of31	String	C:\temp\

The variables from above were added for the following purposes:

- **HierarchyLevel:** The current level of the hierarchy. Default value is 0.
- **RawFileHierarchy:** Location of the raw file that is the destination for each level of the hierarchy output.
- **RawFileSource:** Location of the raw file that contains all previous levels of the hierarchy.
- **RowCount:** Count of the number of rows last added to the hierarchy. Default value is -1.
- **WorkFolder:** This is my [one variable to rule them all](#) that sets the RawFileHierarchy and RawFileSource path.

PACKAGE DATA FLOWS

Now we'll go into the core data flows of the package. I'm going to ignore the Data Flows that are just there because this is a demo. They aren't important to the core of this package.

DF – BUILD HIERARCHY ROOT

This is the first data flow of SSIS package the implements the solution. It will build the first level of the hierarchy. With the data that is being used for this package anyone without a manager is a root level person. Those that match that criteria are sent to the raw file destination.

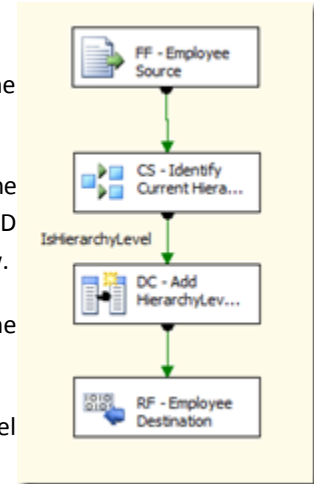
Let's review the individual tasks to see what's going on:

FF – Employee Source: Flat File Source that extracts unsorted rows for the package.

CS – Identify Current Hierarchy Level: Conditional Split task that tests for the rules the identify the root level. In this case that logic was whenever the ManagerID was NULL. Only the rows that meet the condition are passed forward in the data flow.

DC – Add HierarchyLevel Column: A Derived Column task that adds the HierarchyLevel variable in as a column in the data flow.

RF – Employee Destination: Raw File Destination that receives all of root level rows.

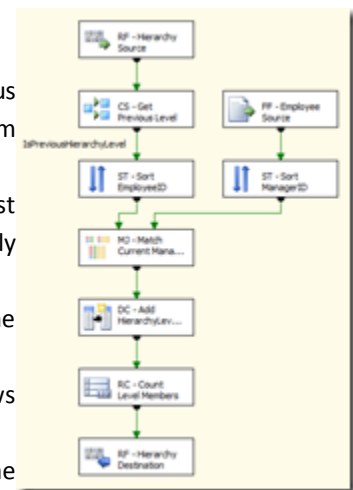


DF – BUILD HIERARCHY LEVELS

Once the root level has been created, it can be used to find the subsequent levels and that logic is implemented in the DF – Hierarchy Levels Data Flow. Each level is passed into this data flow and checked to see if there are any employees in the source data (flat file) that report to this level. Those that match are sent to the raw file destination.

The tasks for this data flow are:

- **RF – Hierarchy Source:** Raw File Source that is a copy of the previous levels Raw File Destination. This contains all levels of the hierarchy from the root to the most recent level added.
- **CS – Get Previous Level:** Conditional Split that returns the most recent level of the hierarchy based on the HierarchyLevel column. Only the rows that meet the condition are passed forward in the data flow.
- **ST – Sort EmployeeID:** Sort that orders the employees from the previous hierarchy level by their employee ID.
- **FF – Employee Source:** Flat File Source that extracts unsorted rows for the package.
- **ST – Sort ManagerID:** Sort that orders the employees from the source file by their manager ID.
- **MJ – Match Current Managers:** Merge Join that joins the employees from the previous hierarchy level with those in the source level. The output from this operation is the employees that are in the current hierarchy level. This is an INNER JOIN operation.



- **DC – Add HierarchyLevel Column:** A Derived Column task that adds the HierarchyLevel variable in as a column in the data flow.
- **RC – Count Level Members:** Row Count task to count the number of employees found at the current hierarchy level. When this sets the variable to 0 the For Loop Container will terminate.
- **RF – Hierarchy Destination:** Raw File Destination that receives all of root level rows.

PACKAGE OUTPUT

If you've followed along (or more likely downloaded the package), you should be able to fire it off and get the results below. Each level has been color coded and shows that in the output the rows are sorted by hierarchy level.

Emp	Ma	Hier
109	0	0
140	109	1
42	109	1
12	109	1
6	109	1
273	109	1
148	109	1
272	6	2
119	6	2
303	6	2
218	148	2
268	273	2
388	273	2
384	273	2
9	3	3
283	3	3
287	3	3
11	3	3
270	3	3
158	3	3
4	3	3
508	21	3

RECURSIVE WRAP-UP

Hopefully the detail that I've provided above is sufficient to describe the solution. If more detail is needed, feel free to leave some comments. I've been pretty happy with the solution and what I've been able to do with it. With volumes up to a million rows it's done well with processing data.

While this package didn't place the employees into that database, it wouldn't be hard to branch the data flow and add the employees into a table. However the data needs to be handled can be done while maintaining the ability to segment each level of the hierarchy.

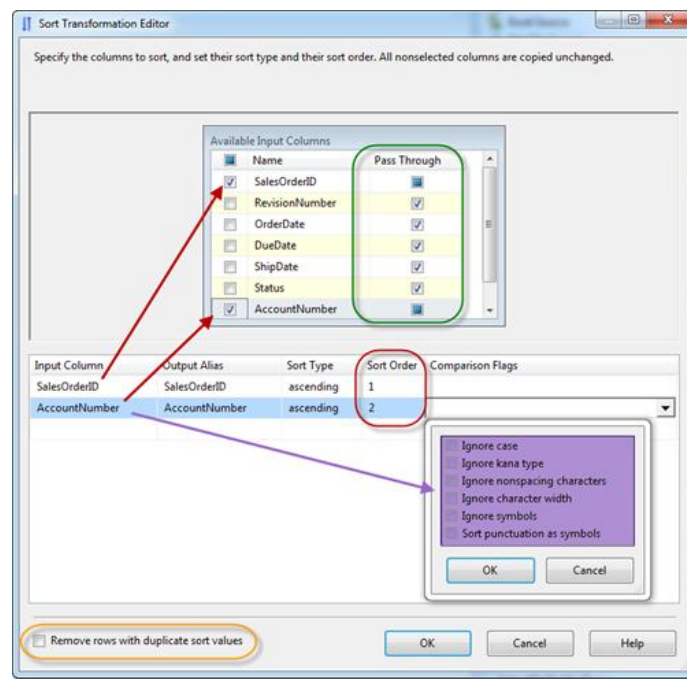
WHAT THE SORTS?!

Welcome to day seventh day of the [31 Days of SSIS](#). If you are just joining the party you may want to read the [introductory post](#). In yesterday's post the SSIS package contained a couple [Sort Transformations](#). Since they've now appeared in a package it seems it would be a good idea to talk about them.

Let's start by calling the kettle black. A Sort Transformation will sort data. There I said it, hopefully that wasn't too confusing. All of the standard sorting mechanisms are there. The data can be sorted ascending or descending. It can also be across one or more columns.

SORT CONFIGURATION

There isn't much to configure in a Sort Transformation after it is placed on an SSIS packages Data Flow. The first thing you do is select the columns for the sort (indicated with the red arrows below). Be certain when they are selected that the order of the sort is chose (red box below). After selecting the columns, pick if there are any additional columns that you want to return in the sort (green box).



STRING COMPARISONS

If you are sorting string values you may need to take an additional step. Since localization and languages can require differences in how data is sorted there are options to implement these rules against the Sort Transformation.

We see this often with the collation of a database. Within SSIS these variations need to be accounted for. This can be done using the comparison flags (purple region above). There are six options for changing how data is compared. They are:

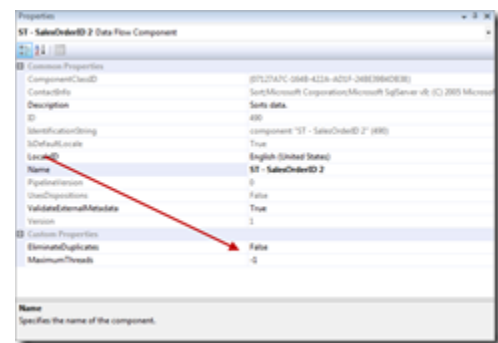
- **Ignore case:** Determines whether uppercase and lowercase letters are considered to be the same value.

- **Ignore kana type:** Determines whether kana type from the Japanese characters are ignored.
- **Ignore character width:** Determines whether single-byte and double-byte representations of the same character as identical.
- **Ignore nonspacing characters:** Determines whether spacing characters and diacritics are considered identical. For instance, whether “ã” is equal to “a”.
- **Ignore symbols:** Determines whether white-space characters, punctuation, currency symbols, and mathematical symbols are considered in the sort.
- **Sort punctuation as symbols:** Determines whether the comparison sorts all punctuation symbols, except the hyphen and apostrophe, before the alphanumeric characters. For instance, if this option is set, “.ABC” sorts before “ABC”.

ADVANCED SETTING

There are two options for the Sort Transformation that can be considered to be “Advanced”. They are the options for handling duplicate values and the maximum thread property.

Duplicate Values: This setting is in the main property (shown above in the orange box). When this is enabled on a sort operation then only one row for each distinct sort value from the sort columns will be returned. A random row from the unsorted columns will be included for these values.

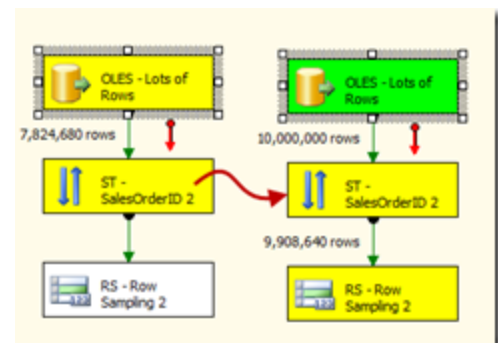


Maximum Threads: This setting is in the property pane for the Sort Transformation (image to the right). This property defaults to -1 which means that the sort will use an unlimited number of threads. This can be changed to any value 1 or higher to control the number of threads that are used.

SORT CONSIDERATIONS

Up to this point, I’ve discussed setting up and configuring the Sort Transformation. From here let’s look a little deeper at the transformation and what happens when it executes.

To start with, when a sort executes it has to load all of the data from the data flow path into the Sort Transformation in order to complete it’s processing. Whether there are one hundred rows or ten million rows – all of the rows have to be consumed by the Sort Transformation before it can return the first row. This potentially places all of the data for the data flow path in memory. And the potentially bit is because if there is enough data it will spill over out of memory.



In the image to the right you can see that until the ten million rows are all received that data after that point in the Data Flow cannot be processed.

This behavior should be expected if you consider what the transformation needs to do. Before the first row can be sent along, the last row needs to be checked to make sure that it is not the first row.

For small and narrow datasets, this is not an issue. But if your dataset are large or wide you can find performance issues with packages that have sorts within them. All of the data load and sorted in memory can be a serious performance hog. I've seen sorts in SSIS packages where there were a million rows being processed and the design required 32 GB of RAM to process the sort.

Speaking of RAM, depending on your server configuration the Sort Transformation can run into memory restrictions. Jamie Thompson ([Blog](#) | [@JamieT](#)) discusses the memory limitations in [this post](#). The 32 GB of RAM needed for the sort in the package mentioned above ran in about 45 minutes. But on a server with only 4 GB of RAM the sort took 38 hours.

SORT WRAP-UP

There are limitations to the use of the Sort Transformation. Because of these limitations, you should consider whether or not sorts are really necessary in SSIS packages.

Are you adding it to process data properly? Or is it there to make the output "feel" better? I've seen cases where data is sorted in an SSIS package because alphabetical orderers "looked nicer".

Alternatively, does the data already arrive sorted? Don't resort data that you already know has a specific sort just to be certain. Make those delivering sorted data responsible for the data arriving in that fashion.

In the end, though, you will need to occasionally use the Sort Transformation. It isn't an evil transformation. It isn't a bad practice to use it. As I pointed out it does have limitations and should be applied sparingly when it is the right fit for the right scenario.

Oh, ooh, oh... it depends. Yup, I said it.

MERGE JOIN IS MY FRIEND

Welcome to day eight post for the [31 Days of SSIS](#). If you are just joining the series you may want to read the [introductory post](#). In another post I talked about the [raw file format for sources and destinations](#).

In this post, I want to discuss a solution I worked on where I used a [Merge Join transformation](#) with Raw Files to join data sets from two sources. This solution is based upon techniques that I've used at a few clients to solve their business needs and to improve the performance of a process that went sideways due to the volume of production data.

PACKAGE REQUIREMENTS

The general idea for this package is that you have two or more datasets that have similar or related data. Unfortunately, this data isn't stored on a platform where the the two datasets can be easily joined together.

One scenario could be where you have a source system with a few million records in it. These records need to be imported into a second system. Since the number of new or changed records is relatively low, you want to trim off the records that the second system wouldn't need for inserts or updates.

Another scenario could be the that you need to import a subset of a large file each day. The records that are imported need to match a second set of data, that has a few thousand records, that has already been loaded into the database. This technique could be used to determine the records that are part of the subset.

There are countless other scenarios that this could be used for. As I walk through the demo package, I hope that it will spark some ideas of areas where you could apply this type of a solution.

MERGE JOIN SOLUTION

This solution can basically be summed up as an application of the [Merge Join transformation](#). So from a simple perspective we are just implementing one of the transformations. When adding this transformation to a package, though, there are a number of considerations to take.

Before the considerations, let's define the Merge Join transformation. This transformation allows data from two data flows to be joined together on specified column(s). The join operation occurs as the datasets are streamed through the transformation and because of this there is only a single pass of the data. This requires that both data flows be sorted on the join column(s). Finally, the data can be joined as either a FULL, LEFT, or INNER join.

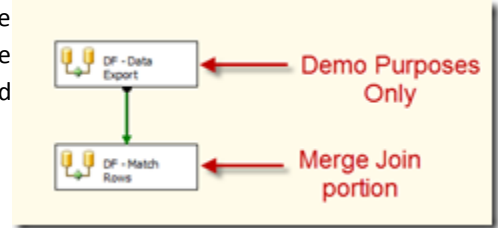
Based on the needs of this transformation, our solution will have two data sources. One of the data sources will be a raw file source. I have opted to use this because in most cases that I use the Merge Join transformation there is a raw file included as part of the solution.

To fulfill the sorting requirement, we'll review two methods for having sorted data within a Data Flow. The first will be by using the [Sort transformation](#). The second will be by modifying the data source such that it knows that the data is sorted. There are times that you will know that data is sorted within the data flow from previous operations. Use this knowledge to prevent the addition of unnecessary Sort transformations.

PACKAGE CONTROL FLOW

With the description above there isn't a lot that will be included in the Package Control Flow. There are two Data flows in the SSIS package for the purposes of having a complete solution. Though in the end only one of them truly has a bearing on the solution.

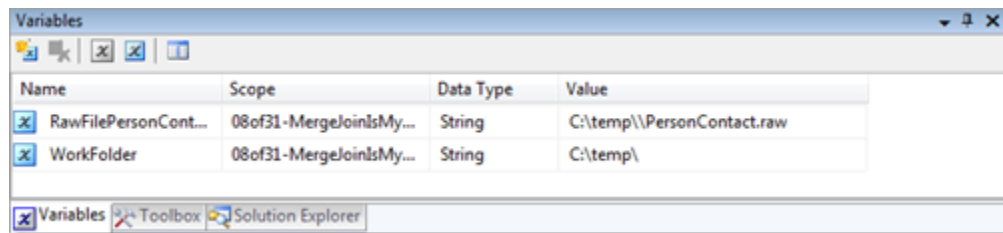
This two Data Flows in the Control Flow are:



- **DF – Data Setup:** Data Flow task included to create a raw file that can be used as the source for the process. This would be deleted if you opt to use this package.
- **DF – Match Rows:** Data Flow task that contains the Merge Join and related transformations. This transformation will join to data sets together similar to querying two table together.

PACKAGE VARIABLES

There are a couple variables that are part of the package to make referencing the raw files easier



The variables from above were added for the following purposes:

- **RawFilePersonContact:** Location of the raw file that is the source and destination for the Person.Contact data
- **WorkFolder:** This is my [one variable to rule them all](#) that sets the RawFileHierarchy and RawFileSource path.

PACKAGE DATA FLOWS

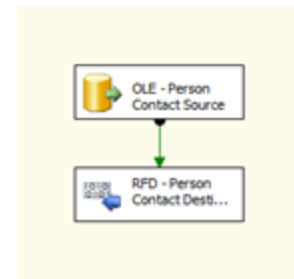
As noted above, there are two Data Flows in this SSIS Package. The Data Flows is where all of the work is happening in this package.

DF – DATA SETUP

This Data Flow task creates the raw file that is being used as source data for the other data flow. If you are unfamiliar with using raw files, I suggest reading "[Raw Files Are Awesome](#)". From a solution perspective, this data flow would likely not be included in a final solution.



The data flow is configured with the following transformations:



OLE – Person Contact Source: OLE DB Source to the AdventureWorks database. The data source retrieves data from Person.Contact. The one item to note with this data source is that

the query against Person.Contact is sorting the data. This will be important to know in the next data flow.

RFD – Person Contact Destination: Raw File Destination that receives all of the records from Person.Contact.

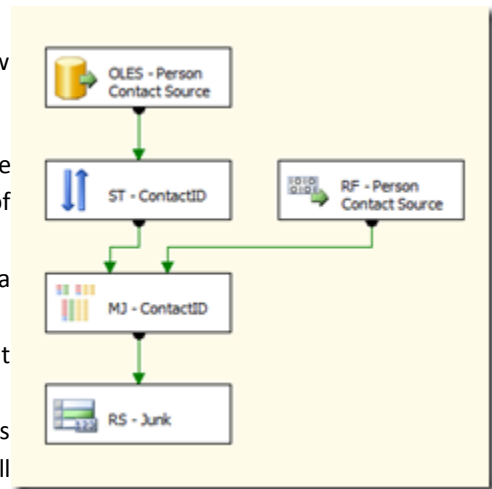
This data flow was pretty bland. But it's only there for this demo and just serves as a model for extracting data to raw files.

DF – MATCH ROWS

The DF – Match Rows Data Flow task is where the real work for the solution occurs. In this data flow, data is taken in through two data sources. One of the data sources is pre-sorted (mentioned in the first data flow) and the data source is configured with this information. The other data source is not sorted and requires a sort within the data flow. Both datasets are joined with a Merge Join transformation to find matching rows.

Putting this all together, we get the following tasks with the data flow shown to the right.

- **OLES – Person Contact Source:** OLE DB Source to the AdventureWorks database. The data source retrieves all of the data from Person.Contact.
- **ST – ContactID:** Sort transformation that sorts the data on the ContactID column.
- **RF – Person Contact Source:** Raw File Source that contains all of the records from Person.Contact.
- **MJ – ContactID:** Merge Join transformation that joins both data flows on the ContactID column. For simplicity all of the columns from both data flows are included.
- **RS – Junk:** A Row Sampling task that is included because it is easy to include and provides a data path from the Merge Join transformation. This is here for attaching [Data Viewers](#) and to see the row counts.

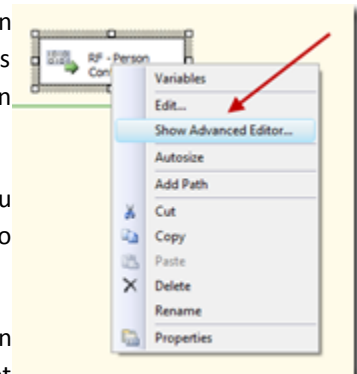


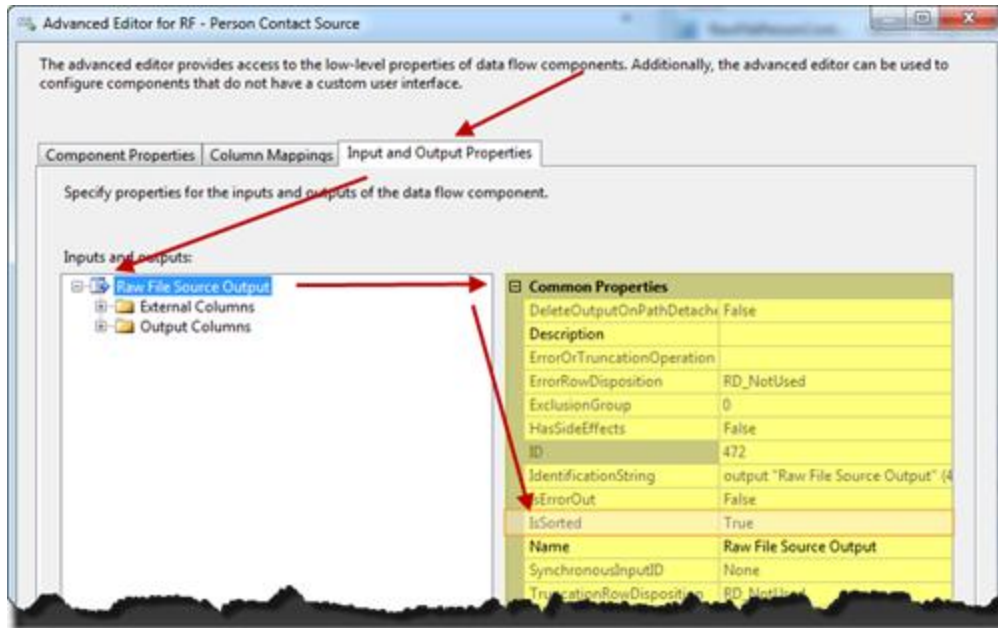
CONFIGURING SOURCE SORT ORDER

As was mentioned, the **RF – Person Contact Source** has been configured in order to allow the SSIS package to know that the data it is retrieving is sorted. Changing the data source to allow this is relatively simple and is done in the same manner on the other data sources.

First, browse to the Show Advanced Editor menu item for the right-click menu on **RF – Person Contact Source**. An example of the right-click menu is to the right. Once this is selected the Advance Editor will open.

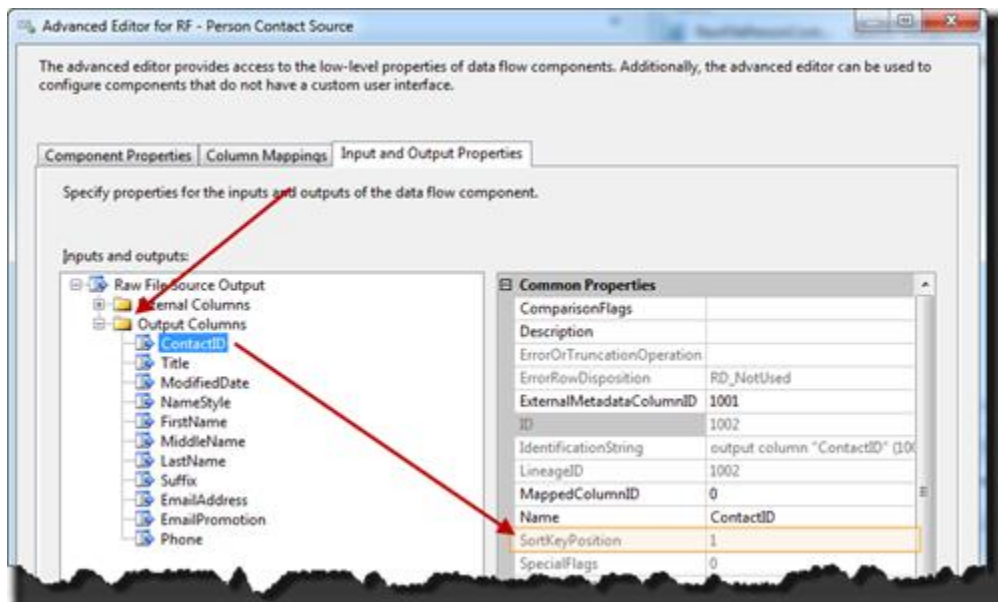
Browse in the Advance Editor to the **Input and Output Properties** tab. On this tab there will be a tree view for the transformation output. Select the root item in the tree view and the Common Properties window will populate. This can be done by following the red arrows to the yellow region.





Within the Common Properties there is an option for IsSorted. Change this property to True. This change will make the SSIS package think that the output from this source is sorted. SSIS will not perform any validations to validate this.

Once the output is configured as sorted, the sort columns need to be selected. To do this, select the Output Columns folder. Then select the ContactID column. Follow the red arrows below to the orange section. This will be the SortKeyPosition. Change this value from 0 to 1.



When this is done, the data source is configured to be a sorted data source. Remember what I said above, SSIS will not perform any validations to validate that data sources marked sorted are sorted.

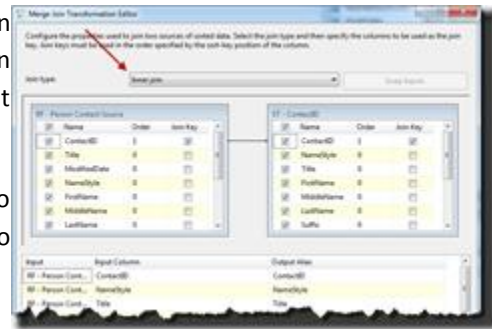
CONFIGURE MERGE JOIN TRANSFORMATION

Configuring the Merge Join transformation is relatively easy. Within the properties window for the transformation choose between INNER, LEFT, or FULL joins. These joins do the same operation that they do within T-SQL.

Next, select the columns that you want the transformation to output. The join columns will already be selected so there is no need to configure those.

It's really as easy as that.

There are a couple custom properties that can be set for the Merge Join transformation. These properties can be used to change the behavior of the joins.



Custom Properties	
JoinType	Inner join
MaxBuffersPerInput	5
NumKeyColumns	1
TreatNullsAsEqual	True

Those properties are:

- MaxBufferPerInput:** This property can be used to tune the memory requirements for the Merge Join. Joining data does incur resource costs. Depending on the data being processed this could have an impact on other packages. Or you may have additional resources you can apply to the execution. This property will assist in controlling that.
- TreatNullsAsEqual:** Exactly as the property states, the property will allow NULL values to be treated as the same values.

WORD OF CAUTION

This transformation can bite you in uncomfortable ways if you don't pay attention to how the data is coming into it. First, the transformation only does one pass across the datasets. If you have unordered records in one of the datasets you could end up with more, less or duplicated data. Pay attention to the sorts. Document the sorts so they aren't removed by mistake.

MERGE JOIN WRAP-UP

The Merge Join can be an extremely useful transformation. With relatively low effort it can join together two datasets and provide an output that spans multiple platforms. The transformation sometimes gets a dodgy reputation because of its need for sorted data. But with a little planning the data can often be pre-sorted to avoid this pitfall.

With the scenarios in the introduction, I didn't get into any specifics on performance and when you'd choose this method or maybe a Lookup transformation. This question can often be answered by looking at two factors:

If you are pre-SQL Server 2008, then when you have static data for comparisons this method can provide a nice boost over a Lookup transformation.

If the number of rows are in the 10's of thousands or past a million records you will likely want to use a raw file over a cache for the Lookup transformation. A lot of time can be spent building a cache to have the index and avoid needing to sort data. Test both ways because the Merge Join solution performance may surprise you.

But what about real performance from a real world scenario. Here's an anecdote from my experiences with the Merge Join solution.

I was once called to take a look at an SSIS package that was using a cache file and a Lookup transformation. The solution was deployed to production and the production data didn't match the development data volumes (no surprise). The solution that ran great was taking hours to complete. I think by the time I had been called it had been run more than 10 hours at a time a few times without completion.

A little rewriting of the package to replace a couple Lookup transformations with this solution and the package was back to development execution timeframes. The last time I had talked to anyone on the package it was executing in about 5 minutes for the operation that had taken hours before.

If you have any questions, please leave a comment. If you have challenges you are facing, I would be interested in hearing about those as well.

OVERRIDE VARIABLES

Day number 9 of the [31 Days of SSIS](#). If you missed the first week then start by reading the [introductory post](#). Since it is Sunday, I'm going to go with something a little lighter and shorter – or at least that's the expectation as this starts.

In a previous post, I discussed [variables and how to use one variable to control multiple variables](#). Today, the discussion will be similar in nature. The idea is to demonstrate how control the behavior of variables defined by expressions at run-time. This may sound all convoluted but the rest will straighten out this confusion.

VARIABLE EXPRESSIONS

As was previously mentioned, the values for variables with expressions are calculated at run-time. The result of which is any value that a variable has prior to the use of the variable in the package is discarded when the variable is called. The value is calculated and returned when the variable is referenced.

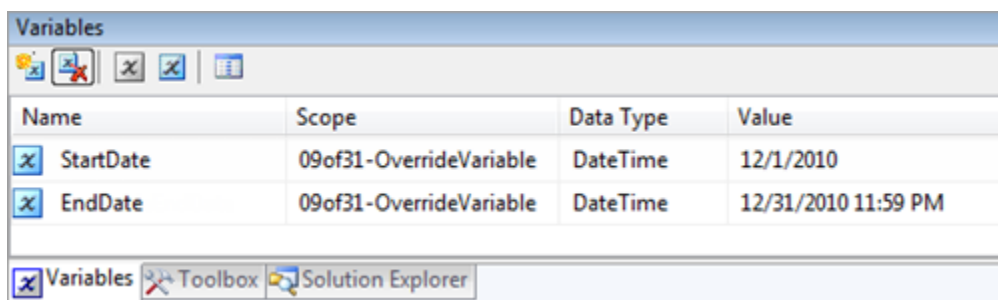
The unintended effect of this is often encountered when you need to change the values in an SSIS package for a one-off execution. If you pass in some custom values through configuration settings or command line options, the values end up being discarded when the expression fires on those variables.

Let's build an example SSIS package to demonstrate. This package will be required to return data for the previous month based on values calculated at run-time.

First, create a new package and add two variables to the package with the following expressions on them:

- **StartDate:** DATEADD("mm", DATEDIFF("mm", (DT_DBTIMESTAMP) "1900/01/01", GETDATE())-1, (DT_DBTIMESTAMP) "1900/01/01")
- **EndDate:** DATEADD("ms", -3, DATEADD("mm", DATEDIFF("mm", (DT_DBTIMESTAMP) "1900/01/01", GETDATE()), (DT_DBTIMESTAMP) "1900/01/01"))

The Variables window will look like the following:



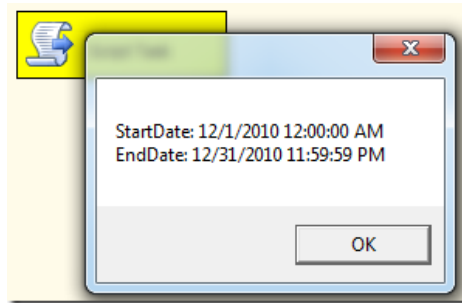
Name	Scope	Data Type	Value
StartDate	09of31-OverrideVariable	DateTime	12/1/2010
EndDate	09of31-OverrideVariable	DateTime	12/31/2010 11:59 PM

Then add a Script Task to the Control Flow. Add two variable in as ReadOnlyVariables. The following code should be added to the Script Task code to replace "public void Main()":

```
public void Main()
{
    MessageBox.Show("StartDate: " + Dts.Variables["StartDate"].Value.ToString() + Environment.NewLine
    + "EndDate: " + Dts.Variables["EndDate"].Value.ToString());
}
```

```
Dts.TaskResult = (int)ScriptResults.Success;
}
```

Now run the package to see the results. You should get a message box in the execution that looks like this:

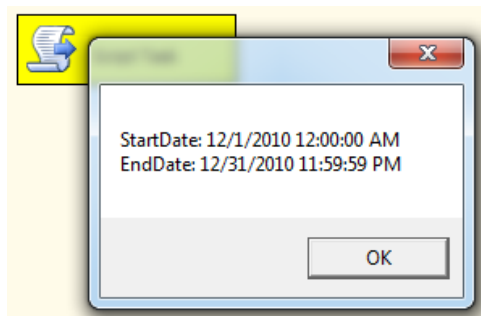


For normal execution, this is perfect.

But let's pretend for a moment that there are end users. Further, let's pretend that they end users somehow managed to put bad data into the database that this package was retrieving data from. These end users go out and fix their data. And now they want you to re-run the package for November 1 to November 30.

Go back to the package and change the values for StartDate and EndDate to November 1 and November 30, respectively. Run the package and what results do you see?

I bet they look a lot like this:



Exactly not what they wanted and hopefully what you expected. The variable are calculated at run-time regardless of the values that appear in the SSIS package when the package starts.

OVERRIDE EXPRESSIONS

Getting around this issue is actually very easy. To get started, add two more variables to the package. The variables should be the following with the listed initial values:

- **OverrideStartDate:** 1900/01/01
- **OverrideEndDate:** 1900/01/01

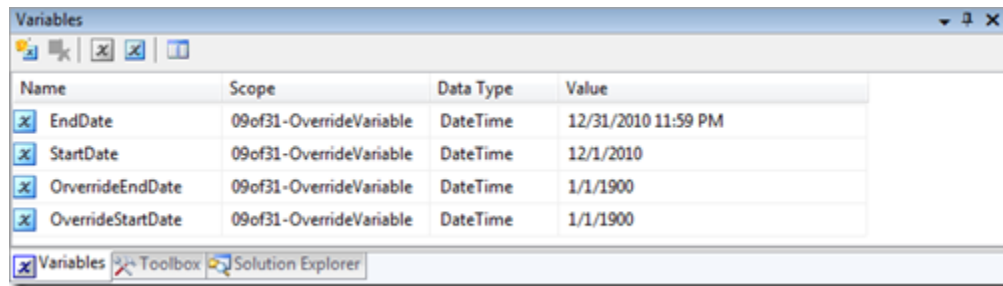
Change the expressions on the other two variables to the following:

- **StartDate:** @[User::OverrideStartDate] == (DT_DBTIMESTAMP)"1900/01/01" ? DATEADD("mm", DATEDIFF("mm", (DT_DBTIMESTAMP) "1900/01/01", GETDATE()))-

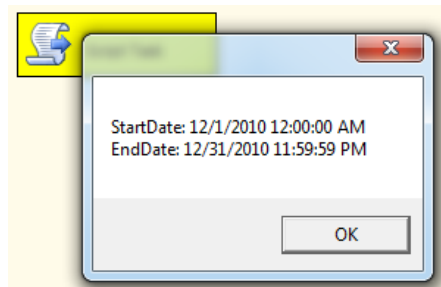
```
1, (DT_DBTIMESTAMP) "1900/01/01") : DATEADD("d", DATEDIFF("d", (DT_DBTIMESTAMP)
"1900/01/01", @[User::OverrideStartDate]), (DT_DBTIMESTAMP) "1900/01/01")
```

- **EndDate:** @[User::OverrideEndDate] == (DT_DBTIMESTAMP)"1900/01/01" ?
DATEADD("ms", -3, DATEADD("mm", DATEDIFF("mm", (DT_DBTIMESTAMP)"1900/01/01",
GETDATE()), (DT_DBTIMESTAMP)"1900/01/01")) : DATEADD("ms", -3, DATEADD("d", 1 +
DATEDIFF("d", (DT_DBTIMESTAMP) "1900/01/01",
@[User::OverrideEndDate]), (DT_DBTIMESTAMP) "1900/01/01"))

The Variables window will look like the following:



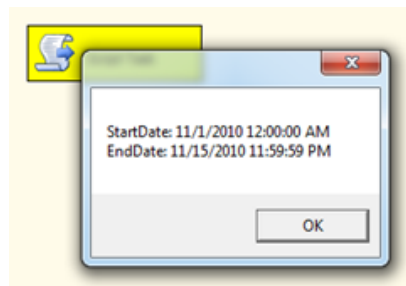
If you execute the SSIS package with the two new variables with the values listed above you'll see results similar to those the first time this was executed:



But the goal here is to run the package with a different time frame. To test this change the variable to the following values:

- **OverrideStartDate:** 2010/11/01
- **OverrideEndDate:** 2010/11/15

Now executed the package and you'll see that the package can run for another time frame.



With a little extra work on variable expressions, packages can be made to more dynamic. These extra configuration point helps provide an insurance policy when end users or clients want a little bit more out of the SSIS package.

And let's face it, they probably already expect the process will be able to do this without having requested the functionality.

ANOTHER CONSIDERATION

Without going into details, there is another thing to consider when writing expressions on variables. That is the use of the `GetDate()` function.

First, remember that variables are calculated whenever the variable is called. Next packages take time to execute. Thus the value of `GetDate()` will change as the package executes.

After looking at these, the value of any expression that uses `GetDate()` will change throughout the execution of an SSIS package. This means you have to be careful when you use `GetDate()` in variable expressions.

As an alternative, consider using `@[System::StartTime]` instead of `GetDate()`. It returned the date and time that a package starts and is static throughout the execution.

WRAP-UP

This post wasn't as short as I expected and I managed to combine another post in leaving room for another in the schedule. If you have any questions regarding variable expressions, feel free to ask.

PIVOT TRANSFORMATION

A few months back, well before the [31 Days of SSIS](#) started, I had been playing with the [Pivot Transformation](#). The transformation seemed to fit an issue that I was working through. After getting to a point where I wanted to [smash my computer](#) I finally figured out how to use it and thought this would be great to share.

The Pivot transformation does pretty much exactly what that name implies. It pivots data along an x-axis which is determined by values in a column. Another column of value are used along the y-axis to determine the columns to include. Since this columns on the y-axis are determined by the data in the source there is some configuring in the Pivot transformation that needs to occur to properly pivot data.

On a general sense, with the Pivot transformation you are taking data similar to that on the left in the image below. Then transforming it into a data set similar to the data on the right.

The image shows two side-by-side windows from SQL Server Data Tools. The left window, titled 'OLE DB Source Output Data Viewer 1 at OLES - Annual Sales', displays a table with three columns: Territory, OrderYear, and TotalDue. The data is organized by Territory (Australia, Canada, Central, France, Germany, Northeast, Northwest, Southwest, United Kingdom) and then by OrderYear (2001, 2002, 2003, 2004). The right window, titled 'Pivot Default Output Data Viewer 1 at PVT - Sales Year.Pivot Default Output', displays the result of the pivot transformation. The columns are Territory, 2001, 2002, 2003, and 2004. The rows are organized by Territory. An arrow points from the left window to the right window, indicating the transformation process.

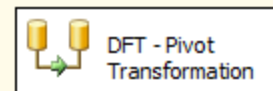
Territory	OrderYear	TotalDue
Australia	2001	1446467.1744
Australia	2002	2380484.8367
Australia	2003	4547123.2777
Australia	2004	3823410.2386
Canada	2001	2173647.1453
Canada	2002	7215430.5017
Canada	2003	8186021.9178
Canada	2004	3628712.8926
Central	2001	1263884.1024
Central	2002	3518185.4756
Central	2003	4015356.874
Central	2004	1771532.7396
France	2001	196531.723
France	2002	1717145.7438
Germany	2001	1446467.1744
Germany	2002	2380484.8367
Germany	2003	4547123.2777
Germany	2004	3823410.2386
Northeast	2001	2173647.1453
Northeast	2002	7215430.5017
Northeast	2003	8186021.9178
Northeast	2004	3628712.8926
Northwest	2001	1263884.1024
Northwest	2002	3518185.4756
Northwest	2003	4015356.874
Northwest	2004	1771532.7396
Southwest	2001	196531.723
Southwest	2002	1717145.7438
United Kingdom	2001	1446467.1744
United Kingdom	2002	2380484.8367
United Kingdom	2003	4547123.2777
United Kingdom	2004	3823410.2386

Territory	2001	2002	2003	2004
Australia	1446467.1744	2380484.8367	4547123.2777	3823410.2386
Canada	2173647.1453	7215430.5017	8186021.9178	3628712.8926
Central	1263884.1024	3518185.4756	4015356.874	1771532.7396
France	196531.723	1717145.7438	4366076.3476	2653948.6196
Germany	1446467.1744	2380484.8367	4547123.2777	3823410.2386
Northeast	2173647.1453	7215430.5017	8186021.9178	3628712.8926
Northwest	1263884.1024	3518185.4756	4015356.874	1771532.7396
Southwest	196531.723	1717145.7438	4366076.3476	2653948.6196
United Kingdom	1446467.1744	2380484.8367	4547123.2777	3823410.2386

To demonstrate the use of the Pivot transformation, we'll go through building a package with the transformation.

PACKAGE CONTROL FLOW

This package will be fairly simple. Start by adding a Data Flow task to the Control Flow. After that's done, we're done with the Control Flow. No variable or special configuration for this package.



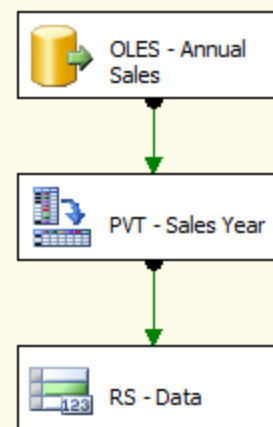
PACKAGE DATA FLOWS

Similar to the control flow, the data flow is going to be very simple. Only one of the transformations in the data flow will be of interest in this case but we'll go through all of it just the same.

DFT – PIVOT TRANSFORMATION

The transformations in the in data flow are the following:

- **OLES – Annual Sales:** OLE DB Source to the AdventureWorks database. The data source retrieves sales data.
- **PVT – Sales Year:** Pivot transformation that will pivot sales data from one record per territory per year to one record per territory with years as columns. The configuration of this transformation will be described in detail below.



- **RS – Data:** Row Sampling transformation that is included just to demonstrate the data movement.

PIVOT TRANSFORMATION CONFIGURATION

To start configuring the Pivot transformation, double-click on it. By doing this the Advanced Editor for the transformation will open. Unlike most other transformations, the Pivot transformation is always configured from the Advanced Editor.

COMPONENT PROPERTIES

The first tab to configure is the Component Properties. For the most part you won't change anything on this tab. I mention the tab just for completeness in describing the configuration.

INPUT COLUMNS

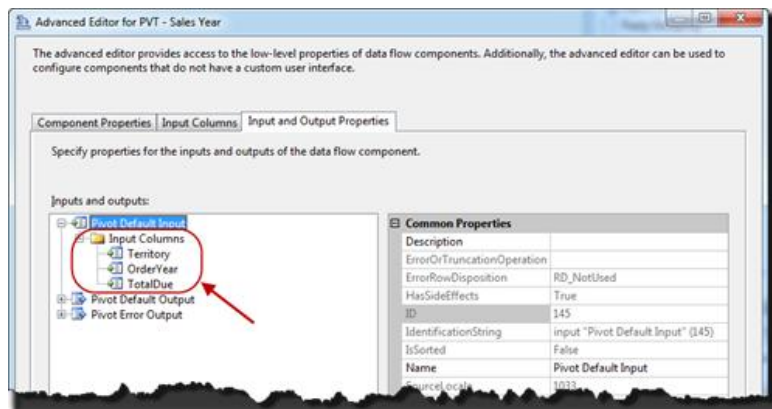


The first place to make configuration changes will be on the Input Columns tab. This tab lists all of the columns available from the input to the transformation. For this example, select all of the columns. For your data, pass through the columns that you will need.

INPUT AND OUTPUT PROPERTIES

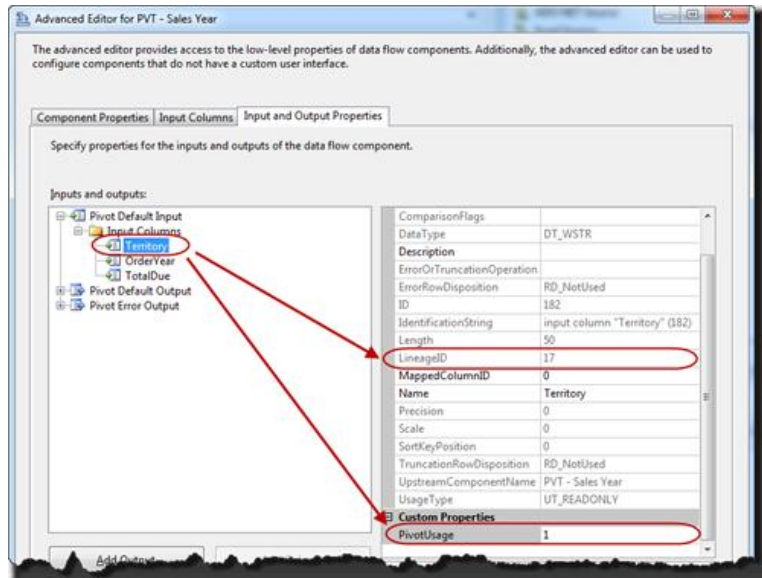
This is the part that takes nearly all of the configuration. I recommend grabbing a pen and paper to take notes as you set this up. Getting the configuration values correct will be critical.

To start with expand the **Pivot Default Input** folder in the tree view. This will show the three columns that were selected in the previous tab.



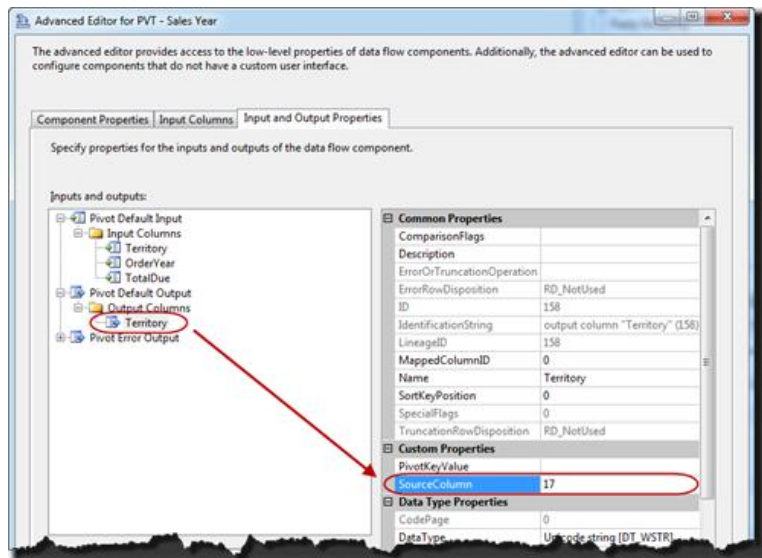
From here highlight the **Territory** column. In the properties window take note of two of the properties; **LineageID** and **PivotUsage**. **LineageID** will be used in configuring future properties of the

transformation. The **PivotUsage** should be set to 1. This value configures this column to be part of the x-axis of the pivot. Rows will be grouped across this column.

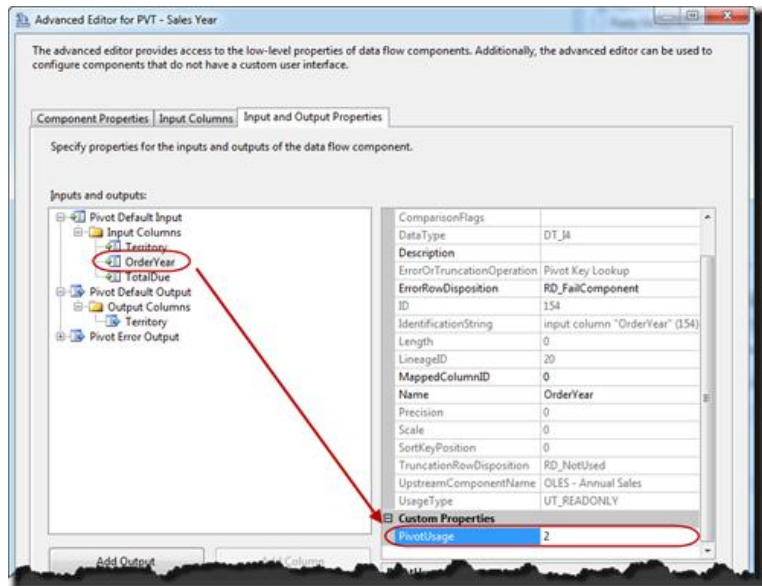


Now expand the **Pivot Default Output**. There will be no output columns in the output. Start by selecting the button Add Column. Name the new column Territory.

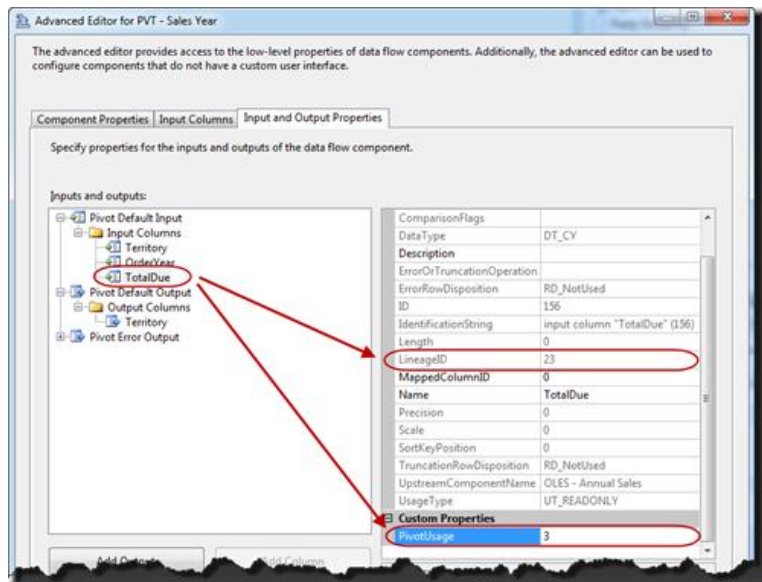
In the properties for the new column change the value for the **SourceColumn** property to 17. This is the value that was just noted above.



Back in the **Pivot Default Input** select the **OrderYear** column. Change the value for the **PivotUsage** on this column from 0 to 2. The value of 2 configures the column to be the y-axis of the pivot. The values in this column will be used to create new columns in the pivot operation.

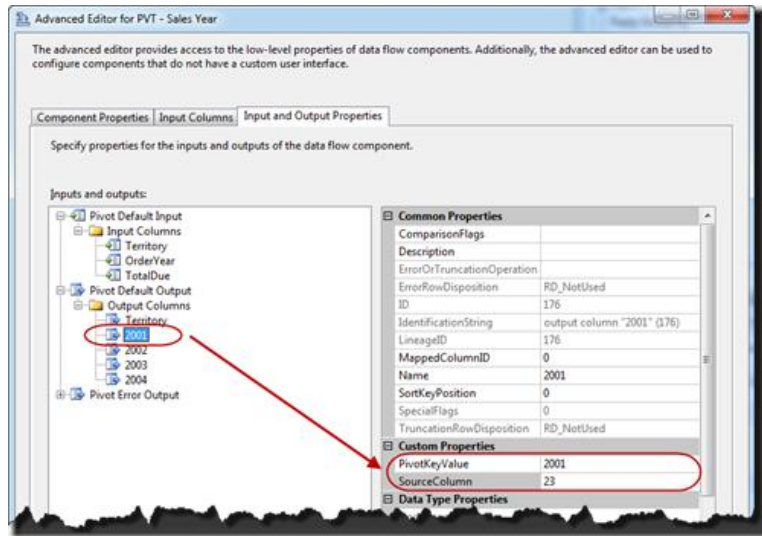


Going back up to the **Pivot Default Input**. Select the **TotalDue** column. Change the **PivotUsage** for this column to 3. The value of 3 configures the values in the columns to be placed in the new columns create by pivoting the data. Take note of the **LineageID** as this will be used for configuring in the next step.



The final step will be to add four new columns to the **Pivot Default Output** folder. These columns should be named 2001, 2002, 2003, and 2004. The name of the columns doesn't necessarily matter.

For each of the columns, configure the **PivotKeyValue** and **SourceColumn** properties. The value for the **PivotKeyValue** property will be the value in the **OrderYear** column for that column. For the 2001 column the value of the property will be 2001. For the **SourceColumn** property use the **LineageID** from above.



At this point, the Pivot transformation is configured. Select the OK button to close out the window. Then run the the SSIS package.

PIVOT TRANSFORMATION WRAP-UP

Once you've setup your first Pivot transformation, you'll see that it isn't that difficult to use. It's a bit tricky the first few times since it doesn't have an easy interface like many of the other transformations.

There are a few considerations that you need to take into account when using a Pivot transformation. These are important and your mileage will suck if you forget these:

1. The Pivot transformation expects, but not check for, sorted inputs. The x-axis values need to be grouped together to get the correct output. SSIS will not error if you forget to do this.
2. The Pivot transformation will error if it encounters a value that has not been handled by the output columns. This is good because you won't accidentally miss something in the source data. This can be bad if you only want to pivot a subset of the columns to the y-axis.
3. Columns that are not part of the pivot operation can be included in the output by setting their **PivotUsage** value to 0.

That about covers the Pivot transformation. Hopefully this helps if you've tried to use this transformation in the past. I know the frustration that setting this transformation up can bring.

UNPIVOT TRANSFORMATION

The last post discussed the [Pivot transformation](#) – that leads us to this post which will be on the [Unpivot Transformation](#). Part of the reason I had been trying to fit the the Pivot transformation into the issue I was having was also because I knew how to use the Unpivot transformation and thought I had better know how to use the Pivot transformation.

Let's reverse that psychology and for this [31 Days of SSIS](#) post we'll look at this transformation and what it can do. In the end, we'll wrap up with why I began using this transformation in the first place. And provide some ideas on how you can use it in the future.

As the name suggests, the Unpivot transformation will unpivot data in a result set. Unfortunately, definitions should use their own self to define themselves. A better way to state this is that values in multiple columns can be transformed into a single column. The end result is to take normalized result set and return name-value pairs instead.

Consider a result set where sales information for each year is stored in a year column. Unpivoting the data would result in a row for every year instead of a column for each year. The data would transform similar to the image below:

The image shows two data viewers side-by-side. The left viewer, titled 'OLE DB Source Output Data Viewer 1 at OLES - Annual Sales.OLE DB Source Output', displays a table with columns: Territory, 2001, 2002, 2003, and 2004. The right viewer, titled 'Unpivot Output Data Viewer 1 at Unpivot.Unpivot Output', displays the result of the unpivot transformation, with columns: Territory, Year, and TotalDue. An arrow points from the first viewer to the second, indicating the transformation process.

Territory	2001	2002	2003	2004
Australia	1446497.1744	2380484.8387	4547123.2777	3823410.2386
Canada	2173647.1453	7215430.5017	8186021.9178	3926712.8926
Central	1263884.1024	3518185.4756	4015356.874	1771532.7396
France	199531.723	1717145.7439	4366078.3475	2853948.6596
Germany	262752.4184	575960.0974	2714826.4297	2386224.5508
Northeast	754833.2045	3275322.1694	3833030.25	1406555.6861
Northwest	2703481.7947	5651688.6685	7494658.0357	4952772.2793
Southeast	1928148.6139	3814944.144	3261402.9982	1566084.6664
Southwest	3272568.5203	9947538.7106	11523237.5187	6470114.826
United Kingdom	322207.5294	1778804.7452	4365879.4375	3039555.8775

Territory	Year	TotalDue
Australia	2001	1446497.1744
Australia	2002	2380484.8387
Australia	2003	4547123.2777
Australia	2004	3823410.2386
Canada	2001	2173647.1453
Canada	2002	7215430.5017
Canada	2003	8186021.9178
Canada	2004	3926712.8926
Central	2001	1263884.1024
Central	2002	3518185.4756

To demonstrate the use of the Unpivot transformation, we'll go through building a package with the transformation.

PACKAGE CONTROL FLOW

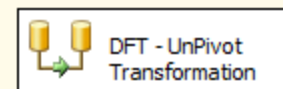
Another fairly simple SSIS package. To start add a Data Flow task to the Control Flow. No variables or special configurations will be needed for this package. Go next to the Data Flow task.

PACKAGE DATA FLOWS

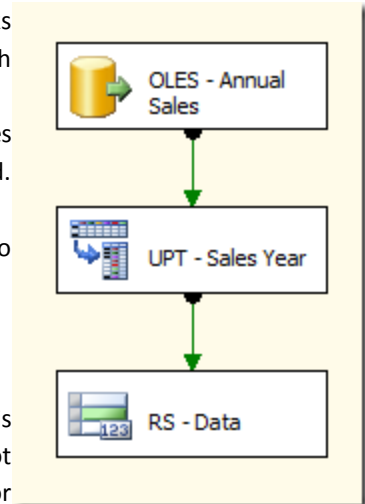
Simple package. Simple data flow. Since there is only this one we'll just right into that data flow and it's transformations.

DFT – UNPIVOT TRANSFORMATION

The transformations in the data flow are the following:



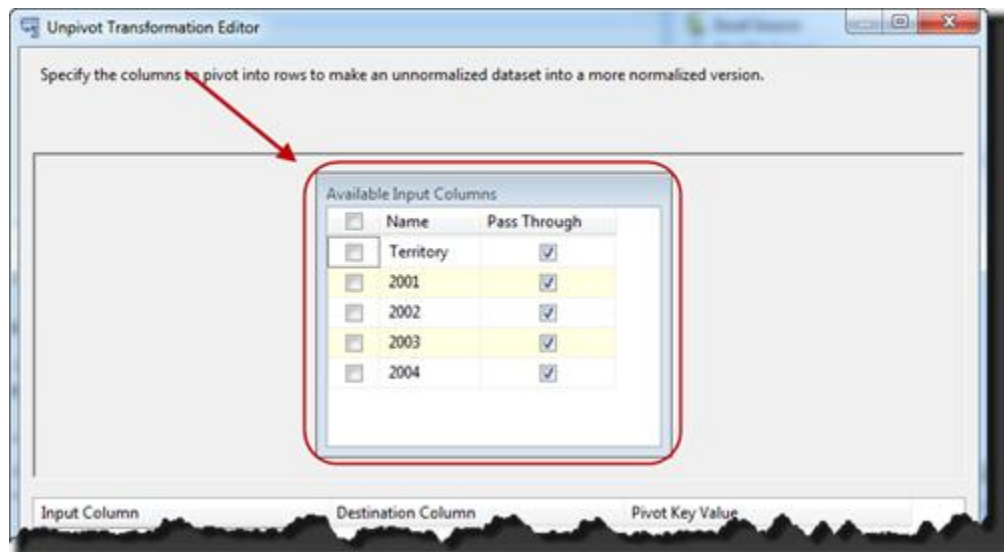
- **OLES - Annual Sales:** OLE DB Source to the AdventureWorks database. The data source retrieves sales data with columns for each year.
- **UPT - Sales Year:** Unpivot transformation that will unpivot the sales data by year. For each column, there will be one row returned. configuration of this transformation will be described in detail below.
- **RS - Data:** Row Sampling transformation that is included just to demonstrate the data movement.



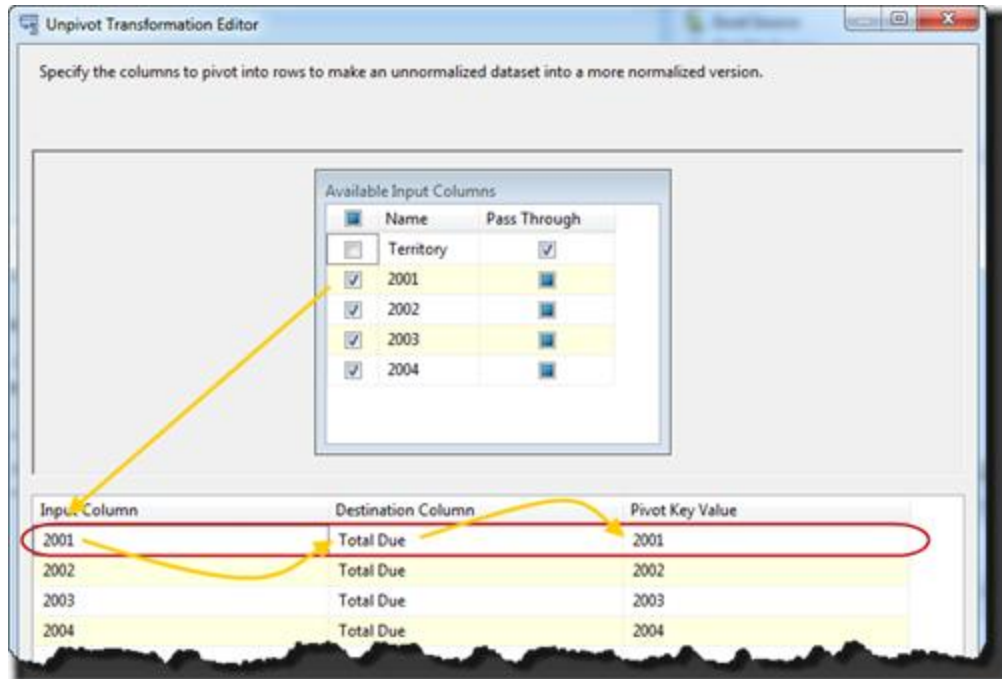
UNPIVOT TRANSFORMATION CONFIGURATION

To start configuring the Unpivot transformation, double-click on it. When this is done, the Unpivot Transformation Editor will open. Fortunately, the Unpivot transformation has an editor that doesn't require the use of the Advanced Editor for Unpivot Transformation. In fact, there isn't much of use in trying to use the Advance Editor over the regular editor.

Let's take a look at how to configure the Unpivot transformation. Initially, you'll see that the columns from the input are listed in a selection area at the top of the window. Initially all of the columns will be configured for **Pass Through**.



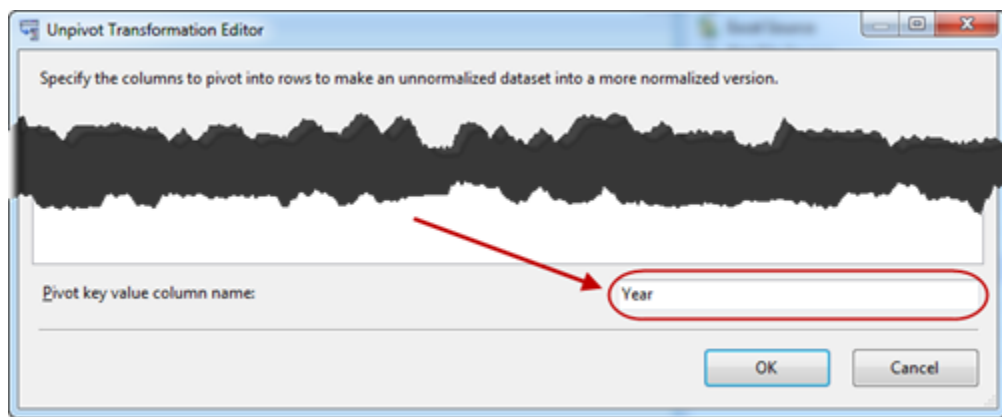
The intention is to return all of the data in the year columns (2001, 2002, 2003, and 2004) in a single column. The values in the rows for these columns are the Total Due dollars. To start this, select the year columns on the left-side checkboxes. Doing this will add these columns as **Input Columns**.



After each column is selected, change the value of the **Destination Column** to **Total Due**. This setting will create the column where the value of each row in the column will be placed.

Optionally, the **Pivot Key Value** can be modified to change the value that each row will have in the name-value pair. For instance, if the data in the 2001 column should be on a row with a value of "Year-2001" instead of "2001" then this is where that configuration will be made.

The last step is to name the column that the Pivot Key Values will be populated into. That is done with the last setting available in the window. The **Pivot key value column name:** text box. Type Year in that box and click OK.



At this point, the configuration of the Pivot transformation is complete. First off the package and the output will be identical to what was in the introduction.

UNPIVOT TRANSFORMATION WRAP-UP

Now you know how to use the Unpivot transformation. It's simple and not too confusing. The next question is what solutions would require the use of this transformation.

It's true, you could use it to unpivot data as it comes out of your SQL Server and prepare it to be delivered to end users for reporting. But in many of those cases, the data would be probably better unpivoted at the source. Or most likely isn't being stored in a manner that requires an unpivot operation.

The case where I get the most use of the Unpivot transformation is when loading data into client databases. Many of the clients I work with have a need to import data from their customers to their database. Each of the data sets from their customer's data has their own nuances. This variation leads to the need for a set of tables that are designed to store name-value pairs.

Sometimes, I load up the client data to staging tables and then do the transfer of each of the columns from the source file to the destination column. The problem here is that it can be tedious to setup. With many different files to configure and load there is an appeal to using the unpivot to prepare the data in the name-value pair while importing it. As opposed to creating custom structures for each column in each file.

In this use case, the unpivot operation has been extremely helpful. Previously, I would previously design tables and write stored procedures. Now, I click, click, click. Write a couple column names. Then the work is done. It's like magic.

That should cover the Unpivot transformation. If there are any questions feel free to leave a comment.

UNPIVOT ALTERNATIVE

This is the twelfth post in the [31 Days of SSIS](#) blog series. This post is going to go through solution that I sometimes use as an alternative to the [Unpivot transformation](#). Probably not too surprising, this solution is going to take advantage of [raw files](#).

The Unpivot transformation is a relatively easy transformation to setup and configure. The trouble is there are situations where you may need to unpivot data where the unpivot transformation is not ideal. In this post, I'll describe one such situation and then review an alternative method to unpivoting data that I have used.

PACKAGE REQUIREMENTS

As a DBA, one of the more common activities that I need to use SSIS for is to load data from customer data files into client databases. This process is usually pretty easy mapping out people, orders, items, etc. to the their respective database objects.

Often times, though, the customers will have a data element or thirty that doesn't match to the client database. This information can be stored in various methods, but the most common method that I run into is structures for storing name-value pairs. Another name for this is the [entity-attribute-value model](#).

Unpivoting data for this type of an import is fairly easy for an average sized file with an average number of columns. This gets tricky with some of the situations that I've run into at times.

For instance, in one solution a file with a couple hundred thousand rows was unpivoted and the resulting output was in the 10s of millions. This happened due to the large number of name-value pairs that needed to be setup for the file. In this case, as each column was unpivoted and validated it forced all of the operations to be done serially.

What I needed was a method that would unpivot the data. It also, though, needed to be able to put the output data in a state that it could be processed in parallel in later stages of the import.

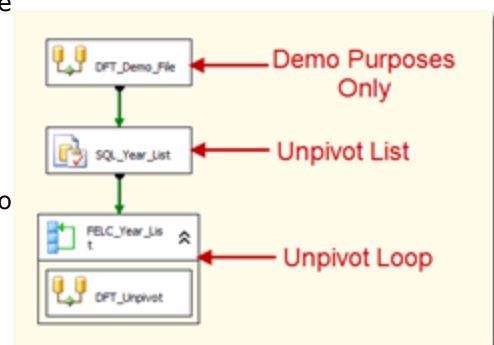
UNPIVOT ALTERNATIVE SOLUTION

The solution to using the Unpivot transformation will need to know what fields are in the file that need to be unpivoted from the customer file. Then for each of these field, the value in the customer file will need to be extracted.

As I mentioned above, after the unpivot each of the value should be in a state that can be processed in parallel. To do this, the values that are unpivoted will be exported into individual files.

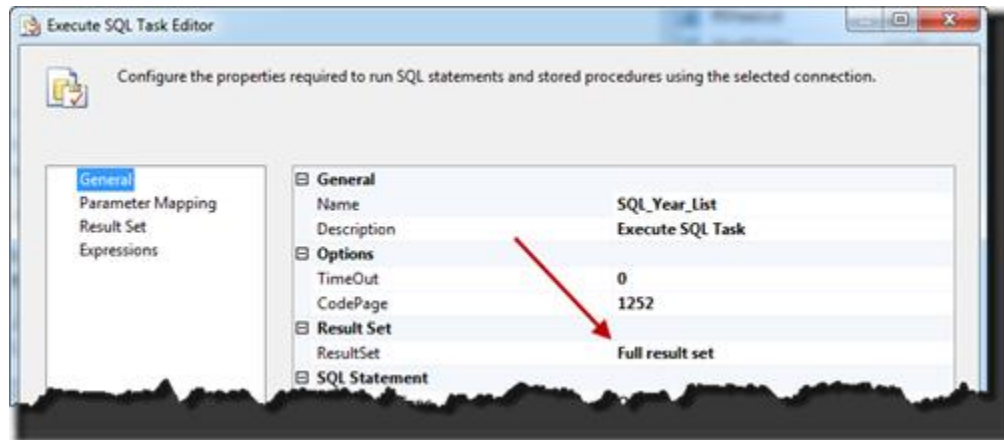
PACKAGE CONTROL FLOW

To accomplish this we will need a few components added to the control flow.

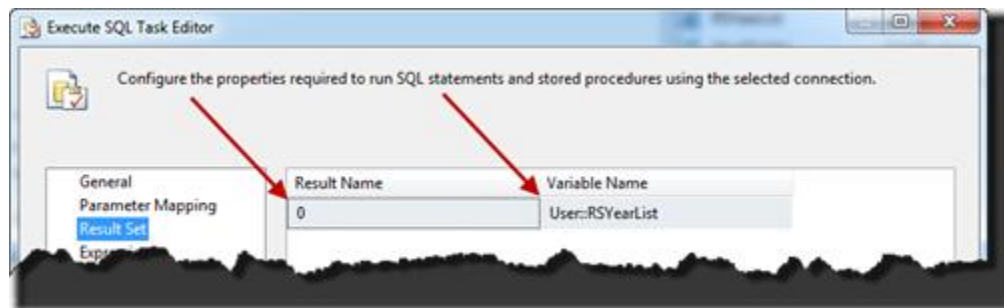


First, add a Data Flow task that will be used to extract a sample file to simulate an import. This will be discussed in more detail below.

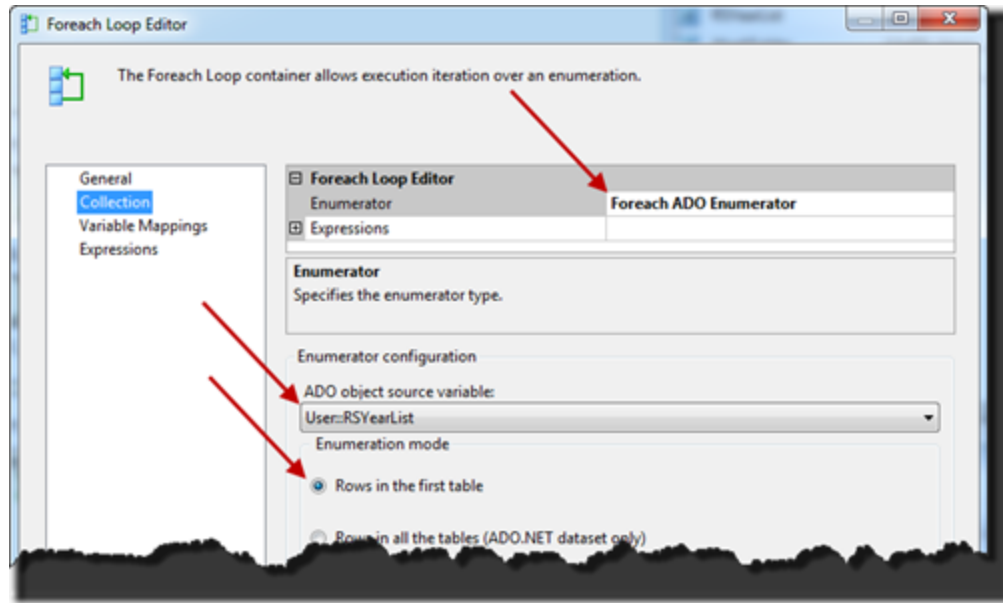
Next add an Execute SQL task. The data source for the Execute SQL task will be a SQL Server database with the AdventureWorks database. The query can be found in the download at the end of the post. It is important, though, to note that the task will be setup to with **ResultSet** set to **Full result set**.



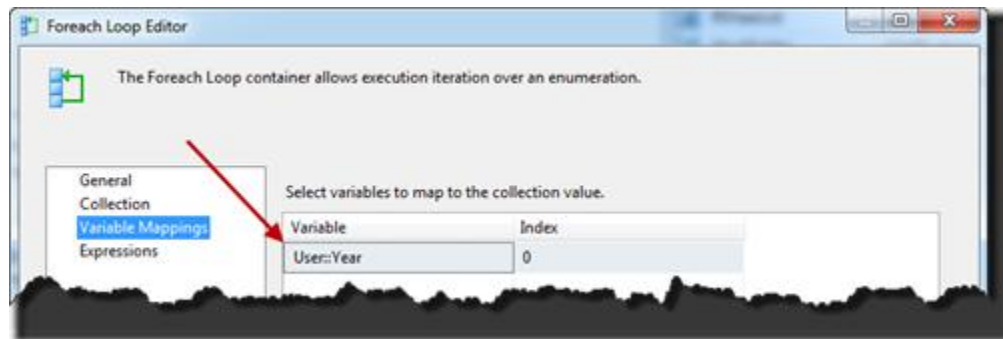
Also, the **Result Set** will be mapped to the **RSYearList** variable.



After this add a **ForEach Loop Container** to the Control Flow. Set the **Enumerator** to **Foreach ADO Enumerator**. The **ADO object source variable** will be **RSYearList**. Leave the **Enumeration mode** to **Rows in the first table**.



Under **Variable Mappings** add a mapping for the variable **Year** to the **Index** of 0. This will tie the variable to the first column in the dataset that will be enumerated.



Within the ForEach Loop Container add a Data Flow task. This task will do the unpivoting work for the package. As with the other data flow task, more details will be provided below.

PACKAGE VARIABLES

As noted above, there will be a few variables that are part of this package.

The screenshot shows the 'Variables' window with a list of package variables.

Name	Scope	Data Type	Value
FileDestination	12of31-Unpiv...	String	c:\temp\SaleDestination_0.txt
FileSource	12of31-Unpiv...	String	c:\temp\SaleSource.txt
RSYearList	12of31-Unpiv...	Object	System.Object
WorkFolder	12of31-Unpiv...	String	c:\temp\
Year	12of31-Unpiv...	Int32	0

The variables from above were added for the following purposes:

- **FileDestination:** Name and location of the text files will be the destination for the unpivoted data. The name of this file will change per column that needs to be unpivoted.
- **FileSource:** Name and location of the text file that is the source for the unpivot operation.
- **RSYearList:** Variable with an object data type. This will store a dataset containing information about the columns to be unpivoted.
- **WorkFolder:** This is my [one variable to rule them all](#) that sets the FileDestination and FileSource paths.
- **Year:** A variable to hold the year value to indicate which column to unpivot.

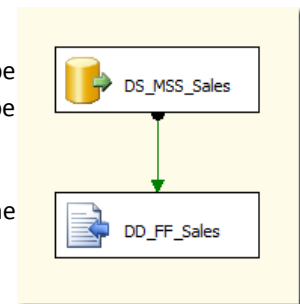
PACKAGE DATA FLOWS

As noted above, there are two Data Flows in this SSIS Package. The Data Flows is where most of the work will happen in the package.

DFT_DEMO_FILE

This Data Flow task creates a text file that will simulate a customer file that needs to be imported. This wouldn't be part of the final SSIS solution since the files would be received from your customers.

- **DS_MSS_Sales:** OLE DB Source that retrieves data from the AdventureWorks database. The data source provides sales information.
- **DD_FF_Sales:** Flat file destination for the sales information.



There isn't much to see here. Moving along then.

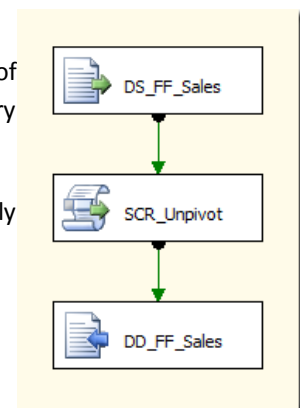
DFT_UNPIVOT

The DFT_Unpivot Data Flow task will be executed one time for each loop of FELC_Year_List. The FELC_Year_List ForEach Loop contained will loop once for every row in RSYearList.

During each execution, the customer file will be read and the column that is currently being unpivoted will be output to destination files.

The transformations in this data flow are:

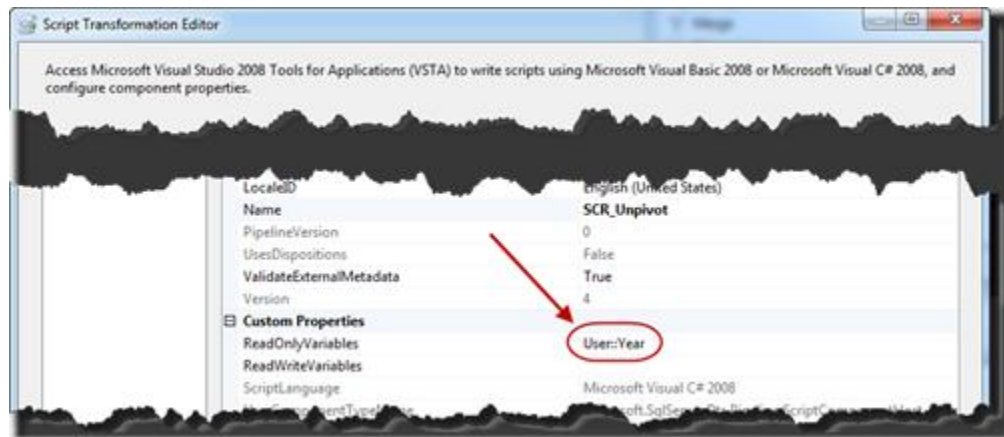
- **DS_FF_Sales:** Flat file source containing the sales information.
- **SCR_Unpivot:** Script Component transformation that selects columns from the source file and outputs the information into a new column. Configuration of this transformation will be detailed more below.
- **DD_FF_Sales:** Flat file destination for each of the unpivoted columns from the sales information.



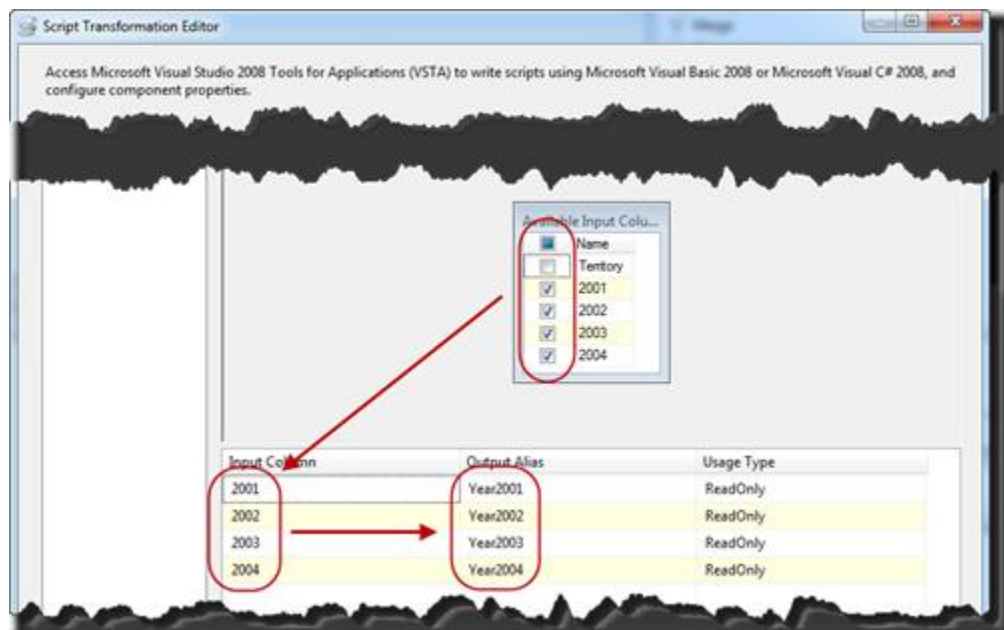
SCR_UNPIVOT CONFIGURATION

In order to use a Script Component transformation to unpivot the data from the source file there are a few things that need to be done to configure the transformation. This section will review the transformation and the steps required to set it up.

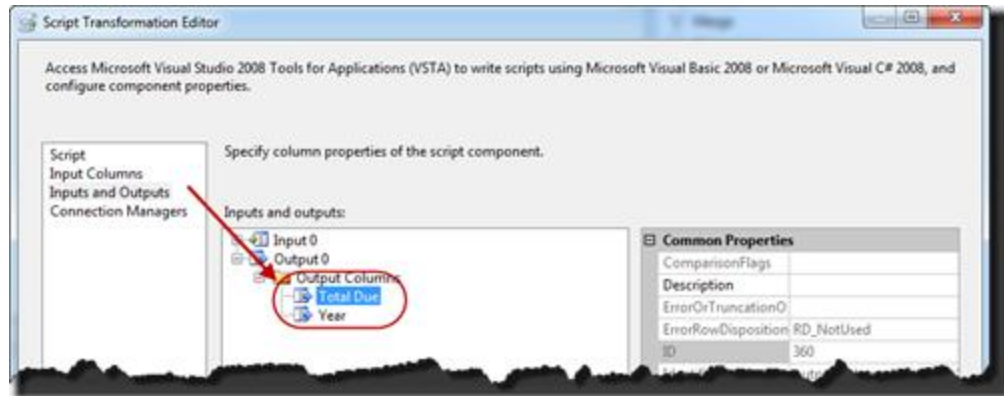
Starting with the first tab, set the **ReadOnlyVariable** to use the **Year** variable. If you are using SQL Server 2008 this can be selected from a list. With SQL Server 2005 you will need to type out the variable name.



Next go to the **Input Columns** tab. In this tab, select the columns 2001, 2002, 2003, and 2004. These columns are the one that will be unpivoted. Notice that they **Output Alias** for each column has the prefix Year appended to it. This is done because columns with all numeric name cannot be used within Script Transformations.



Move next to the **Inputs and Outputs** tab. On this tab add two **Output Columns** to the transformation. The **Total Due** column should have a decimal type. The **Year** should have an integer type.



Finally, the fun part will begin. What's a script transformation without a script. Start by opening the script window for the transformation. For this demonstration the code will be in C#.

In the code window, replace the Input0_ProcessInputRow function with the following code:

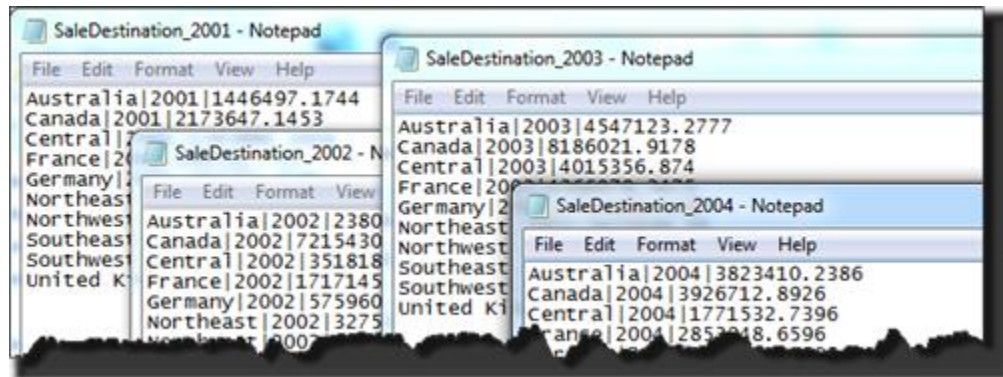
```
public override void Input0_ProcessInputRow(Input0Buffer Row)
{
    if (ReadOnlyVariables["Year"].Value.ToString() == "2001")
    {
        Row.Year = 2001;
        Row.TotalDue = Row.Year2001;
    }
    else if (ReadOnlyVariables["Year"].Value.ToString() == "2002")
    {
        Row.Year = 2002;
        Row.TotalDue = Row.Year2002;
    }
    else if (ReadOnlyVariables["Year"].Value.ToString() == "2003")
    {
        Row.Year = 2003;
        Row.TotalDue = Row.Year2003;
    }
    else if (ReadOnlyVariables["Year"].Value.ToString() == "2004")
    {
        Row.Year = 2004;
        Row.TotalDue = Row.Year2004;
    }
    else
    {
        throw new Exception("Could not locate Custom Field Value for [" +
            ReadOnlyVariables["CustomField"].Value.ToString()
            + "] in Data Flow.  Check configuration of script component.");
    }
}
```

This code will go through the data that is being streamed and based on the **Year** variable will change the values and columns that are assigned to the **Year** and **Total Due** columns. The last part of the code will throw an

error if an unexpected value is encountered. This raises an error if there is an attempt to unpivot columns that do not exist or has not been coded.

UNPIVOT WRAP-UP

Once all of the above is done, the SSIS package can be executed. Doing so will generate four files which contain one column of the unpivoted values in them. The should look similar to the following:



This package may not fit your situation. It will hopefully, though, spark some ideas on other ways to utilize Script Transformations in your solutions. Script Transformations do offer a lot of flexibility to do some great things in SSIS packages. There will likely be a few more of these types of solutions this month.

One of the downsides to this solution is that it does read through the source file multiple times. This can be an issue at times, especially if there is a lot of data to unpivot. It can, though, balance out by the benefits of being able to process the pivoted data in parallel after this step. Also, the difference between pivoting one column and many columns is not that much effort and this can assist in having flexible solutions.

As always, let me know what you think. If anything is unclear, I will update to help clarify.

CREATE XML DOCUMENT

Day thirteen and we are trucking along with the [31 Days of SSIS](#) blog series. The last couple posts discussed methods that could be used to unpivot ([1](#), [2](#)) data to populate data into an entity-attribute-value (EAV) architecture.

PACKAGE REQUIREMENTS

Sometimes, though, it isn't necessary to populate customer data that doesn't fit into the portion of the data model for custom values. Consider for a moment data that needs to be stored for a customer, but will only ever be used for troubleshooting or to be returned in some later process.

Would this data be better stored in the EAV architecture, or some other storage mechanism? As the name of the post infers, it might be better to store that data in XML. You can't throw it away, but you need to store it where it won't get in the way.

This is exactly the issue that was brought up to me a number of months ago. A new import would have millions of records, each of which might have dozens of custom values. Storing these values would drastically change the statistics of the existing EAV architecture that only contained a few million rows by adding millions of more rows.

SCRIPT TASK SOLUTION

For this SSIS package, we need a task that will be able to consume one or more columns. These columns should be added to an XML document that is created and returned by the transformation. The package will then be able to take the XML document created and move it to wherever it needs to be delivered.

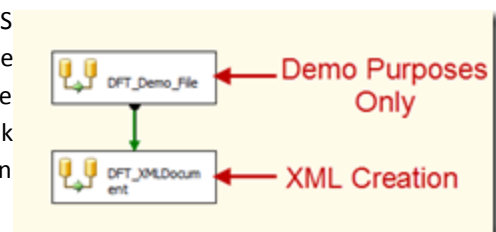
To keep the solution flexible it should be portable to other SSIS packages. The process for creating the XML document should be easily configurable and require limited coding for each new implementation.

To accomplish these needs, we'll implement some code within a Script Component transformation. The code will be dynamic with configuration relying on the columns passed into the transformation.

Before getting too far into explaining this solution, the idea for this solution arose after reading a blog post by John Welch ([Blog](#) | [@John_Welch](#)). In his post, John discussed how to create an [XML Destination Script Component](#). Some of the code in this post is derived from his post. I highly recommend reading this and other posts on his blog – there is some great content on his site.

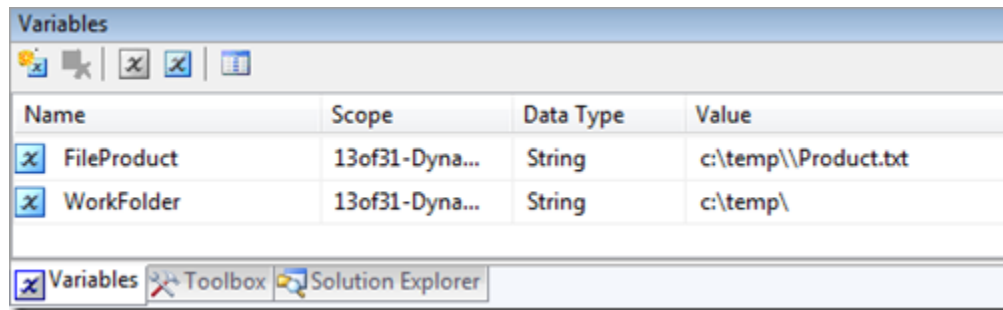
PACKAGE CONTROL FLOW

To demonstrate this solution, we'll build a fairly simple SSIS package. The package only contains a couple Data Flow tasks. The first data flow task creates a file that will be used to demonstrate the functionality in the second data flow task. The second data flow task will consolidate a number of columns into an XML document that can be inserted or update into an XML column for reference.



PACKAGE VARIABLES

Only two tasks, so why not only two variables as well for the package? Well this wasn't intentional, but it was all that I needed.



The variables from above were added for the following purposes:

- **FileProduct:** Name and location of the text file the source for the XML creation transformation.
- **WorkFolder:** This is my [one variable to rule them all](#) that sets the FileDestination and FileSource paths.

PACKAGE DATA FLOWS

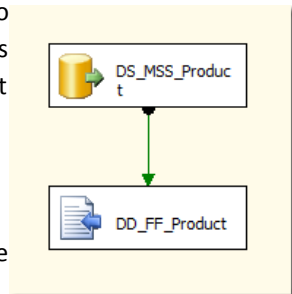
As noted above there are two data flows. In this section, we'll dissect them to explain what you will see when you download the SSIS package.

DFT_DEMO_FILE

Within this data flow task a text file will be created. The purpose of the text file is to provide data for the second data flow. This isn't entirely necessary to demonstrate this functionality but is included to implement the demonstration with the data source that the original design utilized.

There are two transformations in this data flow:

- **DS_MSS_Product:** OLE DB Source that retrieves data from the AdventureWorks database. The data source exports sales information.
- **DD_FF_Product:** Flat file destination for the sales information.

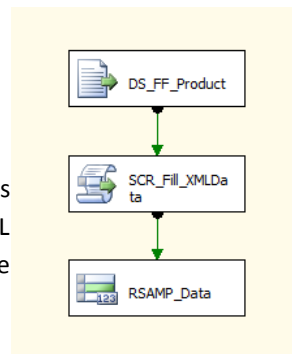


DFT_XMLDOCUMENT

This data flow is where the real action is. The data flow uses the file created in the previous data flow as it's source data. The data is transformed in a Script Component transformation. The last step is a Row Sampling transformation that provides a data flow path that we can hook a data viewer onto.

The transformations in this data flow are:

- **DS_FF_Product:** Flat file source containing the product information.
- **SCR_Fill_XMLData:** Script Component transformation that accepts columns from the data flow path and returns those columns as an XML document in a new column. The configuration of this component will be describe in detail below



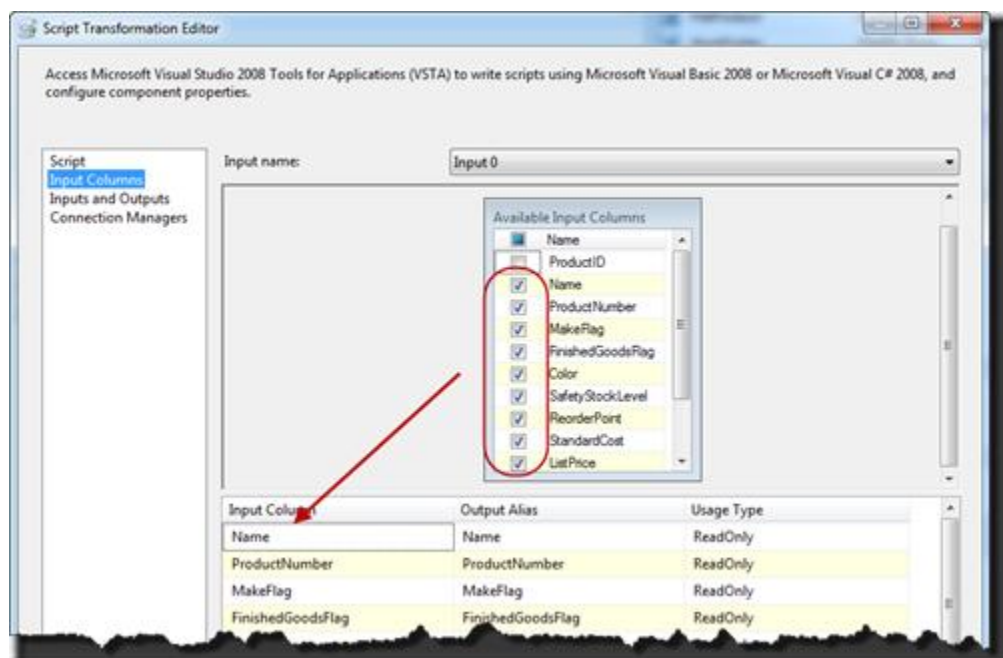
- **RSAMP_Data:** As mentioned above, this is here to provide a place for a data viewer.

SCR_FILL_XMLDATA CONFIGURATION

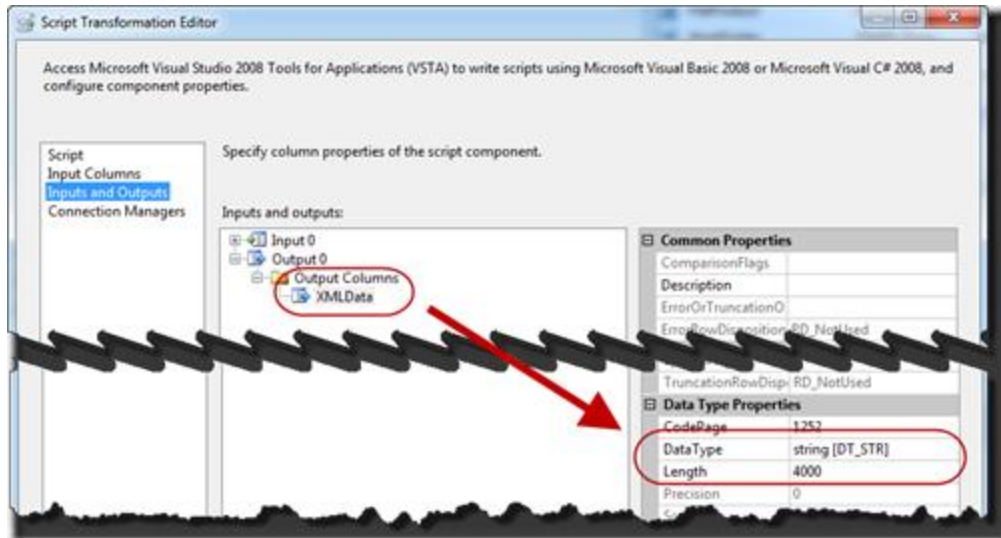
To use a Script Component transformation for building XML there are a few configuration points that will be important to understand. We'll go over setting this up, and how to change the contents of the XML document without changing any code.

First browse to the **Input Columns** tab. On this tab, select all of the columns that you want to include in the XML document that is created. This is the primary configuration point for this task.

The columns selected can be changed at anytime. The only restriction will be the need to recompile the code in order for the underlying collections to be updated. No edits will need to be made.



Next on the **Inputs and Outputs** tab add a new column to the **Output 0** output. This column should be configured as a string data type with a maximum length of 4,000. This is shown below.



Now that these two pieces are configured, go back to the Script tab and select the Edit Script button. The script window will open and you see a basic transformation template.

INPUT0_PROCESSINPUTROW FUNCTION

The first step in setting up the script will be to replace the Input0_ProcessInputRow function with the one below. Before we do that let's review what this script is doing to make certain that you understand the code before you use it.

The code will do the following:

1. For each column that has been selected as an input for the Script Component transformation the function will loop across each column.
2. If the column name is XMLData, don't do anything. Since we will assign the XML document to the column later we don't want to include it in the data being fed into the XML document.
3. Get the name of the column. Spaces and periods are removed from the column name since the Script Component transformation will do this itself.
4. Get the value of the column.
5. Get the name of the NULL column created for the column. When columns are added to a Script Component transformation a NULL is created to allow tests for NULL.
6. If the column is configured as ReadOnly then consider it for the XML document. This was added as a fail safe to the XMLData. It is redundant and could be removed.
7. Check if the column value is NULL. If so, add an empty element to the XML document. Otherwise, add an element to the XML document with the column value.

Now that we've gone through the psuedo-code, here's that function:

```
public override void Input0_ProcessInputRow(Input0Buffer Row)
{
    Type rowType = Row.GetType();
    PropertyInfo columnValue;
    PropertyInfo nullState;
```

```

string XMLData = "<Data>";
string columnName = "";

try
{
    foreach (IDTSInputColumn100 column in ComponentMetaData.InputCollection[0].InputColumnCollection)
    {
        if (column.Name != "XMLData")
        {
            columnName = column.Name.Trim().Replace(" ", "").Replace(".", "");
            columnValue = rowType.GetProperty(columnName);
            nullState = rowType.GetProperty(columnName + "_IsNull");

            if (columnValue.CanWrite == false)
            {
                if (Convert.ToBoolean(nullState.GetValue(Row, null)) == true)
                {
                    XMLData = XMLData + FormatElement(columnName);
                }
            }
            else
            {
                XMLData = XMLData + FormatElement(columnName, columnValue.GetValue(Row, null).ToString().Trim());
            }
        }
    }
}

Row.XMLData = XMLData + "</Data>";
}
catch (Exception ex)
{
    throw new Exception("Error processing column:" + columnName + " " + ex.StackTrace);
}
}

```

The code does what the pseudo-code outlined.

FORMATTING FUNCTIONS

There are two formatting functions that came from the post mentioned above. These allow for converting values to XML element easier with less code. The one distinction with these functions to mention is the inclusion of CDATA. CDATA is used to force the encoding of values that can be in the source data to prevent them from causing the XML document to be malformed.

```

public string FormatElement(string elementName)
{
    return FormatElement(elementName, "");
}

public string FormatElement(string elementName, string data)
{

```

```

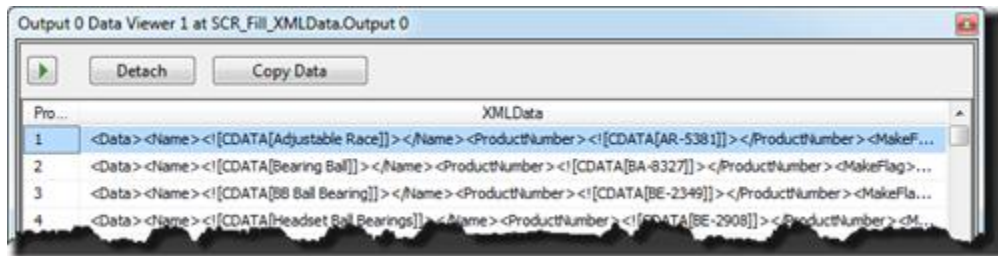
string elementValue;

if (data.Trim() == string.Empty)
{
    elementValue = String.Format("<{0}/>", elementName);
}
else
{
    elementValue = String.Format("<{0}><![CDATA{{{2}{1}{3}}]></{0}>", elementName, data, "[", "]");
}
return elementValue;
}

```

XML SCRIPT TASK WRAP-UP

With that you can execute the SSIS package. If you have a data viewer on the package you'll see something like this:



Pull out one of the values, and you get the following:

```

<Data><Name><![CDATA[Adjustable Race]]></Name><ProductNumber><![CDATA[AR-5381]]></ProductNumber><MakeFlag><![CDATA[False]]></MakeFlag><FinishedGoodsFlag><![CDATA[False]]></FinishedGoodsFlag><Color /><SafetyStockLevel><![CDATA[1000]]></SafetyStockLevel><ReorderPoint><![CDATA[750]]></ReorderPoint><StandardCost><![CDATA[0.0000]]></StandardCost><ListPrice><![CDATA[0.0000]]></ListPrice><Size /><DaysToManufacture><![CDATA[0]]></DaysToManufacture></Data>

```

Wait a minute, this doesn't look like XML! But remember what I mentioned about the CDATA. Drop these values into a variable of the XML data type and select it back out. Try that and you will get the following:

```

<Data>
<Name>Adjustable Race</Name>
<ProductNumber>AR-5381</ProductNumber>
<MakeFlag>False</MakeFlag>
<FinishedGoodsFlag>False</FinishedGoodsFlag>
<Color />
<SafetyStockLevel>1000</SafetyStockLevel>
<ReorderPoint>750</ReorderPoint>
<StandardCost>0.0000</StandardCost>
<ListPrice>0.0000</ListPrice>
<Size />
<DaysToManufacture>0</DaysToManufacture>
</Data>

```

This, of course, looks like what you were expecting. Now go forth and cut and paste this into all of the new SSIS packages where you need this functionality.

The one word of caution what I will add to using this is the limit to 4,000 characters. Right now, strings can't exceed that within SSIS and that can be an issue. If you run into this, this solution will not completely fit for you.

Hopefully, with this and the previous SSIS package you are starting to see that there are a lot of creative things that can be done with Script Component transformations. For the most part, the sky is the limit with these.

INSTANT EXCEL SPREADSHEET

It's day fourteen of the [31 Days of SSIS](#) blog series. It's also Friday, so I'm taking the day off. That doesn't mean that there won't be a post today. What it means is that I have asked [Aaron Drinkwine\(LinkedIn\)](#), one of my co-workers at [Digineer](#), to write a post for today.

This post will go over a pain that plagues us all from time to time. At times we are called upon to export data into Excel. But can we do this without already having an Excel spreadsheet available? Read on and find out...

From Aaron...

TROUBLE IN EXCEL LAND

We all have to face it, spreadsheets are never going away. Business users love to see things in a spreadsheet. They almost thrive on being able to slice and dice it in any way they know how. With that, we've probably all had to deal with exporting to Excel but have found it to be...unpleasant. This seems to be especially true when we try run these projects but have found we must have a template Excel file somewhere in order for SSIS to map to before it will populate it, even in debug mode. Well, it turns out there's a way for us SQL lovers to do it using some SQL logic and alleviate some headaches!

The problem with having a template is that it has to be in its expected location before SSIS will do its thing. If, for whatever reason, that file isn't there you may get a call at 3 AM for a job that failed as a result of the network backup process being interrupted by a bad NIC card, or because the Dread Pirate Roberts came and deleted the template. Whatever the reason, it's no fun getting that call when you should be able to ensure your package can run with as few dependencies as possible.

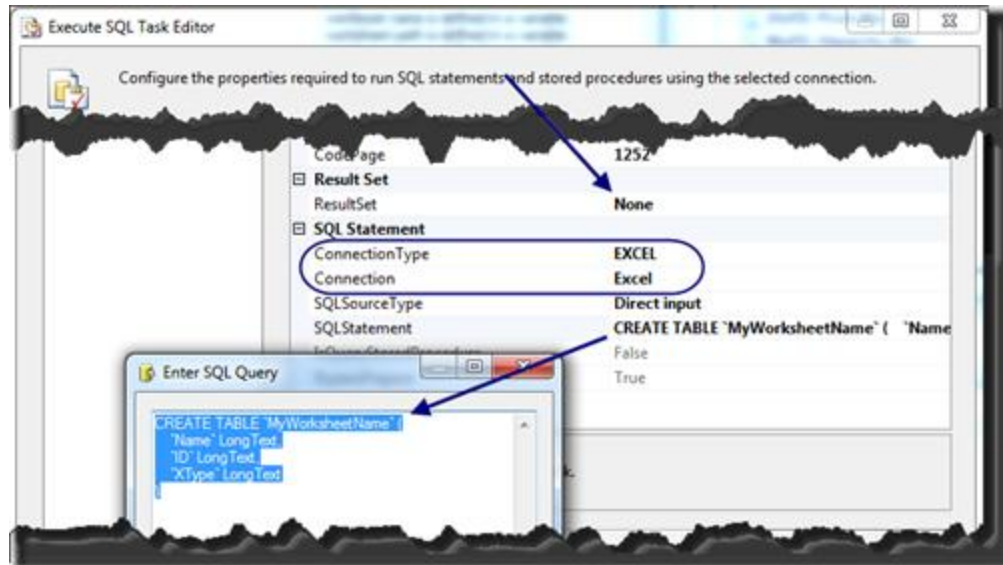
INSTANT SPREADSHEET

Enter 'SQL logic' stage left. By simply adding a couple of Execute SQL Task (EST) steps to your control flow, you can ensure that the Excel file exists for your Data Flow Task (DFT) to map to. You could even set it up to dump to a backup directory in the event the bad NIC card made your designated Reports network directory not available (but that's outside the scope of this post).

The first thing you'll need to do is to make sure the "Run64BitRuntime" property, under the project properties, is set to FALSE. Next, create a Data Source connection to the database of your choice. Now create an Excel Connection to a destination of your choice with the expected worksheet name and all the column names for what you'll be exporting – this is just for the initial setup of the project, later on we'll establish the final destination and not need this file.

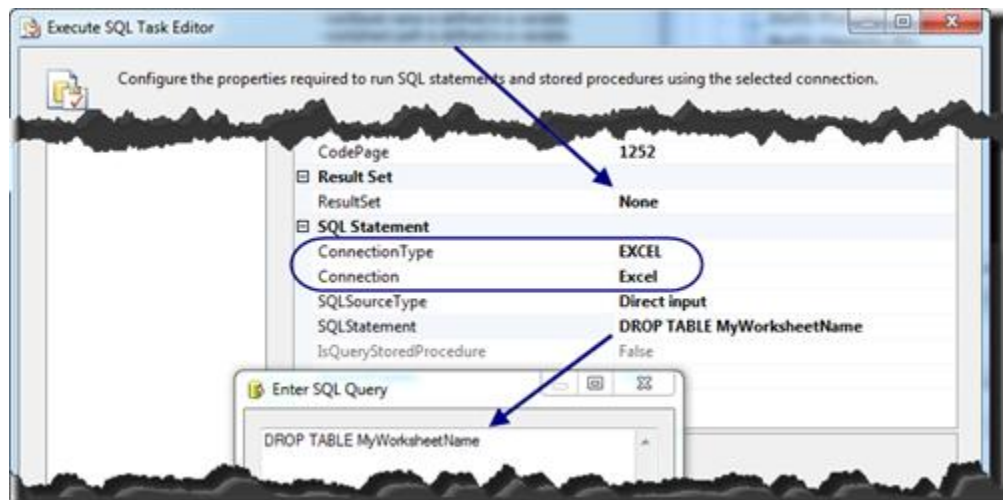
We're now ready to drop an EST onto your Control Flow tab. Using the editor, set the "Connection Type" = Excel and leave the "ResultSet" property = "None". For the "SQL Statement", put in the same CREATE TABLE logic you would when creating a table in SQL – keep in mind Excel will need everything in Unicode and most SQL data types will generate errors. This CREATE TABLE statement will need to be for all the columns you're exporting:

```
CREATE TABLE `MyWorksheetName` (  
  `Name` LongText,  
  `ID` LongText,  
  `XType` LongText  
)
```

Next we can create our DFT to use for exporting. In this step add your SQL source connection and do whatever manipulation of the data you'll need. For the destination, drop an "Excel Destination" down and set it up to map to the previously set up Excel connection (from the initial setup). Map to the worksheet name (a.k.a. table name) and map the columns. That's it, now you're ready to export!

In the event the spreadsheet already exists in your final destination folder during runtime, you will either want to delete it prior to creating it or just overwrite the data in it. To do the former, add a For Each File loop to your control flow looking in the directory for the expected spreadsheet name (including filename). In this you loop, add an EST with a "DROP TABLE MyWorksheetName" statement. Then have the CREATE TABLE step as the next step outside the loop.



By implementing this solution, you could save yourself, and the person who inherits the project, a fair amount of headaches by ensuring SSIS can see the Excel it's expecting.

EXCEL WRAP-UP

Using Excel with SSIS can be a pain for many reasons. Hopefully, Aaron's SSIS package is as helpful to you as it's been to those of use that have used this in the past. In the future, this pain no longer has to cause you frustrations.

EVERY WHICH WAY OUTPUTS

Saturday morning and you can watch your [favorite cartoon](#), or jump into some fun with SSIS. Today is the fifteenth day of the [31 Days of SSIS](#). This month I'm writing blog posts that cover some of the things I've learned and development techniques that I've used with SSIS.

One of my favorite transformations to use within SSIS packages is the [Script Component transformation](#). Back in the day with DTS, we had to use vbscript, and our solutions were quite limited with their capabilities. In this post, I want to take a look at one of the capabilities that would not be possible with vbscript that we are able to accomplish today.

MULTIPLE OUTPUTS REQUIREMENTS

Before we go into the solution, we need a problem. A problem that can be mapped to this solution and in which it also makes sense to choose this solution. For this problem let's start with a common task – a data export.

This data export is going to export some data from a single table. Depending on the data in the table, there are three possible text files that will be populated. The routing of the three text files will be based on the ProductCode column. The first file will receive all values less than 50 or starting with the letter "A". The second file will received all values 50 or greater or starting with the letter "B". The third file will receive all other product codes.

As contrived as these requirements may seem, they are nearly identical to a request that I received a few months back. Those requirements matched others that I have encountered throughout the years.

MULTIPLE OUTPUTS SOLUTION

There are a few alternatives that can be looked at for these requirements. We'll review a couple options and then discuss implementing one of the solutions. The solution that we implement will be the one I consider the most simple, extensible, and maintainable.

The first option, the one I expect most will default to, will be to to use a Conditional Split transformation. For those that don't know, this transformation performs a case statement across a number of expressions. For each case statement, an output is created from the transformation.

Now doing this might prove to be a bit tricky since the expression for the ranges will be a bit tricky. The expression will need to change the ProductCode value to an int to check the ranges or contain all of the numerical values in the evaluation. With expressions this complex, it would probably be easier to troubleshoot the package if the values were populated to a new column in the Data Flow with a Derived Column transformation. After the new column is added, then use a Conditional Split transformation to split out the rows.

The solution is a bit better, but not to the point where I would be satisfied. The complexity of the expression will make adding new values to the ranges or maintaining the expression difficult. Can you imagine having to update someone else's package that has this kind of an expression in it? After you curse out the former developer, you'll spend the rest of the day making the change and then validating your changes.

You could improve the flexibility of the package by moving all of the logic from the Derived Column transformation to a Script Component transformation. In this solution, you'd add in a new column on the Data Flow path and populate a value that the Conditional Split transformation can use, as was mentioned above. By using .NET code you will be able to break apart the logic to split the rows into cleaner IF-THEN statements, which the next

developer will find fairly easy to maintain and troubleshoot. Also, since you are in a development environment, data types can be easily changed and ranges can be compared versus a list of values.

At this point, though, there is still one more change that can be made. This change is the real point of this entire post. The previous solutions would determine the path to use and then use a Conditional Split transformation to split apart the rows. That transformation really isn't needed. Instead the Script Component transformation can be configured to split the rows and redirect them within itself. This is the solution that we will be going throughout the rest of this post.

PACKAGE CONTROL FLOW

The focus of this SSIS package will be in the details of the Script Component transformation. To get to that transformation, add a Data Flow to the Control Flow.

PACKAGE VARIABLES

There are no variables for this solution. ~~Freaky...right?!~~

PACKAGE DATA FLOWS

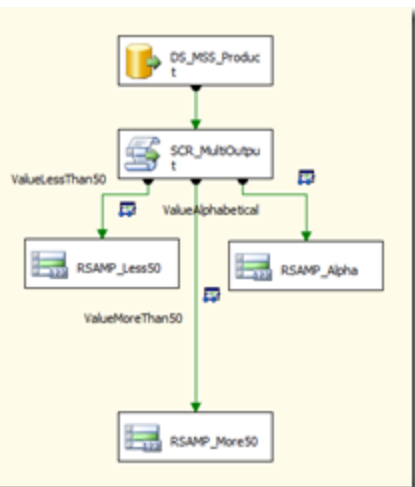
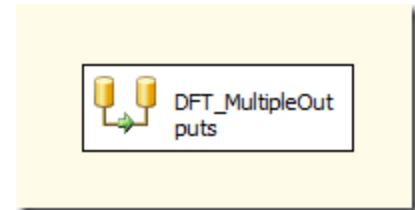
There hasn't been a lot to this package thus far, and the data flows will be no different. There is just a single data flow for this package.

DFT_MULTIPLEOUTPUTS

Within this data flow, data will be imported into the data flow. A Script Component transformation will split that data along three outputs. These three outputs will move data into three Row Sampling transformations.

The transformations in the data flow are:

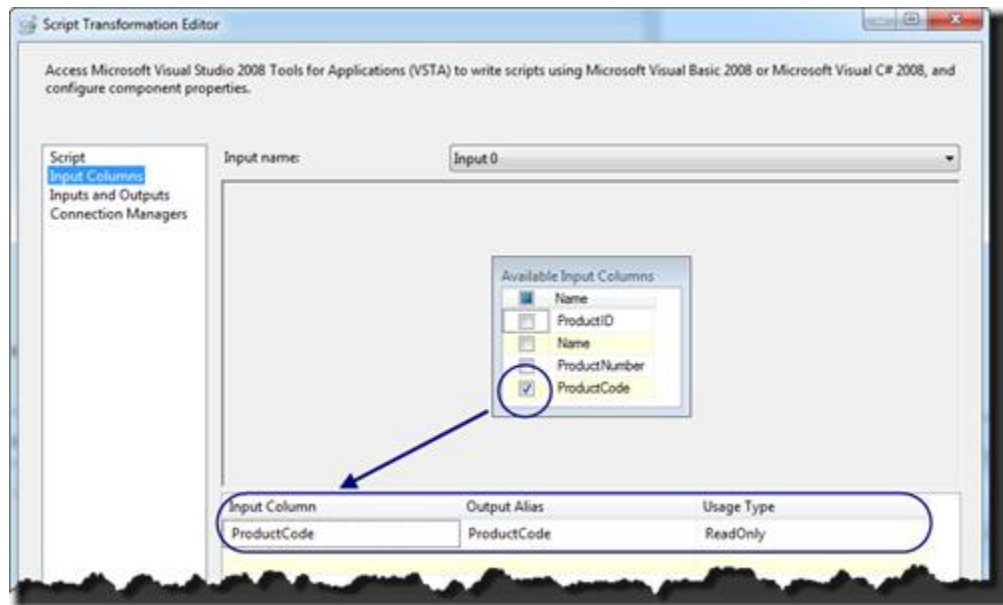
- **DS_MSS_Product:** OLE DB Source that retrieves data from the AdventureWorks database. The data source brings product data into the data flow.
- **SCR_MultiOutput:** Script Component transformation with multiple outputs. Product data is processed by the transformation to send the data to one of the three output options.
- **RSAMP_Less50:** Row Sampling transformation that was included to allow a data flow path and data viewer for rows with ProductCode between 0 and 50 or ends with the letter "A".
- **RSAMP_More50:** Row Sampling transformation that was included to allow a data flow path and data viewer for rows with ProductCode that is 50 or more or ends with the letter "A".
- **RSAMP_Alpha:** Row Sampling transformation that was included to allow a data flow path and data viewer for rows with ProductCode that is non-numeric and doesn't end in "A" or "B".



SCR_MULTIOUTPUT CONFIGURATION

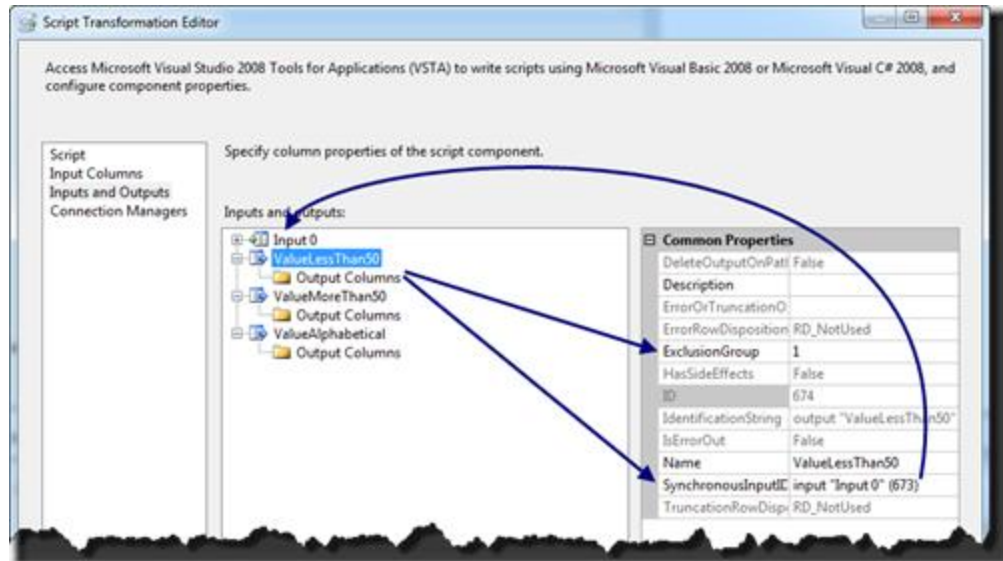
The main item to configure in this package is the SCR_MultiOutput Script Component transformation. The other items are fairly simple and can just be copied from the package included below.

To begin configuring the script component transformation, open the transformation editor. Select the **Input Columns** tab. On this tab, select **ProductCode** from the **Available Input Columns**.



After that change is made, choose the **Inputs and Outputs** tab. In this tab create two new outputs and rename the existing output. These should be **ValueLessThan50**, **ValueMoreThan50**, and **ValueAlphabetical**.

Once these are created they will need two additional changes made to them. The first is to set the **ExclusionGroup** property to 1. This property dictates whether the output is a distinct group of output rows; for any row to appear in this output the row must be directed to it. Second, change the **SynchronousInputID** to the **Input 0** for the transformation. Doing this will give the output the same metadata as **Input 0**.



These steps will modify the Script Component transformation to provide three outputs. If you close the transformation it can be connected to other transformations; such as the Row Sampling transformations mentioned above.

If you closed the transformation, open the editor for it again. Select the Edit Script button on the Script tab. A Visual Studio 2008 Tools for Applications design window will open next. There are a few functions in the window, the one we will be modifying is `Input0_ProcessInputRow`.

First let's replace the code for Input0_ProcessInputRow with the code below. After that, I'll review what the code will accomplish.

[illegible]

```

Row.DirectRowToValueMoreThan50();
}
else
{
Row.DirectRowToValueAlphabetical();
}
}
}

```

First, an attempt is made to convert the value for ProductCode to an int. This returns a true value if the ProductCode is an int and populates the value to iValue. If iValue is less than 50 then the row is directed to the ValueLessThan50 output through the DirectRowToValueLessThan50 method. This method is created when the ExclusionGroup property was set for the output. If iValue is 50 or above, then the row is directed to the ValueMoreThan50 output with the DirectRowToValueMoreThan50 method.

When the TryParse returns a false value, then the second code block is checked. If the ProductCode value ends with “A” then the DirectRowToValueLessThan50 method is called. When the ProductCode value ends with a “B” then the DirectRowToValueMoreThan50 method is executed. Finally, the remaining rows are sent to the ValueAlphabetical output with the DirectRowToValueAlphabetical method.

MULTIPLE OUTPUTS WRAP-UP

At this point the Script Component transformation configuration is completed. The transformation will now split the rows between the configured outputs.

But why choose this solution over the other solution options? There are three reasons, which were previously named, for going with this route:

- **Simple:** Instead of two or more transformations that together implement this logic there is only a single transformation. At times the need to implement logic over multiple transformations can muddy the waters within the package. Since this is all contained in a single transformation, outside dependency issues are avoided.
- **Extensible:** When the rules change and the ranges need modification, the logic in the Script Component can be easily modified and new outputs can be added without needing to configure multiple transformations.
- **Maintainable:** The SSIS expression language is easy to use, provided you know all of the “gotchas” and things that will bite you. ~~Stay tuned to tomorrow’s post for more on that.~~ When you look at the staff in your environment, there are likely more people that can read and write C# code than can do the same with the SSIS expression language. This solution lowers the bar on the people that can review, write, and troubleshoot the SSIS package.

I’ve said it before and will say it again, Script Component transformations are extremely useful and open up the possibilities for what can be done within SSIS. Be sure to consider them in your solutions, especially when the logic includes many transformations. There may be a way to consolidate the logic and reduce your headaches.

10 THINGS ABOUT EXPRESSIONS

Sixteen days in and we are past the half way point for the [31 Days of SSIS](#) blog series. In a few of the previous post I made reference to the use of the expression language within SSIS. Today I'm going to cover ten things that you should probably know about them.

The SSIS expression language is an interesting blend of T-SQL and Visual Basic functions. Some work in ways we expect and others do not. And while a lot of it is familiar enough that we can get by, there is some things that are a bit foreign and exclusive to SSIS.

1. EXPRESSION EVALUATION

This first one is one that I learned the hard way. Expressions are evaluated at the time in which the expression is accessed. This means that each time you reference the expression it's value will be determined. This has an impact when you use variables that change during execution and what their output will be.

Also, if you have a variable with a value that is defined by an expression, the expression will override the value that you set for the variable. I've fallen for to this a number of times in the past. Setting a variable value, either through BIDS or at run-time with DTEXEC, will have no effect if the variable value is evaluated by an expression.

2. USE SYSTEMSTARTDATE FOR TIMESTAMPS

As mentioned above, expressions values are determined when the expression is evaluated. Because of this, the GETDATE() function in SSIS will provide the current date and time each time it is called and the date and time at the beginning of the package execution will be different that at the end of the package execution. As an alternative to GETDATE() function, use the SystemStartDate variable. This variable is set at the start of the package execution and static throughout the package's execution.

3. CONDITIONAL LOGIC

Inevitably, at some point when writing expressions you will want to perform some sort of conditional logic. With T-SQL and Visual Basic, we can write IF-THEN-ELSE code blocks. In Access, we have the immediate if (IIF) function. And in Excel and C# there is the use if IF(). Of course, none of these were good enough for SSIS.

Instead in SSIS, the expression language uses ? and : for conditional logic. These are written as:

boolean expression ? true value : false value

After writing a boolean expression the question mark is use to indicate that the boolean expression should be evaluated. If the boolean expression is true, then return the first value. The colon is used to indicate the false value.

4. AND AND OR

A few times I've encountered package with multiple data paths coming out of Conditional Split transformations which are subsequently merged back together with Union All transformations. The cure for this is usually a quick tutorial on how to use the AND and OR.

Quite simply, AND translates to && and OR translates to || within the SSIS expression language. And erroneous expression might look like this:

```
@[User::Value1] == 1 AND @[User::Value2] == 1 OR @[User::Value3] == 1
```

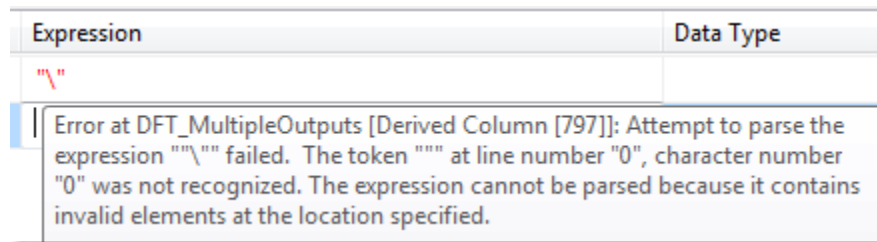
But can be fixed as the following:

```
@[User::Value1] == 1 && @[User::Value2] == 1 || @[User::Value3] == 1
```

Make sure you utilize AND and OR operators rather than adding additional needless transformations.

5. BACKSLASH

Whenever you need to add a backslash into an expression through string concatenation, you may come upon an unexpected error. When backslashes are entered, they will cause the expression to error with the following error:



Long and the short, backslashes require an escape from themselves. The remedy to this is to double up your backslashes.

6. TOO MANY PARENTHESIS

Parenthesis are nice. They take portions of your expressions and group them together. This allows you to both visually and logically control the expression. But what you may not know is that you can go too far with parenthesis. Take the following expressions for example:

```
(100 - (@[User::Value1] + @[User::Value2])) == 0 ? 0 : ((@[User::Value3] / (100 -  
@[User::Value1] + @[User::Value2]))) * 100)
```

Place this in a variable expression and it will compile and works nicely. But try and run some values in it and you will occasionally get a divide by zero error. The logic is correct but the parenthesis seem to confuse the expression when it executes.

Drop out some of the parenthesis and you will have not problem. Changing over to the expression below removes the issue:

```
100 - @[User::Value1] + @[User::Value2] == 0 ? 0 : @[User::Value3] / (100 -  
@[User::Value1] + @[User::Value2]) * 100
```

7. TOO FEW PARENTHESIS

Of course, you need to be careful not to have too few parenthesis. A very common mistake that I will see with too few parenthesis is when the results of condition logic is being returned. The goal might be to return a 100-character string value with the following logic:

```
(DT_STR,100,1252) "1"=="1"? "1" : "0"
```

The trouble with this expectation is that the data conversion at the start of the expression only affects the boolean expression. It does not affect the output from the true or false values. Instead, the expression will return a 1-character unicode string.

A simple rewrite with a couple of parenthesis and the expression runs as expected:

```
(DT_STR,100,1252) ("1"=="1"? "1" : "0")
```

It doesn't take much to fix this. The key to parenthesis is to only use them use you need them and to make certain the encapsulate the intended targets.

8. DATE FUNCTIONS

With SSIS packages you won't make it very far without using some form of date functions. Trouble is we have most of the T-SQL date functions but we don't use the same methods for passing in the parameters. The difference here is that the parameters for date portion of date functions are included within double-quotes. Instead of DATEADD(d, 1, GETDATE()) the expression would be written as DATEADD("d", 1, GETDATE()).

Another thing to mention with date functions is the lack of the CONVERT functionality in T-SQL. That handy function can easily reformat date and time values in T-SQL. Unfortunately, expressions require the use of DATEPART (or similar functions) to pull all of the date values apart and then to have them concatenated back together.

9. MISSING FUNCTIONS SUCK

There are some functions that you will want that are not available in the expression language. Some functions you may look for that are not available are:

- LEFT
- RIGHT
- STUFF
- ISDATE
- ISNUMERIC
- MONTHNAME

There are usually two possible outcomes because these functions are missing. First, to get around them we often need to use other functions (SUBSTRING vs. LEFT/RIGHT) to get to the results. This indirect method is perfectly valid but makes the code slightly more difficult to decipher. Other times, we will need to move logic from Derived Column transformations in the Script Component transformations. This would be the case for ISDATE and ISNUMERIC functions.

10. CONCATENATION

Let's not finish with bang. For this last item, when you need to concatenate values together in an expression, you will be using the plus (+). Fortunately, the syntax matches what we are used to with T-SQL. This isn't terribly interesting or difficult to remember.

The other piece with concatenation, that is similar to T-SQL, is that it does not perform implicit conversions. If you have a integer and a string, the integer will need to be converted to a string before the values can be concatenated.

EXPRESSION WRAP-UP

All of the expression items above are things that I often find myself explaining and teaching to others. The expression language isn't so different from T-SQL but there are enough differences that each adds to a developers frustration and can result in people giving up.

Do you have any pointers that you would recommend to people when working with expressions? Please, write them in the comments below.

ENVIRONMENTAL VARIABLE

It's time to go back to discussing the environment when it comes to SSIS packages. For the last few posts of the [31 Days of SSIS](#) series I've talked about developing packages. As important as developing packages is, we must be prepared to develop them in a way that they can be securely deployed.

As part of this need, this post will focus on the use of [environmental variables](#) with SSIS packages. Environmental variables allow variables to be specified and set within an operating system environment. The value is available to all applications and exists from one reboot to another. A very common example of an environmental variable is TEMP variable which directs the operating system to use C:\Window\Temp. At least, that is where it is set on my Windows 7 laptop.

WHY ENVIRONMENTAL VARIABLE?

Since environmental variables are specific to an environment, they can be used to store information specific to that environment. In this context of the term "environment", I am referring to your development, test, and production environments. Each of these will have some static values that are dependent on the environment.

When looking at these environments, there are some common factors that all SSIS packages will have regardless of the purpose of the package. For instance, all of the SSIS packages that reference the AdventureWorks database in the development environment will use one server. All of those same SSIS packages will reference a different server or instance when they are executed in the test or production environment. These settings that are specific to your environments are the configuration values of SSIS packages that are prime candidates to use with environmental variables.

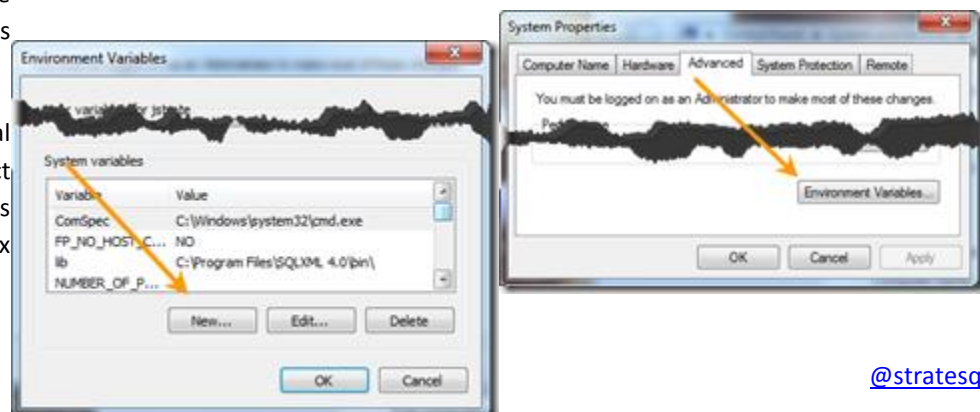
When environmental variables are used properly with SSIS packages, they can be deployed to any environment and perform within the confines of that environment. This when an SSIS package is developed on my workstation, the local environmental variable can reference the development servers. When I deploy it to the test environment, the testers can execute the package and it will automatically reference the test environment. Once they validate the package, it can be deployed to production and the changes for that environment will just occur.

ENVIRONMENTAL VARIABLE SETUP

Let's suppose, as mentioned above, we have a number of SSIS packages that will be referencing the AdventureWorks database. We will want the packages that run on my laptop to point to a local instance. When executed on other servers we will need to point to other instances or servers.

In order to accomplish this we need an environmental variable and to configure it in an SSIS package. To start this setup, open the System Properties window for the desktop or server that needs to have the environmental variable created. This can be found in Computer → Properties → Advanced system settings. From this window choose the Environmental Variables button.

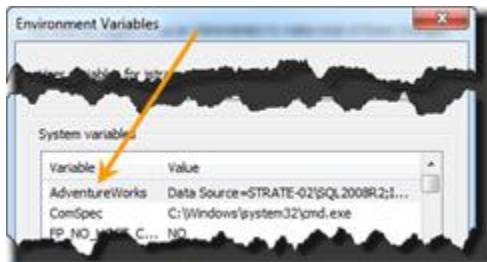
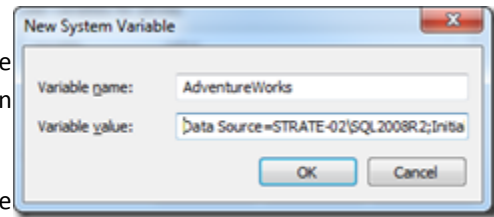
From the Environmental Variable window select the New... button. This will open a dialog box



where you are able to type in the name of the variable and its value. For this demonstration, name the variable AdventureWorks. That value should be – Data Source = STRATE-02\SQL2008R2; Initial Catalog = AdventureWorks; Provider = SQLNCLI10.1; Integrated Security = SSPI; Application Name = EnvironmentalVariable;.

Of course, that connection string will be modified slightly to reference your own SQL Server. I'm pretty certain my server is locked down enough to prevent you from gaining access to it.

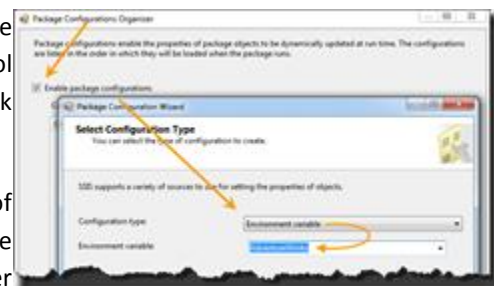
After this is done, the environmental variable will be available in the list of variables for the environment. If you used the variable named



above, it will appear at the top of the list in the Environmental Variables window. Unfortunately, there is one more task that needs to be completed to finish setting this up. Your desktop, laptop, or server will need to be restarted. Until then the variable will be there but it won't be accessible to SSIS. Forgetting this can be a pain so I will reiterate – reboot after you setup the variable.

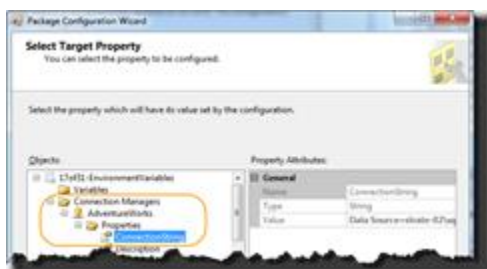
Now that you have a variable to work with, open up an SSIS package. Add a connection to AdventureWorks in Connection Managers. From here we are ready to begin.

Open the Package Configuration Organizer window. This can be accessed through the menu at or by right-clicking in the Control Flow. In this window select Enable package configurations and click the Add... button.



In the Package Configuration wizard, choose the Configuration type of Environmental variable. The drop-down box will list the AdventureWorks environmental variable along with all other

environment variable on the system. Select this item and click Next.



On the next screen, you need to associate the environmental variable with a property in the SSIS package. The property we will want is Connection Managers -> AdventureWorks -> Properties -> ConnectionString. Browse to this item and then select Next.

The last screen will allow you to name the configuration setting. It can be named anything that you'd like. Often I choose to name the configuration after the environmental

variable variable for clarity.

Once you click on Finish, the setup of the environmental variable will be complete. All you need to do now is set this up in your other environments and drop the package in and run it. As long as the connection string is pointed to the correct server, this and all other similarly configured packages will have one place that controls how they execute in each environment.

ENVIRONMENTAL VARIABLE WRAP-UP

I find using environmental variables for directing connectivity of SSIS packages to be one of the more perfect uses for it. As long as everyone is on the same page, a lot of the work in deploying packages is removed and you can focus on the features of the SSIS packages rather than the functionality.

The other benefit, as mentioned in the last section, is consolidation of configuration information. Rather than having 5, 10, 20, or more packages to edit when a server moves or information changes – the environmental variable allows for a single point of configuration for the entire environment. One edit and everything starts working with the new configurations.

SQL SERVER CONFIGURATION

The last post in the [31 Days of SSIS](#) talked about the use of [environmental variables](#). As I mentioned in that post, environmental variables are great for values that are specific to individual environments

One problem with environmental variables that I mentioned is often the operating system needs to be rebooted after adding new environmental variables. In the real world SSIS packages will be deployed more often than you probably want to go around rebooting your servers. Due to this an alternative to environmental variables is required. The alternative we will be discussing today is SQL Server configurations.

WHY SQL SERVER CONFIGURATION?

SQL Server configurations allow properties in SSIS packages to be set in a table and retrieved at run-time. When the package is designed, the items are selected through the configuration manager and inserted into the configuration tables.

Conceptually the properties set through environmental variables and SQL Server configurations are the same. Both are properties that are set at the package level that will be used for all executions of the package within that environment. Both need to be added to the package at design time.

The purposes and intent behind using each of these is a slightly different. These differences are due to where the configurations are located and how they are set. SQL Server configurations changes can be inserted into the configuration table; after which they are immediately available. As opposed to environmental variables which require the reboot.

SQL Server configurations are more secure, in my opinion. Since anyone logging onto a server can see environmental variables. To get to the SQL Server configuration you need have access to the database and the table. With transparent data encryption the configuration can be secured one step further.

These differences point help provide guidance on how each should be used. SQL Server configurations are a good fit for configuration changes that can change at deployments, between executions, or because the sky is blue. The typically are associated to only as single or a few packages. On the other hand, environmental variables are more appropriate for properties that change infrequently – such as during SQL Server upgrades or hardware migrations. They are also a good store connection strings for databases that are accessed through Windows authentication.

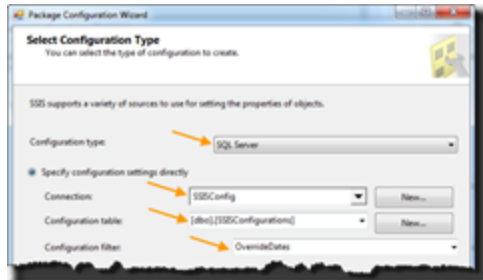
SQL SERVER CONFIGURATION SETUP

At this point, there's been enough talking about SQL Server configurations. Let's dig back to the SSIS package from the [Override Variables](#) post and configure it to use SQL Server configurations. For the first couple steps I'm not going to guide you through them since they are either basic or are part of previous posts.

Start by creating a database for the SQL Server configurations to be stored in. I suggest naming that database SSISConfig, ~~because it's so obvious~~. Next add an [environmental variable](#) to your machine named SSISConfig with the connection string to the SSISConfig database. As I mentioned, long term values go in environmental variables; like the SSISConfig connections string. Lastly, add a connection to SSISConfig in the Override Variables SSIS package, found above. At this point, we are ready to begin setting up a SQL Server configuration.

First open the **Package Configuration Organizer** window from the main menu or right-click menu in the Control Flow. Once open, select **Enable Package configurations**. And then select the **Add...** button.

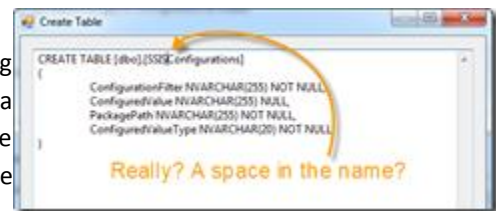
Once this is done, we need to setup the connection string to the SSISConfig database on the SSISConfig connection. Do this through the use of environmental variables and map the value to the connection string for the connection.



Now that the SSISConfig connection is setup, we need to start adding SQL Server configurations to the SSIS package. Start similarly to setting up the environmental variable by clicking the **Add...** button on the **Package Configurations Organizer**.

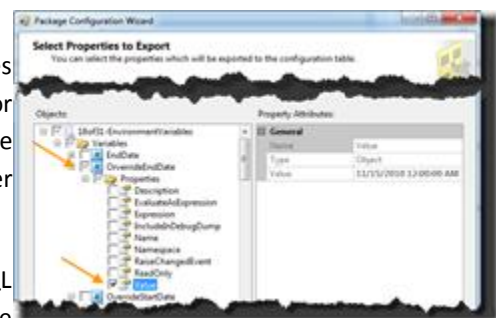
In the **Package Configuration Wizard**, choose the configuration **Configuration type** of **SQL Server**. Then change the **Connection** to **SSISConfig**, it will likely default to AdventureWorks due to the way the alphabet works, ~~captain obvious~~.

The Configuration table list will be empty if this is your first time using SQL Server configurations. Click the New... button to create a configuration table. Typically I use the table suggested, minus the space in the name. Change Configuration table to the name of the table you created.



The next piece to add is the Configuration filter. This can be anything that you want it to be. Make sure it describes what you are adding to SQL Server configurations. It should be unique enough that it isn't confused with configuration information. It should also be general enough that it can be shared between packages when it is applicable. For this example, set the filter to OverrideDates. With the filter set, click Next....

On the Select Properties to Export screen browse to the variables within the package in the properties tree view. Find the property for OverrideStartDate and OverrideEndDate. Expand both of these properties and select the Value property to add to the SQL Server configuration. The click on Next....



On that last page of the wizard you need to add a name to the SQL Server configuration. The name can be anything that you would like it to be. For simplicity sake, I usually give it the name of the SQL Server configuration filter.

Run the package and you will see that it executes similar to how it did previously. If you edit the values in the configuration table, you will get

SQL SERVER CONFIGURATION WRAP-UP

SQL Server configurations are my preferred method for configuration environment specific values for an SSIS package. They are easy to manage and can be monitored and secured without much effort.

There is an additional benefit that SQL Server configurations have over environmental variables. Once a configuration has been setup, bringing those configurations into an SSIS package doesn't require the extra work to map the values. SQL Server configurations map to the XML path for the property and automatically know what properties to modify because of that.

One thing I didn't point out, that is worth noting, is the data within configuration filters can be changed without having to edit a package. If you miss a property that needs to be set at run-time, you can put together the configuration path to the property and insert the property to the table. The next time the package runs, it will pick up the new property. This, of course, assumes that you already have a SQL Server configuration setup on the package.

DTSCONFIG CONFIGURATION

The last two posts in the [31 Days of SSIS](#) talked about the use of [environmental variables](#) and [SQL Server configurations](#). Between those two types of configurations, you can cover most situations where you need to configure an SSIS package.

There is one more type of SSIS package configuration that I'd like to make sure that I cover in this series. That is the XML configuration file. XML configuration files are, usually, setup at design time; which is then be applied at run-time. I say usually because if you need to ninja in a configuration change after a package is deployed, you can create the XML configuration independent of the SSIS package and apply it to the package without having to associate it to the package.

In essence, XML configuration files act pretty much like config files for .NET applications. They list out a number of properties and apply them to the SSIS package. The XML configuration files are broken into two parts, the heading and the configuration details

HEADING

The heading lets you know who created the XML document and when, and for which package. This information isn't used when the package executes, and I don't put much effort into making certain this always relates to the package. This is mostly because it doesn't matter, but also because I share XML configuration files between packages (I'll talk more about this later).

The heading, along with the first couple tags of the configuration looks like this:

```
<?xml version="1.0"?>
<DTSConfiguration>
<DTSConfigurationHeading>
<DTSConfigurationFileInfo
GeneratedBy="Just tell everyone StrateSQL told you to do this."
GeneratedFromPackageName="value doesn't matter"
GeneratedFromPackageID="{value still doesn't matter}"
GeneratedDate="1/19/2011 12:00:00 PM"/>
</DTSConfigurationHeading>
```

There is information on who created, which package, the package ID, and the date generated. As I mentioned, this information is just informational and has no bearing on execution.

CONFIGURATION

The configuration portion of the XML configuration file is where all of the pertinent details for configuring your SSIS package will be kept. The configuration contains the path to the property and the value for the property. Each configuration will look like the XML fragment below:

```
<Configuration ConfiguredType="Property" Path="\Package.Variables[User::ClientCode].Value"
ValueType="String">
<ConfiguredValue>Z0M8I3</ConfiguredValue>
</Configuration>
</DTSConfiguration>
```

Your XML configuration files will have a string of configuration nodes put together with the element at the bottom to close the document.

WHY XML CONFIGURATION?

Why would we want to use XML configurations when we can easily use environmental variables and SQL Server configurations? Those two seem to cover the gauntlet of what we need to configure. Do we need a third option?

We do. This is needed because there was a common theme I included in the other two configuration types that limit their usefulness. Environmental variables and SQL Server configurations are applied at the environmental level; which means that every execution of the SSIS package will use those values. If a package needs to run a certain way always, these two will fit the bill.

If you are like me, though, you try to write SSIS packages that encapsulate business processes and are generic enough that they can be reused in different scenarios that utilize that process but not necessarily the same conditions.

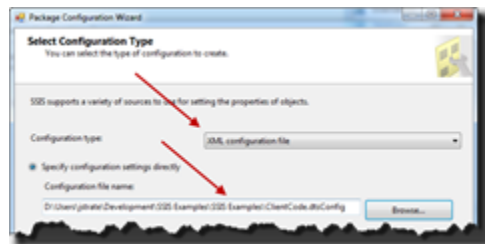
The perfect example of this is when your business has multiple clients send you data. Do you write one process for all of your clients or one process that all your clients can use? I prefer the later and develop as many of my packages as possible with that idea in mind.

XML configurations provide just the vehicle for filling this need. You can have one version of the configuration file for client [ZOM8I3](#) and another version for the [B4C0N](#) client. When you execute the package, just pick the correct XML configuration file and the package will run under that setup.

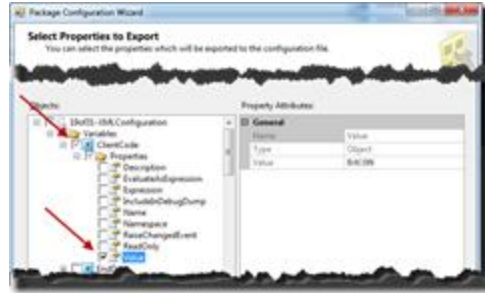
Along with using multiple XML configuration files with an SSIS package, you can also use a XML configuration file with multiple SSIS package. This helps reduce the number of configuration files needed and reduces the overhead to choosing this type of configuration.

XML CONFIGURATION SETUP

Setting up XML configurations is similar to how environmental variables and SQL Server configurations are setup. To start, open the **Package Configuration Organizer** window from the main menu or right-click menu in the Control Flow. Since we already have configurations setup we don't need to enable **Enable Package configurations**. If it isn't, do that now. Then select the **Add...** button.



On the next screen select **XML Configuration File** from the **Configuration type**. Then choose a file name in the **Configuration file name**. Don't be overly concerned with this name since in the grand scheme of things it may not matter. More on that later. After you have a file name, click the **Next...** button.



The next screen is **Select Properties to Export** where the properties that will be configured will be selected. For this example, select the **Value** property of the **ClientCode** variable. This will place the property into the XML configuration file. If there are others to add, they can be added to the configuration file as well. When you are finished, click the Next... button.

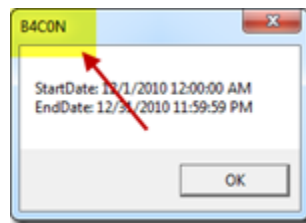


FIGURE 1 GET BACON

On the last screen, give the XML configuration a meaningful name. Then click the **Finish** button.

SETUP COMPLETE

Now that the setup is complete, we should execute this a couple times. First, just try running it as is. The value for ClientCode from the XML configuration file will be the same as the SSIS package. When it runs you'll get a message box with the B4CON in the upper corner.

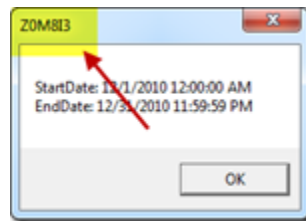


FIGURE 2 GOT ZOMBIES

Browse out to the XML configuration file that was previously created and change the value for ClientCode in that file from B4CON to ZOM8I3. Save the file and then execute the SSIS package one more time. Amazingly, and as expected, the value that will be returned in the upper corner of the message box will be ZOM8I3.

XML CONFIGURATION WRAP-UP

That about does it for setting up XML configuration files. Before finishing this post, there are a couple things that need mentioning regarding the use of XML configuration files.

To start, avoid the temptation to only use these to configure SSIS packages. Keep environmental configuration data in environmental variables and SQL Server configurations. XML configuration files should contain information that is specific to that particular execution of the SSIS package. Not the values that will be set for every execution.

Also, when you deploy your SSIS packages you will likely not have the [same folder structure](#) in that environment as exists on your development machine. This is to be expected. This can cause an validation message to appear when you run you SSIS packages that looks like the following:

Information: The package is attempting to configure from the environment variable "SSISConfig".

*Information: The package is attempting to configure from SQL Server using the configuration string
""SSISConfig";"[dbo].[SSISConfigurations]";"OverrideDates";".*

Information: The package is attempting to configure from the XML file

"D:\Users\jstrate\Development\SSIS Examples\SSIS Examples\ClientCode.dtsConfig".

This means that the SSIS package couldn't find the XML configuration file where you configured it to be. This isn't terrible. Since you will associate the XML configuration with the SSIS package through the DTEXEC command line or SQL Agent job the error isn't a big deal. You can configure your packages to look for the files in a common location that will be in all environments or do what I do; which is to delete the configuration entry after the XML configuration file has been created. Remember, I mentioned that I share XML configuration files between SSIS packages. I don't create a new one to share for each package each time I think about sharing them.

NO MORE PROCEDURES

Time to shift off of configurations and environments and talk more about SSIS packages and best practices. If you haven't been following this series and are just joining in, we are at day 20 of the [31 Days of SSIS](#). Now that is out of the way, let's move on.

Today, we are going to discuss a recent best practice that I've been promoting surrounding the use of stored procedures as source objects for data flows. Or to put it more succinctly that in **SSIS packages data sources should use table-values functions instead of stored procedures for the SQL Command text**.

BEST PRACTICE ROOTS

A couple months ago, I came across the following error message:

SSIS Error Code DTS_E_OLEDBERROR. An OLE DB error has occurred. Error code: 0x80004005.
An OLE DB record is available. Source: "Microsoft OLE DB Provider for SQL Server" Hresult: 0x80004005
Description: "Protocol error in TDS stream".
SSIS Error Code DTS_E_PRIMEOUTPUTFAILED. The PrimeOutput method on component "OLE DB Source" (7) returned error code 0xC0202009. The component returned a failure code when the pipeline engine called PrimeOutput(). The meaning of the failure code is defined by the component, but the error is fatal and the pipeline stopped executing. There may be error messages posted before this with more information about the failure.

The odd thing about this error is that it only occurred when executing the SSIS package through DTEXEC or SQL Agent. Running the package in BIDS presented no issue and the package was always successful. Redeploy the package and the same issue occurred.

After some research, I found a post on [Protocol error in TDS stream](#). From there I went to the post [Stored procedures are not parameterized views](#) by Adam Machanic ([Blog](#) | [@AdamMachanic](#)). And finally, I checked out Jamie Thompson's ([Blog](#) | [@JamieT](#)) post [SSIS: Using stored procedures inside an OLE DB Source component](#).

To summarize what these posts talk about, stored procedures do not have a defined output schema. The output can change depending on the parameters supplied. Furthermore, the metadata that SSIS uses may not be based on the execution path that you intend to use when SSIS package executes.

OUTPUT CONTRACT

When SSIS packages are built there needs to be a reasonable expectation that the SQL command text for the data source will provide the columns that the SSIS package requires. It would be great if stored procedures could be designed in a fashion that allowed for one and only one output. This isn't the case and there are no indications that this will change and probably no true need for it to change.

There is, though, a defined metadata output for table-valued functions. As they say in the Highlander, "there can be only one." Whether the table-valued function is inline or multi-line, there is always a defined set of columns that will be returned. The SSIS data source knows specifically what will be returned from the function.

It's still true that someone can change the table-valued function and cause similar issues to occur after deployment. There is the insurance that side effects from table changes will not negatively affect the execution of the SSIS package with the obscure message above that seems more ominous than it actually is.

BEST PRACTICE WRAP-UP

Having read this, you may be wondering if this is much ado about nothing. If I hadn't uncovered this issue a few months back I might be thinking the same time. I've used SSIS since SQL Server 2005 was released and that was the first time I recall having this issue.

There was time between now and when I originally encountered this issue. With one of my clients there was a major release between when I first encountered the error and now. Would you be surprised to find out that we ran into the same issue three more times? Working on these issues after a release is no picnic and had I followed the advice I give now these issues likely would not have happened.

Guess what I recommended so that we wouldn't deal with these again in the future? Now, that you've heard my recommendation, what do you think?

CONTROLLING YOUR FLOW IN THE CONTROL FLOW

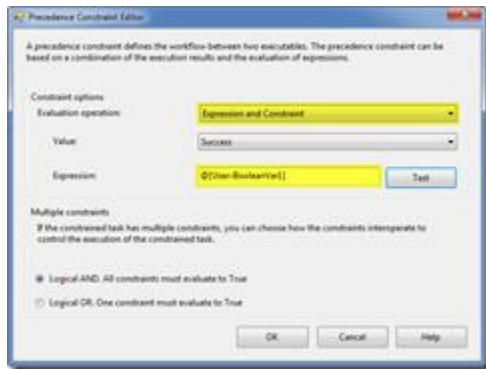
Remember, [what I said last Friday](#)? I'm going to do it, again. Which means, I'm taking the day off. There will still be a [31 Days of SSIS](#) post, you're reading it right now.

This post was being written by Chris Fish ([LinkedIn](#)). He's another of my co-workers at [Digineer](#) and is on the team with me that works with SSIS and on SQL Server projects.

GUEST BLOGGER

Hello, and welcome to my first attempt at writing a blog entry. I'd like to thank Jason Strate for letting some of us commandeer his platform on Fridays. The topic I am writing about is, how to deal with some odd Control Flow scenarios.

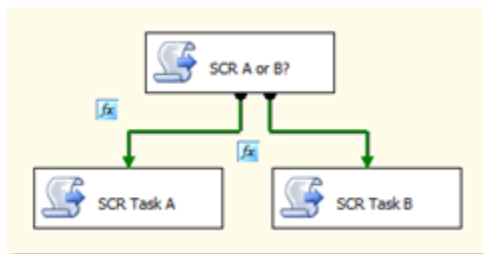
WHAT THE FLOW?



I recently had a need to split the flow of tasks based on the results of an earlier task. So I went to the tool box to get the Conditional Split Control Flow Task. But wait, WTF? (that's Why The Face, by the way) Where's the Conditional Split? I even had Stan, a Digineer co-worker, look and he had the same reaction. There is no Conditional Split for the Control Flow! Instead, the same functionality is accomplished by adding expressions to the Precedence Constraint. Double click on the arrow between the tasks and the Precedence Constraint editor opens. Change the Evaluation operation to a value that includes "Expression" and then enter an expression that evaluates to true or

false.

An annoying un-feature, is there is no expression builder available in this modal window so you have to either type in everything from memory or copy and paste from somewhere else. The expression field is case sensitive as well.

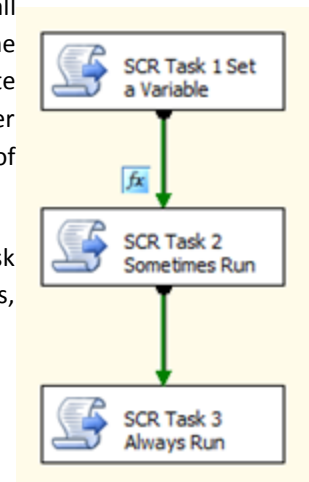


After setting up this Precedence Constraint, you'll see an  by the arrow. That alerts you that the Precedence Constraint has an expression associated with it. To have a true conditional split, you'd set up other Precedence Constraints with the appropriate expressions. Keep in mind that you need to include all of the logic in your expressions to limit the precedence. The control flow will evaluate

each branch as its own flow and will act as a multicast if you're not careful. In other words all branches with expressions that evaluate to true will continue, regardless of what the other branches are doing.

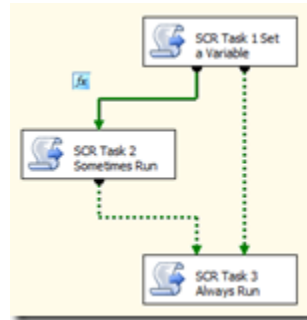
For readability and another reason I'll get into later, I prefer to add a blank Script Task that has a meaningful name in front of any Precedence Constraints with expressions, especially if it is meant to act as a conditional split.

KEEPING WITH THE FLOW

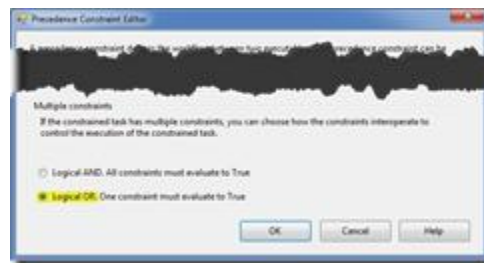


Let's go a step further. What if you need to conditionally run a task, but want to run another task after it, whether you run the conditional task or not? In other words, let's say you have 3 tasks. Task 1 sets a variable that is used to determine if Task 2 is run. Task 3 needs to all the time, but needs to run after Task 2 if Task 2 runs.

This won't work because if Task 2 doesn't run, Task 3 won't run. You could set up your tasks like this:



And use the Logical OR option in the Precedence Constraint like so:



But that won't guarantee that Task 3 will run after Task 2 if Task 2 needs to be run. This would simply run Task 3 as soon as Task 1 is complete, regardless of what Task 2 is doing. This can be handy to force some processes to run a certain way, but doesn't meet our requirement.

IN SEQUENCE

Enter the Sequence Container. By wrapping the conditional tasks in a Sequence Container, the flow will run Task 2 when it needs to and when all tasks are complete within the container, move on to Task 3. If Task 2 doesn't run, the tasks in the container are still considered complete and flow moves on.

You probably noticed the extra Script Task at the top of the container. That is our empty task for readability again, except it is required here since you can't have a Precedence Constraint before the first task in a container. The Precedence Constraint must be within the container instead of before the container in order to satisfy the flow logic we need.

FLOW WRAP-UP

Well there wasn't anything earth shattering here, but hopefully this sparked some ideas and will save you some time when you need to build some logic into your Control Flows in the future. Thanks for reading.



DATA FLOW EXPRESSIONS

Today, we continue on with the [31 Days of SSIS](#) blog series. Yesterday's post was on the use of [expressions with Precedence Constraints and how to control the Control Flow](#). Today, we'll venture a little further in with the use of expressions, and a head into the Data Flow and how you can use expressions within that.

As you may or may not be aware, most tasks and connections within SSIS can be manipulated to some extent with the use of an Expressions collection. Expressions collections allow for properties, such as Connection Strings and the Disabled property, to be assigned at run-time. The values for these properties can be determined with the SSIS expression language that I previously blogged a bit about.

SSIS CONFIGURATION PROBLEM

I mentioned that the Expressions collection was available for tasks and connections. Unfortunately, that I know of, there are no Expressions collections available on any transformations. Personally, I always found this to be a failing, since I often wanted to configure transformations at run-time.



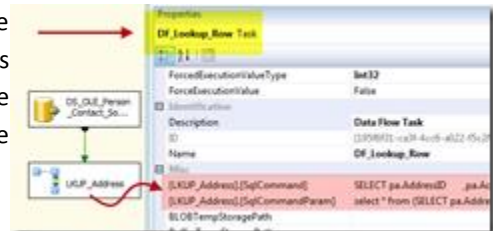
One good example of this is the Lookup transformation. With this transformation, you are able to choose a table, or view as the source. You may also choose to write a query as the source for the lookup.

The two things you cannot do that you will want to do, is either add parameters to the query or use a variable to set the query at run-time. Both of those options are available to OLE DB Sources from SQL Server, but not for the Lookup transformation.

There are a number of situations where I have run across this need. Specifically, when I am using multi-tenant databases the domain of the lookup should be restricted to a single client or customer within the database. To solve this, I will sometimes utilize the [Merge Join solution](#) I talked about a while back. There is a different solution that you can go with, that doesn't require more and different components.

SOLUTION THROUGH DATA FLOW EXPRESSIONS

The different solution is to leverage the Expressions collection on the Data Flow task. If you haven't noticed, when some transformations are added to the Data Flow, there are new properties added to the Data Flow task. These new properties will be exposed in the Expressions collection.



As an example, if you add a Lookup transformation to a Data Flow task, there will be two new properties added to the task. The properties are [SqlCommand] and [SqlCommandParam]. These properties are tied to the query that is added to the Lookup transformation. Modify these parameters, and you will modify the data set that is used by the Lookup transformation.

LOOKUP TRANSFORMATION SETUP

Rather than talking in platitudes, let's look at how to set this up in a package. To follow along, download the package at the end of the post. I'm going to assume that you have that package, and will not be talking about setting up the components outside of the Lookup Transformation changes.

Let's begin. To start with, the SSIS package contains a Lookup transformation named LKUP_Address. Currently, the transformation is using the following query to lookup addresses:

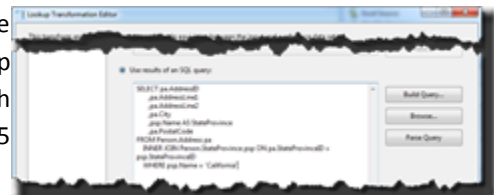
```
SELECT pa.AddressID
,pa.AddressLine1
,pa.AddressLine2
,pa.City
,psp.Name AS StateProvince
,pa.PostalCode
FROM Person.Address pa
INNER JOIN Person.StateProvince psp ON pa.StateProvinceID = psp.StateProvinceID
WHERE psp.Name = 'California'
```

But when we execute the package, it needs to use the following query:

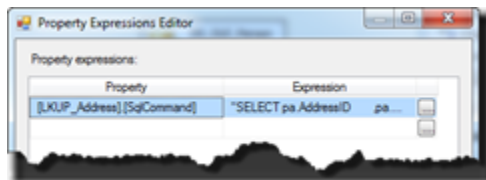
```
SELECT pa.AddressID
,pa.AddressLine1
,pa.AddressLine2
,pa.City
,psp.Name AS StateProvince
,pa.PostalCode
FROM Person.Address pa
INNER JOIN Person.StateProvince psp ON pa.StateProvinceID = psp.StateProvinceID
WHERE psp.Name = 'Washington'
```

While this is a contrived example compared to what would be used in a real environment, it does demonstrate how this functionality can be used.

Now that we know what we need to do, let's move forward with the changes that are required. If you take a look at the Lookup transformation, you'll see that the transformation is populated with the first query from above. If we run with this query there will be 725 rows sent to the Lookup Match Output.



We need to change this, so that it will execute with the state of Washington instead. As I mentioned above, there are two properties added to the Data Flow task. To get to changes that we require, we need to change the value for the [SqlCommand] property.



On the properties for the Data Flow task, click on the ellipsis at the end of the line for the Expressions property. This will open the Property Expression Editor. Select the property [LKUP_Address].[SqlCommand] from drop down. Then set the expression to:

```
"SELECT pa.AddressID
,pa.AddressLine1
,pa.AddressLine2
,pa.City
,psp.Name AS StateProvince
,pa.PostalCode
FROM Person.Address pa
INNER JOIN Person.StateProvince psp ON pa.StateProvinceID = psp.StateProvinceID
WHERE psp.Name = 'Washington'"
```

When that is done, execute the package again, and the count for the Lookup Match Output will change from 725 to 379. It probably shouldn't seem this simple to solve to change the query for the Lookup transformation at run-time. The truth be told, it is.

DATA FLOW EXPRESSIONS WRAP-UP

Using the Expression collection is a great way to extend the flexibility of the Data Flow task directly, and your SSIS package indirectly. Most transformations offer a property or two to the data flow, that can be accessed through the Expression collection. While the example was on the Lookup transformation, the Derived Columns, Conditional Split and others also allow for run-time property changes.

By using these new ways to improve performance, by limiting lookup sets (Lookup transformation) or changing how data flows through the package (Conditional Split transformation) are made available. Take the time to look into these, and make sure to leverage them when applicable.

The funny thing about Expression collections at the Data Flow task level is that up until last year, I hadn't even been aware of them. They are easy to miss if you haven't taken the time to do a little exploring. Take a hint from here to look at these and dig a little further, you might find out that something that gets you "every time" is there but just not where you thought it would be.

GENERATING ROW NUMBERS

It's Sunday afternoon and a good time to be putting up post twenty-three for the [31 Days of SSIS](#) blog series. A list of the other posts in this series can be found in the [introductory post](#). If you haven't had a chance to read through that, I would recommend taking a look at it after reading this post.

PACKAGE REQUIREMENTS

For today, we're going to review a solution I devised for a problem I came across a while back. For this problem, I had to generate row numbers for rows in a file. My first inclination had been to use the Aggregate transformation, but it does aggregations which this problem wasn't requiring.

What I needed here was to add a row number column to the data set. For each account number in the data set, the row number would start at one and increment for all rows that matched to account number. This needed to be done to data being received in a flat file and also prior to when it was loaded to the SQL Server database.

ROW NUMBER SOLUTION

Solving this issue would actually be fairly simple in SQL Server. All that would be needed is to add the ROW_NUMBER() function to the query. Then partition the data on the the account number and all is said and done. Unfortunately, things don't work out that simply with SSIS.

As I mentioned, the Aggregation transformation would not work for this problem. It could count the number of rows or average some values. Also, no other transformations had similar functionality that could be used directly to solve this problem.

Fortunately, there is an indirect way to solve this problem. I've talked about the flexibility of the Script Component transformation a couple times before. Once again, that flexibility will come in handy. By using that transformation along with the Sort transformation the requirements can be accomplished.

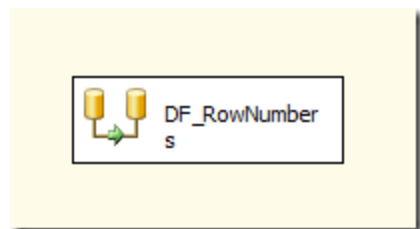
To start with, we need to sort the data that needs the row numbers. It should be sort along the groups that would be the partitions with using the ROW_NUMBER() function. The sort also needs to include the columns for the sorting for the order of the row numbers. Next, a Script Component transformation will be used to add the row numbers. By comparing each group with the other group values, the row number value can be determined.

Since we have the requirements and the solution description, lets take a look at what the package for this would look:

PACKAGE CONTROL FLOW

There isn't a lot to this SSIS package from the control flow perspective. There will be a single Data Flow task and no others tasks and no variables. The Data Flow task is:

- **DF_RowNumbers:** Data flow task that contains the Sort and Script Component transformations.



PACKAGE DATA FLOW

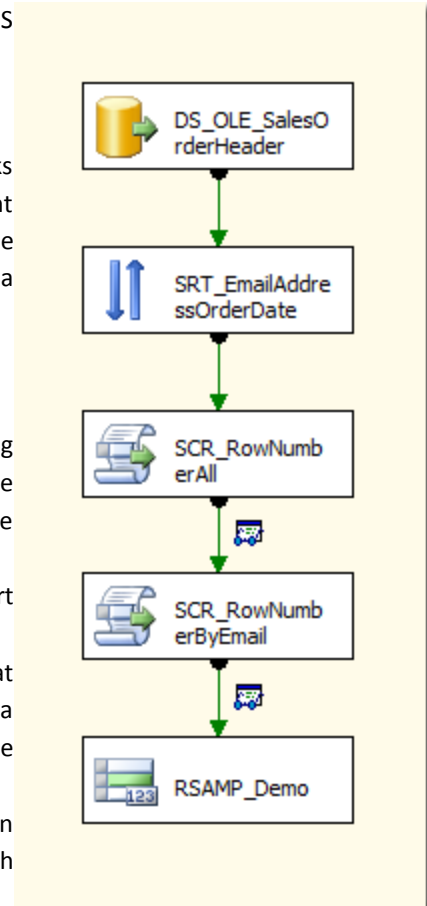
As mentioned above, there is only one Data Flow task in the SSIS Package. Since all the work happens there, let's get on to it.

DF_ROWNUMBERS

In this data flow, we'll be bringing in some data from the AdventureWorks database. The data will be sorted and then sent into two Script Component transformations. The first will create a row number across all rows. The second will create a set of row numbers for each e-mail address in the data being processed through the SSIS package.

To accomplish all of this, the data flow has the following transformations:

- **DS_OLE_SalesOrderHeader:** OLE DB Data Source to bring data into the data flow. When I encountered this issue in the "wild", it was with flat files but this data source will provide the same effect of bringing in data.
- **SRT_EmailAddressOrderDate:** Sort transformation to sort all of the rows in the data flow by EmailAddress then OrderDate.
- **SCR_RowNumberAll:** Script Component transformation that adds row numbers to each row beginning at the start of the data set through to the end. The row numbers are sequential across the whole set.
- **SCR_RowNumberByEmail:** Script Component transformation that adds row numbers to each row. The first row for each EmailAddress group starts a new sequential set of row numbers.
- **RSAMP_Demo:** Row Sampling transformation that was added solely to provide a Data Flow path to put a Data Viewer on.

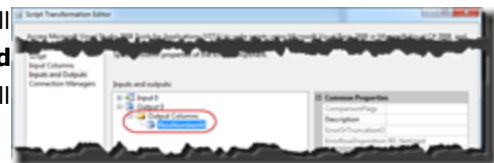


SCRIPT COMPONENT TRANSFORMATION SETUP

The most important areas to configure for this SSIS package are the Sort transformations and the Script Component transformations. Setting up the Sort transformation should be fairly easy. The Script Component transformations will take a little bit more work.

SCR_ROWNUMBERALL

To start configuring the Script Component transformation, you will first need to add an output column to the **Inputs and Outputs** tab. In this example, the column is named RowNumberAll and it is a 4-byte integer.



Now launch the script editor from the script tab and replace the contents of **ScriptMain : UserComponent** with the following code.

```
int rowNumber = 1;

public override void Input0_ProcessInputRow(Input0Buffer Row)
{
    Row.RowNumberAll = rowNumber;
}
```

```

rowNumber = rowNumber + 1;
}

```

This code will keep track of the state of the row numbers through the variable rowNumber. It will also increment the value after each row is processed.

With that, the transformation is configured. If executed, a data viewer on the data flow path will provide results similar to those in the image to the right.

FirstName	LastName	EmailAddress	RowNumber
A.	Levenets	al@adventureworks.com	1
A.	Levenets	al@adventureworks.com	2
Aaron	Carl	aaron@adventureworks.com	3
Aaron	Carl	aaron@adventureworks.com	4
Aaron	Carl	aaron@adventureworks.com	5
Aaron	Carl	aaron@adventureworks.com	6
Aaron	Carl	aaron@adventureworks.com	7
Aaron	Carl	aaron@adventureworks.com	8
Aaron	Carl	aaron@adventureworks.com	9

SCR_RowNumberByEmail

Next, let's look at the Script Component transformation that will be used to add row numbers across each partition of EmailAddress. The same output column as above will be added to this transformation. This time the column will be named RowNumberByEmail. Then add in the column EmailAddress as one of the **Input Columns**.



As before, launch the script editor from the script tab. The contents of **ScriptMain : UserComponent** should be replaced with the code below.

```

string emailAddress = "";
int rowNumber = 0;

public override void Input0_ProcessInputRow(Input0Buffer Row)
{
    if (emailAddress == Row.EmailAddress)
    {
        rowNumber = rowNumber + 1;
    }
    else
    {
        rowNumber = 1;
    }

    emailAddress = Row.EmailAddress;
    Row.RowNumberByEmail = rowNumber;
}

```

The code in this transformation is similar to the previous Script Component transformation. Except in the previous transformation it incremented the row number for each row, where now it will check to see if the EmailAddress has changed before setting the row number. If the EmailAddress has changed then the row number is reset to 1. Otherwise the row number is incremented by 1.

FirstName	LastName	EmailAddress	RowNumber	RowNumberByEmail
A.	Levenets	al@adventureworks.com	1	1
A.	Levenets	al@adventureworks.com	2	2
Aaron	Carl	aaron@adventureworks.com	3	1
Aaron	Carl	aaron@adventureworks.com	4	2
Aaron	Carl	aaron@adventureworks.com	5	3
Aaron	Carl	aaron@adventureworks.com	6	4
Aaron	Carl	aaron@adventureworks.com	7	5
Aaron	Carl	aaron@adventureworks.com	8	6
Aaron	Carl	aaron@adventureworks.com	9	7

Go ahead and execute the SSIS package again and the data viewer results will look like those in the image above

ROW NUMBER WRAP-UP

It doesn't take much to implement this logic in your SSIS package. All that is needed is a couple transformations and a little outside of the box thinking. The key to remember is that if there isn't a specific transformation for your business needs, there is probably a way to put it together with the Script Component transformation.

Why would someone need to add a row number to a data set in an SSIS package? The main reason I would consider it would be to add an "identity" value to a row. This can allow rows that have no identifying aspects to be sliced and diced into various transformations and still retain a value that can reliably tie the information back together. Also, if the identifying information for a row spans multiple columns, this single, narrower column can assist in improving performance.

With a couple of tweaks, this solution can also be varied from just adding a row numbers to solving other requirements. One such that comes to mind immediately would be the add running totals to a data flow. Instead of incrementing the row numbers add the values from the current row to the previous rows.

As always, the package for this post is below. If you have any questions, feel free to leave a comment.

ONE PACKAGE, UNLIMITED DATABASES

Another day, another blog post all about SSIS in my continuing journey through [31 Days of SSIS](#). A list of the other posts in this series can be found in the [introductory post](#). If you haven't had a chance to read through that, I would recommend taking a look at it after reading this post. ~~Yup, I just copied this from yesterday's post. Plagiarism!~~

Yesterday, I jumped back to looking at another use for the [Script Component transformation](#). Today, I want to make another jump and look at another use for Expressions and ForEach loops. In this case, we will look at a package where one package will be used to make similar changes to multiple databases.

PACKAGE REQUIREMENTS

A number of years ago, I was working with a single tenant database application. That database had a state/province table that was "kinda" similar in each database. This "kinda"-bit had nothing to do with any specific business requirements. It was this way because nobody ever got around to updating all of the databases whenever a change was made in one database or another. As is probably obvious, new states don't show up too often but the state of this data was terrible.

Management of the data in these tables fell down to me, as the DBA. When they were out of synch it was my problem so I needed to do something to resolve the situation. What I wanted was a method that I could manage the table from one location and have each of the databases updated in turn. I also needed a solution that was light-weight and low tech. The people that would manage the solution after it was pushed out were savvy with Access and a bit afraid of SQL Server.

ONE PACKAGE SOLUTION

To solve this issue, I opted for an SSIS solution. SSIS has capabilities to bring the data across to the databases and determine what needs to be inserted, updated, or deleted. Alternatively, I could have just used change management and DML scripts to manage the data. The downside to that was that, at that point, that process was what caused the problem. Also, I could have looked at using replication. I opted against that since I wanted to keep the implementation low tech. Replication does require a bit of know-how to keep it going.

Of course, there should be a package per database. That would be impossible to manage and put the solution back with the existing process. There also shouldn't need to be one execution of the package per database. This needed to be a process that would just run and work itself out with one or one hundred databases.

To accomplish these needs, the package will utilize a ADO record set that contains a list of all of the databases that need to be managed. That record set will be used to iterate a loop that will make all the necessary changes to that databases that subscribe for the updates.

With the solution outline, we'll begin to go through the package to detail how this is done and hopefully help you learn a couple new things today.

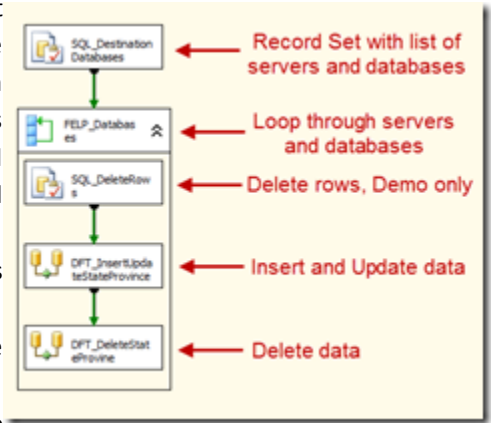
PACKAGE CONTROL FLOW

There are a few things that will be occurring in the Control Flow to take note of in this package. First, the we need a list of all of the servers and databases that contain the tables that we want to update. With that information the package will loop through each of the server and database combinations. Finally, for each iteration of the loop,

the package will insert, update, and delete data as is required to keep the data in the client database tables identical to the data in the central table.

This logic will be implemented with the following Control Flow components:

- **SQL_DestinationDatabases:** This is a T-SQL that fetches a list of servers and databases that need the information. For the example, the information is just in a couple SELECT statements. In a live environment, this would be tied into a table that lists that servers and databases. The server and database names are populated to variables.
- **FELP_Databases:** ForEach Loop container that iterates across each occurrence of server and database name.
- **SQL_DeleteRows:** Demo only task to delete and update data to make all of the data flow paths do something.
- **DFT_InsertUpdateStateProvince:** Data Flow task to determine whether rows should be inserted or updated.
- **DFT_DeleteStateProvince:** Data Flow task to determine whether rows should be deleted.



PACKAGE VARIABLES

There are a number of variables in the package to help control how everything is processed.

Variables			
Name	Scope	Data Type	Value
DatabaseName	24of31-OnePackageManyData...	String	Client_DB1
RSDatabases	24of31-OnePackageManyData...	Object	System.Object
ServerName	24of31-OnePackageManyData...	String	STRATE-02\SQL2008R2

The variables from above were added for the following purposes:

- **DatabaseName:** Name of the database that needs to receive central data
- **RSDatabases:** Record set that contains the list of servers and databases that need to receive central data.
- **ServerName:** Name of the server that contains a database that needs to receive central data

CONNECTION MANAGER

Usually I don't highlight the connections, but in this case it is necessary. The SSIS package contains two connections:

- **AdventureWorks:** The database with all of the central data. Not truly a central database but it is playing the part for this demo.

- Subscriber:** The database receiving the data from the central database. The Connection String for this connection is controlled by an expression that is tied to the DatabaseName and ServerName variables. The settings for these variables can be seen to the right. By doing this, you are able to change the connection of your SSIS package not just at the beginning of run-time, BUT also in the midst of execution. Connections to databases do not have to remain constant throughout execution.



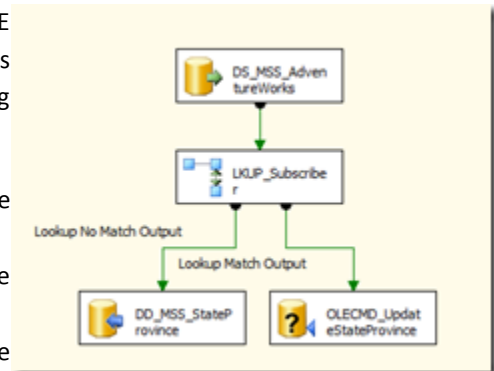
PACKAGE Data FLOWS

With the connection information configured, it's time to look at the data flows of the SSIS package. There are two data flows in this SSIS package

DFT_INSERTUPDATESTATEPROVINCE

This first Data Flow task will control the INSERT and UPDATE operations on the table that is being managed through this package. To accomplish these operations the following transformations are used:

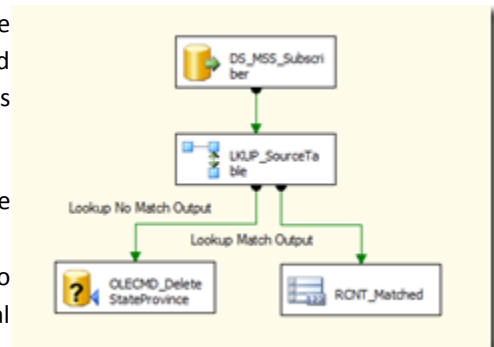
- DS_MSS_AdventureWorks:** OLE DB Source to retrieve central data for distribution.
- LKUP_Subscriber:** Lookup transformation to determine if the record exists in the subscriber database.
- DD_MSS_StateProvince:** OLE DB Destination in the subscriber database that receives the data. This is just a straight insert.
- OLECMD_UpdateStateProvince:** OLE DB Command to perform updates for each row that currently exists in that database. This isn't ideal, but I'm saving a more ideal solution for determining deltas for a future post.



DFT_DELETESTATEPROVINCE

The second Data Flow task controls the DELETE operations on the table. Since rows may be deleted on the central table, they should also be deleted on the subscribers. To accomplish these operations the following transformations are used:

- DS_MSS_Subscriber:** OLE DB Source to retrieve subscriber data for validation against the central data.
- LKUP_SourceTable:** Lookup transformation to determine if the subscriber record still exists in the central database.
- OLECMD_DeleteStateProvince:** OLE DB Command to perform deletes for each row that no longer exists in the central database.
- RCNT_Matched:** Row count of rows that match the central database. This is just for informational and troubleshooting needs.



LAST TIDBITS

Before you can run the package, you will need to create a couple of dummy databases. The script for these is below:

```
CREATE DATABASE [Client_Db1]
GO

USE [Client_Db1]
GO

CREATE TABLE [dbo].[StateProvince](
[StateProvinceID] [int] NULL,
[StateProvinceCode] [nchar](3) NOT NULL,
[CountryRegionCode] [nvarchar](3) NOT NULL,
[IsOnlyStateProvinceFlag] [bit] NOT NULL,
[Name] [nvarchar](50) NOT NULL,
[TerritoryID] [int] NOT NULL,
[rowguid] [uniqueidentifier] NOT NULL,
[ModifiedDate] [datetime] NOT NULL
) ON [PRIMARY]
GO

CREATE DATABASE [Client_Db2]
GO

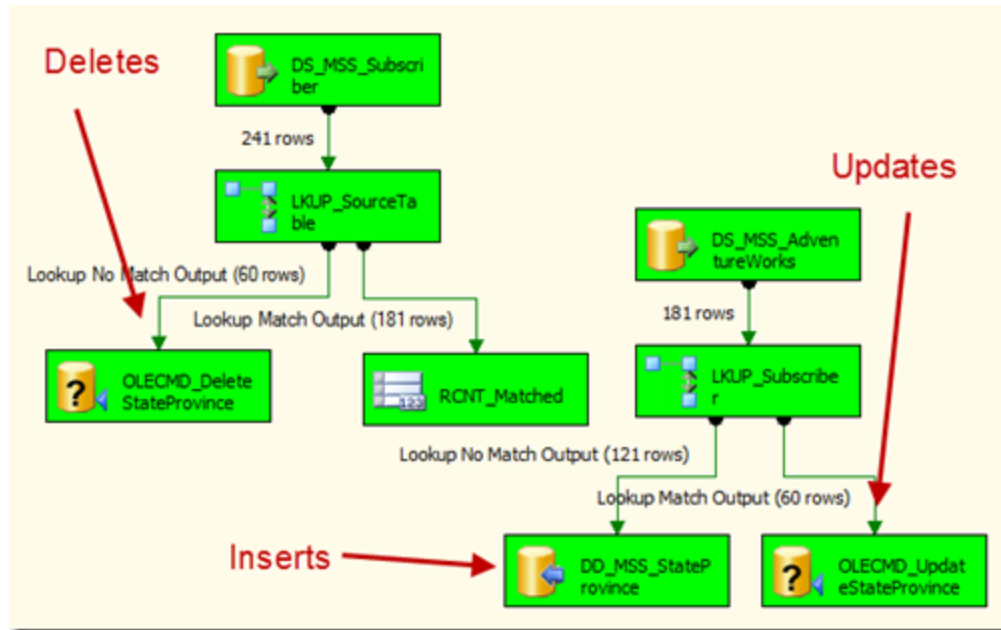
USE [Client_Db2]
GO

CREATE TABLE [dbo].[StateProvince](
[StateProvinceID] [int] NULL,
[StateProvinceCode] [nchar](3) NOT NULL,
[CountryRegionCode] [nvarchar](3) NOT NULL,
[IsOnlyStateProvinceFlag] [bit] NOT NULL,
[Name] [nvarchar](50) NOT NULL,
[TerritoryID] [int] NOT NULL,
[rowguid] [uniqueidentifier] NOT NULL,
[ModifiedDate] [datetime] NOT NULL
) ON [PRIMARY]
GO
```

PACKAGE RESULTS

Go ahead and execute the packages a couple times. The first time the results will be kind of boring since everything will be an insert. The second execution, though, there will be some rows that flow into the update and delete paths. This is due to the demo only T-SQL statement to mimic changes in the database.

The results for the second execution should look like the image below. Of course, I've cropped both data flows over each other into a single image.



ONE PACKAGE WRAP-UP

One of the things that I really like about SSIS is it's continued flexibility and ease of use. With just a record set, a couple of variables and a loop, an SSIS package can change from updating one database to updating as many as you need. Hopefully this provides some inspiration to your own processes where you can consolidate packages and processes.

I should mention that while I use the OLE DB Command in this package it isn't necessarily overt endorsement of it's use. Considerations for using it are many and enough for another blog post. For today, I'll just say that you need to be careful of the fact that this transformation will execute one T-SQL statement per row that it processes in the data flow path.

ONE PACKAGE, UNLIMITED TABLES

One day at a time and we have reached the twenty-fifth post in the [31 Days of SSIS](#). If you enjoy this post, take a look at other posts in this series. They can be found through the [introductory post](#).

Yesterday's post demonstrated an SSIS package that connected one package to [unlimited databases](#). For today, I thought I would take it down one more notch and connect one package to unlimited tables.

A while back, I was presented with an interesting situation. There was an archival process that was controlled by a SQL script with hundreds of lines of code. This script would dynamically generate a execution script for the archival process that had tens of thousands of lines of code.

Unfortunately, there were two big problems with this solution. First, maintaining the archival process was a bear. It contained a combination of data in tables, hard-coded configurations in scripts, and extended properties that were required and undocumented. Second, troubleshooting issues was near impossible. Since it was difficult to maintain it was easy for things to get missed. There were also many situations where transactions were started and not committed. Error handling was sporadically implemented and messages that described any issues were lacking.

PACKAGE REQUIREMENTS

Shortly after becoming aware of this situation, I proposed a change in direction from the existing solution to a solution centered on SSIS. The idea was that instead of scripts with thousands of lines of T-SQL, an SSIS package or two could move the data from the production to the archive server. On this server, a generic merge statement could be used to import the new and updated records. The idea was to have some common components that would generically load everything.

One of the major requirements for this was the need to be able to archive data from any number of tables. Initially, the process would cover a couple hundred tables. Over time, though, it might need to archive those plus a few hundred more tables. The solution had to be robust enough that it could expand at any time with minimal configuration.

UNLIMITED TABLE SOLUTION

An immediate issue that came up with architecting this solution was that Data Flows in SSIS are strongly typed. If they are configured for there to be 10 columns that are all integer data-types; then that is what they are. There is not capacity to change them at run-time. For this reason, consideration of the use of Data Flow tasks was removed as a part of the solution.

Instead, I opted to turn to a couple technologies that have been around SQL Server for quite some time. These are BCP and BULK INSERT. While SSIS is relatively new, BCP and BULK INSERT have been around for a while. Their age, though, should not be held against them when designing solutions. In the end, we need a solution. Not something built on the latest and greatest. With these two technologies in hand, the SSIS package can be built.

The overall idea will be to take a list of tables that need archived. This can be stored in a table, based on extended properties, or tied into the implementation of other features (such as CHANGETRACKING or Change Data Capture). However you obtain the list of tables, the point is the need for a list that can be placed into a record set.

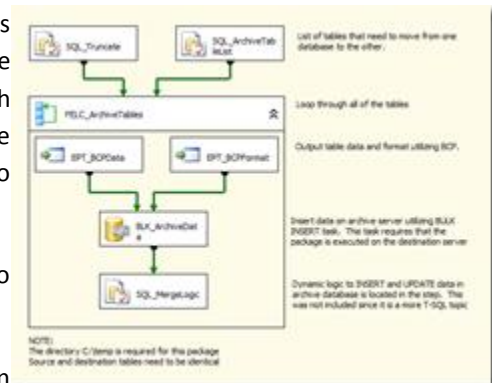
After that, the list of tables will be used to iterated through with a ForEach Loop. With each table in the loop, the SSIS package will BCP out the data from the table in one database to a file. The file will then imported into the archive database using BULK INSERT.

The last step will be to merge the data imported into the existing tables in the archive database. For the purposes of this demonstration we'll have a place holder for this but no logic to manage it.

PACKAGE CONTROL FLOW

Differing from most other SSIS packages, all of the work in this package will occur in the Control Flow. As discussed above, the control flow will obtain a list of tables. It will then iterate through each of the tables. In each iteration, the package will BCP out the data needed. After which the package will import the data into archive tables.

There are a number of tasks in the control flow that are needed to implement this solution. Those tasks are:



- **SQL_Truncate:** Execute SQL task that truncate the data in the archive tables. This is just to help show that the process is functioning. Demo purposes only for this task.
- **SQL_ArchiveTableList:** Execute SQL task that retrieves a list of tables that need to be archived. In a full implementation this would query the stored list of tables that need to be archived.
- **FELC_ArchiveTables:** ForEach Loop container that cycles through each of the tables retrieved from the SQL_ArchiveTableList task.
- **EPT_BCPData:** Execute Process task that runs a BCP.exe command. This task retrieves all of the data for the table and populates it to an output file.
- **EPT_BCPFormat:** Execute Process task that runs a BCP.exe command. This task retrieves the format for the table. The format will be required to import the data in the next step.
- **BLK_ArchiveData:** BULK INSERT task that consumes the BCP data and format files. The data is inserted from those files into the archive table.
- **SQL_MergeLogic:** Execute SQL task that is a placeholder for INSERT, UPDATE, and DELETE logic. This is included for demo purposes to show where this would be located in a larger package.

PACKAGE VARIABLES

There are a number of variables in the package to help control how everything is processed.

Name	Scope	Data Type	Value
BCPData	25of31-OnePackageM...	String	AdventureWorks. out "C:\temp\output.dat" -S "STRA...
BCPFormat	25of31-OnePackageM...	String	AdventureWorks. format nul -S "STRATE-02\SQL2008...
SQL_RS	25of31-OnePackageM...	Object	System.Object
SQL_Table	25of31-OnePackageM...	String	

The variables from above were added for the following purposes:

- **BCPData:** Expression containing the arguments for the BCP utility called in the EPT_BCPData task
- **BCPFormat:** Expression containing the arguments for the BCP utility called in the EPT_BCPFormat task
- **SQL_RS:** Record set that contains the list of tables that need to be archived.
- **SQL_Table:** String variable that contains the name of the current table being archived

VARIABLE EXPRESSIONS

Part of what makes this package work is the use of the [BCP utility](#). In order for the BCP utility to operate, the arguments for BCP.exe need to change per iteration. The expressions below control this behavior.

- **BCPData:** "AdventureWorks." + @[User::SQL_Table] + " out \"C:\\temp\\output.dat\" -S \"STRATE-02\\SQL2008R2\" -T -c"
- **BCPFormat:** "AdventureWorks." + @[User::SQL_Table] + " format nul -S \"STRATE-02\\SQL2008R2\" -T -c -f \"C:\\temp\\output.fmt\""

PACKAGE DATA FLOWS

This should have been expected. There are no data flows in this SSIS package.

UNLIMITED TABLES WRAP-UP

There are a number of items that would be included in this solution to flush it out completely. For brevity, I've focused this post on the use of BCP and BULK INSERT. Some of the things that were either glanced over or not mentioned are:

- **Incremental updates:** For most implementations, you will want to only ship a few columns over to the archive server for each execution. Ideally these would be the records that have changed. There are various ways to do this; such as triggers, change date columns, CHANGETRACKING, or Change Data Capture to name a few.
- **Foreign key ordering:** If the tables in the archive server have foreign key relationships similar to those of the production database, then the tables will need to be processed in the order that doesn't violate the foreign keys.
- **BULK INSERT locality:** Not mentioned but a good point to remember, BULK INSERTS need to use files that are local to the SQL Server instance. This means the package will need to be run on the archive server.
- **BULK INSERT task:** There have been SQL Server 2008 CUs that break the functionality of the BULK INSERT task. In these cases, I've recommended switching over from a BULK INSERT task to an Execute SQL task that uses a BULK INSERT statement.
- **Staging database:** When I've used this solution with clients, the data is usually landed to a staging database. This provided a layer of disconnect where the SSIS package could drop and create whatever it needed during processing.
- **Staging table schema:** One assumption that is being made with the BCP and BULK INSERT process is that the schema and layout of the tables between the two environments are identical. For this reason, I will often re-create the destination tables when the package runs to guarantee the schema.

Merge piece: The T-SQL to INSERT, UPDATE, and DELETE data that has been imported to the archive database with data was not included. This is primarily because of the limited scope of this post. If you want to guess... it uses dynamic SQL.

What do you think of this solution? It's honestly one of my favorite designs since I was able to accomplish something I initially thought might not be possible.

SMALL PACKAGE, BIG WIN

Welcome to day twenty-sixth post of the [31 Days of SSIS](#). We're getting near the end now. If you have not read the other posts, they can be found through the [introductory post](#).

This series has been a combination of my thoughts on practices for working with SSIS and proof of concept packages that represent solutions I've worked on in the past. We've done a few days of packages now, so I thought I would shift gears again and talk about one of the best practices that I try to follow.

This best practices is to **keep the unit of work for each SSIS packages small, focused, and limited in the task that it is trying to accomplish**. This means that when I write an SSIS package one of the goals will be to limit the number of Data Flows to as few is logically possible.

For instance, if I have three files to process then I will usually build this as three SSIS packages. If all three of those packages have a shared process with identical inputs, then I may create a fourth SSIS package and pass data from the first three packages to the fourth with a Raw File.

PRACTICE IMPACT

There are a some good reasons for developing small SSIS packages. These reasons are tied into their impacts on your environment. I could say it'll **ease development**, but rather than doing that I want to look at the impact that this practice will have on impact.

- **Development Impact:** When you are developing smaller packages you end up with smaller amounts of work that need to be accomplished. We work in a mostly agile world where people count on us to deliver often. By delivering smaller packages and breaking apart the work, you can move towards the delivery and achieve the same goals the other development teams are already working towards. This is the difference between an SSIS package that can can import most of the files and the SSIS solution that has 9 of the 10 packages done.
- **Performance Impact:** As packages are broken down into smaller units of work, you will begin to find that some of these packages will have dependencies on other packages. Other times, you will find that some perceived dependencies are false and don't really exist. They may have been create to help the visual flow of the data. As these opportunities are discovered ETL frameworks can be utilized to leverage parallelism within the process execution.
- **Maintenance Impact:** With packages that are broken down into smaller tasks, the cost and effort of maintenance of the SSIS packages is reduced. If an SSIS package has 20-30 different components and updates a dozen or so tables there is always the concern and risk that one change to one import will have an actual or perceived side effect. When the SSIS package that updates TableA is changed, it's helpful to know that the changes to TableB won't have to be retested since it is in another SSIS package.
- **Troubleshooting Impact:** If you've had to troubleshoot monster SSIS packages before, you already will understand this point. The smaller an SSIS package is and the more focused the package's task, the easier it will be to troubleshoot issues. It becomes easier because the list of things could have led to the issue are reduced. It there is an issue with a single task it is easy to identify since
- **Recycled Impact:** As the story below will detail, as SSIS packages are broken down to smaller tasks you may also find ways to leverage them in ways that you never considered in the original design. This will open up you environment to other opportunities to solve new problems with existing architectures.

A TWO-PART STORY

Let's examine a recent use of this best practice that I had that really exemplifies the package. Let's start with a small package.

SMALL PACKAGE

In a recent engagement I had to add in some new functionality to an existing SSIS process. To protect the innocent, let's say that the requirement was to geo-code addresses after they arrived. To do so, I was going to be using a web service. The geo-code information then needed to be placed in a special geo-code table. The one additional requirement was that this didn't need to happen for all imports and sometimes would need to be run some time after the original import was already executed. ~~These aren't the true requirements, but they are a close enough approximation.~~

During design, I had a couple options that I had considered using:

1. The functionality could be added to the package that did the address inserts and updates. After the record was added, the geo-code information could be retrieved from the web service. The data would then be added to the geo-code table. For imports that used the geo-code web service, there would be a flag to turn this functionality on or off.
2. The functionality could be added as a new SSIS package. The new SSIS package would receive an input file, [probably a raw file](#), that was generated when the first package executed. The imports that needed geo-coding would then have this SSIS package run against the output from the previous package.

Of course, I went with the second option. This was implemented fairly well and after a bit of development was ready to deploy. The new SSIS package did a great job with its place in the existing architecture.

BIG WIN

But wait... of course, there is more. The part of this that really sells the concept of smaller, focused SSIS packages is what happened a couple weeks after deploying the new SSIS package.

On a snowy afternoon, I received an e-mail wondering if there was any way a few hundred addresses could be run through the new process. This wasn't part of the initial requirements, but they thought to ask anyways. The problem was there was a different way in which addresses were loaded and they needed geo-coding as well. The existing process allowed the user to request that one address at a time would be geo-coded.

The win here is that with a query and an export of the results to SSIS, the manual work was eliminated. Since the design focused on a single task that was independent of the import it could be leveraged for previously unexpected uses. We got mad props for the design of the package and one person saved themselves from a serious case of sore wrist.

WRAP-UP

This practice isn't absolute and every one of my packages aren't always small and focused. Though I do prefer to develop in this direction and look to opportunities to do this as often as possible.

What are your thoughts on this practice? Anything that you would like to add? Feel free to leave a comment.

ADDING SOME DELTA FORCE

Welcome to day twenty-seventh post of the [31 Days of SSIS](#). We're getting near the end now. If you haven't yet, I recommend reading the other posts, they can be found through the [introductory post](#). This series has been a collection of the things that I've picked up over the years in my work with SSIS.

In a presentation that I did this week I was asked about how I handled updates with the SSIS package I was demoing. This was the package that I discussed in the post [One Package, Unlimited Databases](#). I explained the process that I use in that webcast but thought it would also make a good topic.

DELTA FORCE REQUIREMENTS

Before we get into the details, I'd like to take the question and restate it as requirements. By doing this, I'm hoping that by reading it in this way requirements you encounter might spark a connection. Through that connection you'll recall this post and be able to apply this concept.

The general need is a package that will receive data from one data source. In this case, we are receiving data from a central database. When the data is received, rows that do not exist will be insert. Rows that already exist as records in the tables will be updated.

The updates to the records should only be applied to records where there is a change to at least one column. Existing records should not be modified if there are no changes in any of the columns.

DELTA FORCE SOLUTION

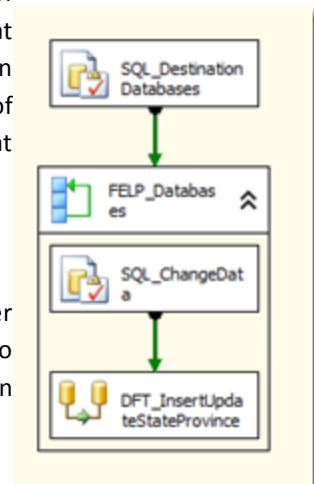
The first part of the requirements was already implemented in the [One Package, Unlimited Databases](#) post. Pieces that were discussed as part of that solution won't be detailed out here. We'll pick up the solution from the second half of the requirements.

One option we could choose would be to go cheap and easy. To do this we could modify the OLE DB Command with a WHERE clause and then call this requirement done. The problem with this option is that if there are 1,000 in the path of existing rows, then 1,000 UPDATE statements will be executed. The trouble with this is that there might only be 2 updates that need to occur. Due to this, I wouldn't consider this option ideal.

Another option is to use the Script Component transformation to check the values of the columns. We don't want to execute SQL statements against the database that may or may not need to be executed. Instead, the Script Component transformation can be used to filter out all of the rows that haven't changed. Instead of sending all of the existing rows to the OLE DB Command, it will instead only receive those rows that have changed.

PACKAGE CONTROL FLOW

Since most of the interesting things in the Control Flow were described in the earlier post, they will not be expanded on too much again. Our focus will be on drilling into the Data Flow which we will just to shortly. Since they aren't related to this variation of the solution, I won't be looking at that variable and connection configurations.



There are four tasks that comprise the Control Flow:

- **SQL_DestinationDatabases:** This is a T-SQL that fetches a list of servers and databases that need the information. The server and database names retrieved are populated to variables.
- **FELP_Databases:** ForEach Loop container that iterates across each occurrence of server and database name.
- **SQL_ChangeData:** Demo only task to delete and update data to make all of the data flow paths do something.
- **DFT_InsertUpdateStateProvince:** Data Flow task to determine whether rows should be inserted or updated. This will contain the bulk of our solution.

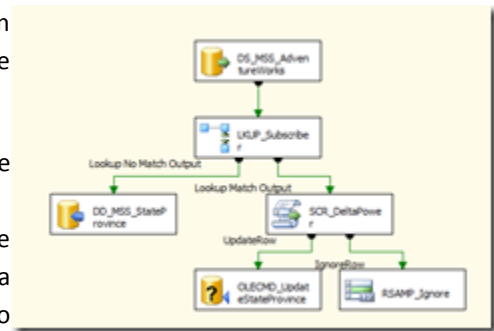
PACKAGE DATA FLOW

If you recall from the previous post, there were two data flows in the package. Since the second one doesn't impact the focus on the delta change, it has been removed. In a full solution to cover inserts, updates, and deletes it would be reaming. With that said, we'll dig into the data flow that is remaining.

DFT_INSERTUPDATESTATEPROVINCE

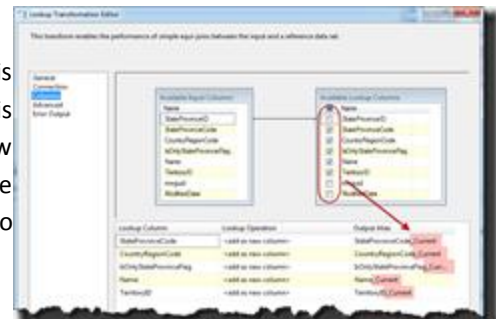
This Data Flow task controls the INSERT and UPDATE operations on the table affected by this data flow. The data flow is similar to before with the exception to a couple additional transformations.

- **DS_MSS_AdventureWorks:** OLE DB Source to retrieve central data for distribution.
- **LKUP_Subscriber:** Lookup transformation to determine if the record exists in the subscriber database. There is a slight change in this transformation, described below, to assist in performing the delta checks.
- **DD_MSS_StateProvince:** OLE DB Destination in the subscriber database that receives the data. This is just a straight insert.
- **SCR_DeltaPower:** Script Component transformation that checks each row that exists in the destination table to see if any of the columns in the row has changed compared to the destination table.
- **OLECMD_UpdateStateProvince:** OLE DB Command to perform updates for each row that currently exists in that database. This isn't ideal, but I'm saving a more ideal solution for determining deltas for a future post.
- **RSAMP_Ignore:** Row Sampling transformation that exist purely for viewing pleasure. It allows us to see how many rows are ignored when the package executes.



LKUP_SUBSCRIBER

As I mentioned above, there is a little variation into how this transformation is configured. In the previous post, this transformation was purely used to determine if a row in the data flow path also existed in the table. The lookup was matching on the StateProvinceID without returning any data. This time we want to return some data.



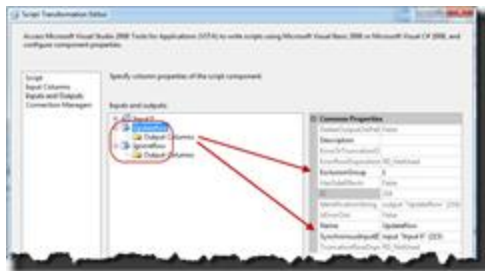
Specifically, the columns StateProvinceCode, CountryRegionCode, IsOnlyStateProvinceFlag, Name, and TerritoryID need to be returned. These are the columns that we want to check to see if any of the values have changed.

One important thing to note is that all of these columns already exist in the Data Flow path. To prevent column naming irritations, append the value “_Current” to each of the columns. This will make them stick out and avoid name irritations.

When this is complete, this transformation is done.

SCR_DELTAPOWER

The true effort of the delta check will be occurring in the SCR_DeltaPower script component transformation. There are two main points of configuration for the transformation. These will be the outputs and the script.



We will configure this component to provide multiple outputs along two paths. The paths will be UpdateRows and IgnoreRows. The names should indicate the state of the data that we plan to add to each of the paths. Each path will contain all of the rows from the input and should be mapped to the input. If you are unfamiliar with how to do that, please read [Every Which Way Outputs](#).

Once the output is configured, it's time to work on the guts of the script. If you haven't done column comparisons in SSIS packages before the code structure might seem obvious. Unfortunately, with the possibilities of NULL values in columns, this can prove tricky to the novice.

Before explaining the script, open the script window and replace the contents of public class ScriptMain : UserComponent with the code below.

```
public override void Input0_ProcessInputRow(Input0Buffer Row)
{
    //Check state of StateProvinceCode
    if (!Row.StateProvinceCode_IsNull && !Row.StateProvinceCodeCurrent_IsNull)
    {
        if (Row.StateProvinceCode != Row.StateProvinceCodeCurrent)
        {
            Row.DirectRowToUpdateRow();
            return;
        }
    }
    else if ((Row.StateProvinceCode_IsNull && !Row.StateProvinceCodeCurrent_IsNull)
        || (!Row.StateProvinceCode_IsNull && Row.StateProvinceCodeCurrent_IsNull))
    {
        Row.DirectRowToUpdateRow();
        return;
    }

    //Check state of CountryRegionCode
    if (!Row.CountryRegionCode_IsNull && !Row.CountryRegionCodeCurrent_IsNull)
    {
        if (Row.CountryRegionCode != Row.CountryRegionCodeCurrent)
    }
```

```

{
Row.DirectRowToUpdateRow();
return;
}
}
else if ((Row.CountryRegionCode_IsNull && !Row.CountryRegionCodeCurrent_IsNull)
|| (!Row.CountryRegionCode_IsNull && Row.CountryRegionCodeCurrent_IsNull))
{
Row.DirectRowToUpdateRow();
return;
}

//Check state of IsOnlyStateProvinceFlag
if (!Row.IsOnlyStateProvinceFlag_IsNull && !Row.IsOnlyStateProvinceFlagCurrent_IsNull)
{
if (Row.IsOnlyStateProvinceFlag != Row.IsOnlyStateProvinceFlagCurrent)
{
Row.DirectRowToUpdateRow();
return;
}
}
else if ((Row.IsOnlyStateProvinceFlag_IsNull && !Row.IsOnlyStateProvinceFlagCurrent_IsNull)
|| (!Row.IsOnlyStateProvinceFlag_IsNull && Row.IsOnlyStateProvinceFlagCurrent_IsNull))
{
Row.DirectRowToUpdateRow();
return;
}

//Check state of TerritoryID
if (!Row.TerritoryID_IsNull && !Row.TerritoryIDCurrent_IsNull)
{
if (Row.TerritoryID != Row.TerritoryIDCurrent)
{
Row.DirectRowToUpdateRow();
return;
}
}
else if ((Row.TerritoryID_IsNull && !Row.TerritoryIDCurrent_IsNull)
|| (!Row.TerritoryID_IsNull && Row.TerritoryIDCurrent_IsNull))
{
Row.DirectRowToUpdateRow();
return;
}

//No Changes, ignore it
Row.DirectRowToIgnoreRow();

```

There are a couple items in the code that I want to all out. First, before I check to see if the values of the source and destination columns are different (pink highlighting), I check to see if both columns are not NULL (yellow highlighting). This is the issue I mention above with NULL values.

```
//Check state of StateProvinceCode
if (!Row.StateProvinceCode_IsNull && !Row.StateProvinceCodeCurrent_IsNull)
{
    if (Row.StateProvinceCode != Row.StateProvinceCodeCurrent)
    {
        Row.DirectRowToUpdateRow();
        return;
    }
}
```

Next, if row isn't picked up above then we check to see if one column is NULL and the other is not NULL. And vice versa.

```
else if ((Row.StateProvinceCode_IsNull && !Row.StateProvinceCodeCurrent_IsNull)
|| (!Row.StateProvinceCode_IsNull && Row.StateProvinceCodeCurrent_IsNull))
{
    Row.DirectRowToUpdateRow();
    return;
}
```

This is completed for each column that needs to be checked. Any rows that don't get kicked out to the UpdateRow output get sent to the IgnoreRow output.

WRAP-UP

Yet another method that the Script Component transformation can be used to add functionality to an SSIS package. Through the use of a solution such as the one above needless SQL statements can be prevented from being sent to your SQL Servers.

One of the things that you might be thinking is "Why the heck is StrateSQL using an OLE DB Command?" To that I say, good point. But is the transformation really that bad?

Commentary on the use of the OLE DB Commands transformation could fill an entire post all itself. What I will say here about the transformation is that in cases where the volume of rows being processed is not substantial, it is acceptable to use this transformation. You definitely need to understand the current and future number of rows that will be processed.

JIM CROCE AND THE FOREACH (FILE) LOOP CONTAINER

It's Friday and I'm still taking Friday's off in my [31 Days of SSIS](#) blog series. As I mentioned the last two Fridays, I got someone else to write the blog post instead. I figured people would probably still be showing up, so I thought it best to get someone on here writing.

This time we have Mark Vaillancourt ([Blog](#) | [@markvsql](#)) taking over the reigns. As with the other two guest bloggers, he's a co-worker at [Digineer](#) and is on the team with me that works with SSIS and on SQL Server projects.

Enough on the introduction, on with the show...

GUEST BLOGGER

When Jason Strate ([blog](#) | [Twitter](#)) offered spots for guest bloggers during his [31 Days of SSIS](#), I jumped at the chance. Yet more mentorship from Mr. Strate. And when I decided to blog about advantages of using the Foreach (File) Loop Container, I immediately thought of the late [Jim Croce](#). I'm sure you did, too, right? I know; it's pretty obvious.

Mr. Croce has a song called [You Don't Mess Around With Jim](#) that makes a more than suitable example for this post. This great song tells the story of "Big Jim Walker," a local tough guy that no one wanted to mess with. The chorus pretty much sums it up:

***You don't tug on Superman's cape
You don't spit into the wind
You don't pull the mask off that old Lone Ranger
And you don't mess around with Jim***

So, you get the idea that the name of the toughest guy around is going to "Jim" for some time to come.

It is rather like working on an SSIS project to import a file and being told that the name of the import file will always stay the same. Do you actually build the package with the name of that file hard coded in the Connection? A better option would be to put that name in a configuration file so it is easy to change if you get notified to do so. But neither of these will allow your solution to continue without error if the name of the file does not match what is expected. Thus, your solution is more fragile than it needs to be.

In situations like these, I use the Foreach Loop Container configured with the Foreach File Enumerator. I learned it from Aaron Drinkwine ([Twitter](#)), a colleague at Digineer. Aaron was the [guest blogger](#) on 1/14 in this series.

This method has excellent advantages which include the following:

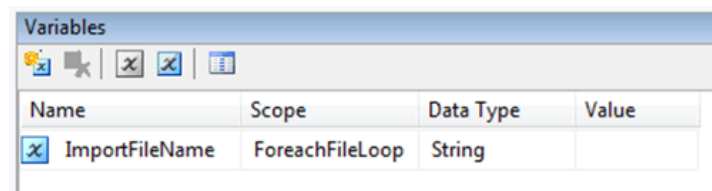
1. You can have the Foreach File Enumerator look for a pattern instead of a full name. For example, you could have it look for all .txt files in the selected folder. Or you could even configure it with part of a file name. This allows your solution to dynamically deal with filenames that are unexpected as long as they match the overall pattern.
2. The container includes functionality for dynamically capturing in a variable the actual name of the file that was read. You can then use this in your logging, notifications, emails, etc.

I typically then use this value in expressions to build the necessary strings involved in archiving the file (using the File System Task) after import. I learned this from Aaron, too. Jason discusses how to have variables derive their

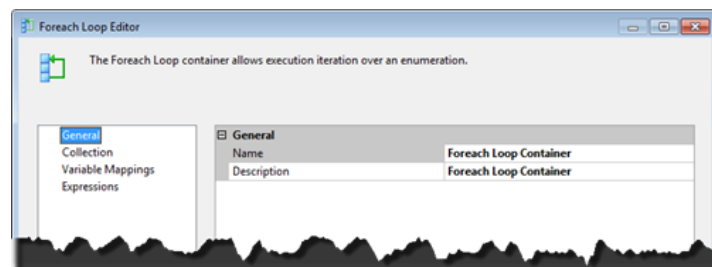
value from other variables using expressions in post 5 of this series: [31 Days of SSIS – One Variable To Rule Them All \(5/31\)](#).

If you don't archive import files in your solutions, I highly recommend it. I have been in situations where I built an SSIS package for a client that was importing from a third party. There were frequent data quality issues that the third party consistently tried to blame on my process. Each time, I was able to send them their own file showing that the issues came from their data, not the import process itself. This helped what was already a strong relationship with this client to get even stronger.

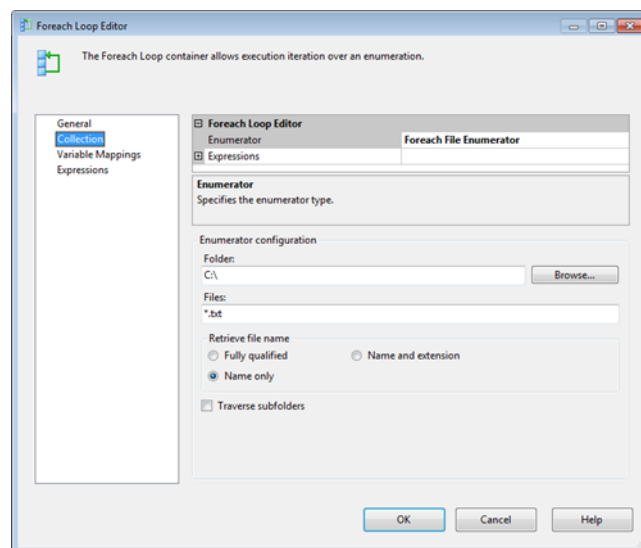
Before we go too far, let's create a package variable called ImportFileName with its type set to String. The Scope of our variable is set to the package, which is called ForeachFileLoop.



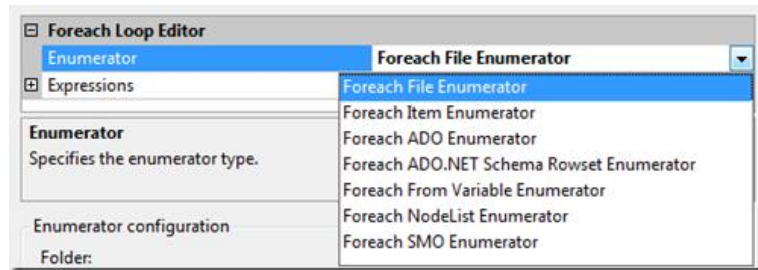
Let's take a look at the Foreach Loop Container and go through a configuration example. The General page of the Foreach Loop Editor allows you to customize the name of the container as well as add a description if you wish.



The Collection page is where you specify some crucial options.



You must choose an Enumerator, which determines what kind of object your container will iterate through. The Foreach File Enumerator, which we will be using today, is the default.



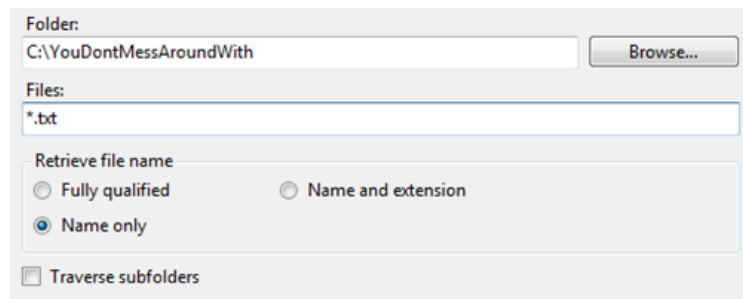
The Foreach File Enumerator has the following options on the Collection page:

Folder: This is the target folder that will be searched. I have created one called YouDontMessAroundWith.

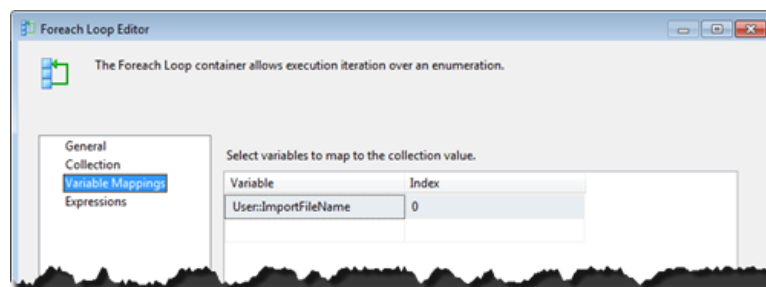
Files: This is where you set the pattern the enumerator will use in determining whether to pick up the file for processing.

Retrieve File Name: This determines how much of the file name and path are captured for later use. For our purposes, just the Name will be sufficient.

Traverse Subfolders: This allows you to choose whether the enumerator will include subfolders of the target folder when searching. Since our target folder has no subfolders, we will leave this option alone.



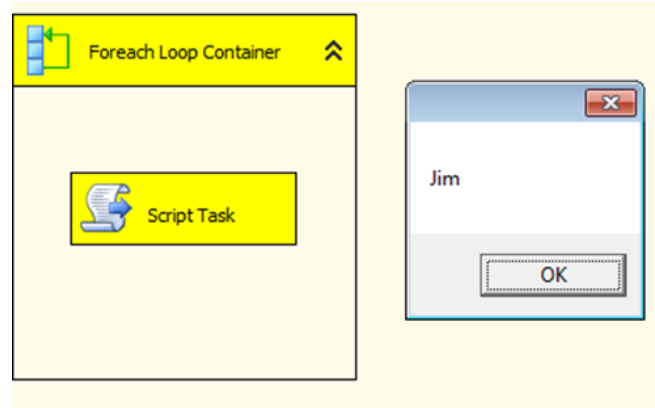
On the Variable Mappings page of the Foreach Loop Editor, you choose what variable(s) will be used. For our example, we will select our ImportFileName variable to house the file name captured. Notice that it is referred to as User::ImportFileName.



Our simple example will not be using the Expressions page of the Foreach Loop Editor, so we will leave that alone.

Since Jim is the man people don't mess with, I have created a file called Jim.txt in the YouDontMessAroundWith folder. This means when our package runs, the value of the ImportFileName variable will be set to "Jim" for later use.

I added a quick Script Task inside my Foreach Loop Container that just displays the value of the User::ImportFileName variable in a message box. When I execute my package, I get the following:

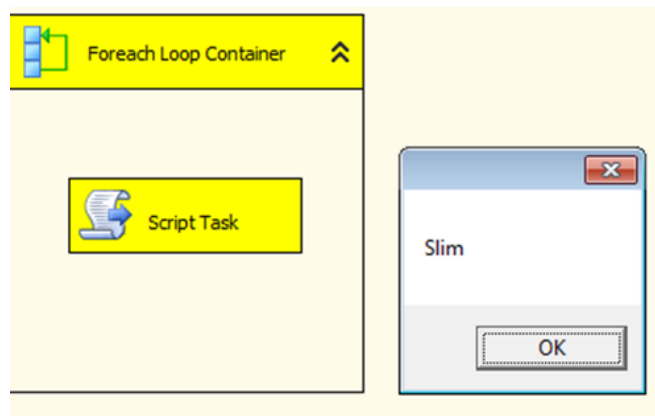


In the song, another fella, Slim, comes around and it doesn't go well for Jim. In fact, it goes so poorly that the chorus of the song changes.

***You don't tug on Superman's cape
You don't spit into the wind
You don't pull the mask off that old Lone Ranger
And you don't mess around with **Slim*****

To correspond with this unexpected change, I have replaced my Jim.txt file with Slim.txt. Now, if I had just hard coded the file name, or even used a config file, my package would fail.

Since I used the Foreach Loop Container, I get the following when running the same exact package with the new file.



We have a resilient, flexible solution with very little effort. So, we can even write our own version of the chorus that could be used throughout the entire song.

***You don't tug on Superman's cape
You don't spit into the wind
You don't pull the mask off that old Lone Ranger
And you don't mess around with User::ImportFileName***

EXPORTING EXECUTION PLANS

A day late for the twenty-ninth post of the 31 Days of SSIS, but it is still coming out. And if things go right, I'll be caught up where I need to be by the end of the day. If you're behind on this series, check out the introductory post.

FILE EXPORT REQUIREMENTS

When considering a post for today, I got to thinking about something that I'll do for clients when on performance tuning engagements. As a consultant, I can't always make the time to get out to the client to gather information, and provide feedback on whether I want to look at one plan or another. To get around this, I'll often have them execute an SSIS package that can provide me this information.

To demonstrate this functionality, let's suppose that I am investigating some potential issues with parallelism at a client. These issues typically occur in the wee hours of the night. Since I've read my post [Can You Dig It? – The Plan Cache](#), I have a query to run to find plans that are using parallelism. Unfortunately, when we look at the plan cache each morning, there is nothing of interest to investigate.

We need to find out what information is in the plan cache around the time that the issues are occurring. Since this information does not persist in the plan cache, we need this information placed somewhere less volatile than the SQL Server plan cache.

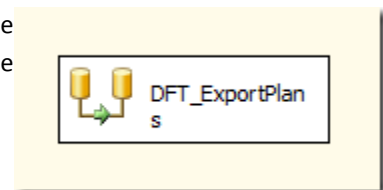
FILE EXPORT SOLUTION

Hopefully, we can agree that exporting data into a file with SSIS is easy. What about when you want to export the contents of a column in each row into a file? That is, fortunately, just as easy. Accomplishing this is as easy as creating a file name and using the Export Column transformation.

Since the information that we want to get is in the plan cache, we can use a query against the plan cache to retrieve all of the plans that utilize parallelism. In the export, we'll return a list of all of the plans that were found. At the same time, the package will use the Export Column transformation to save each of the execution plans to a file.

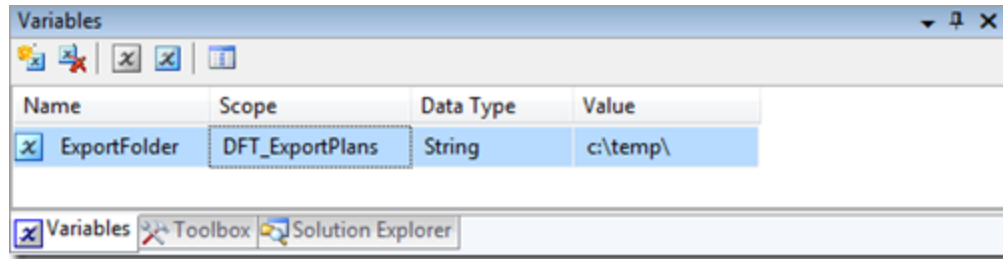
PACKAGE CONTROL FLOW

Our focus for this package is to look at the Export Column transformation. Due to this, our control flow will be very simple. Add a data flow task name DFT_ExportPlans and the control flow is almost done.



PACKAGE VARIABLES

We will need to add a variable to the package. This way, if the server doesn't have the same folder structure as my development environment there won't be any issues.



The variable from above is:

ExportFolder: Added to direct where to store the flat file with statistics and the execution plans.

PACKAGE DATA FLOW

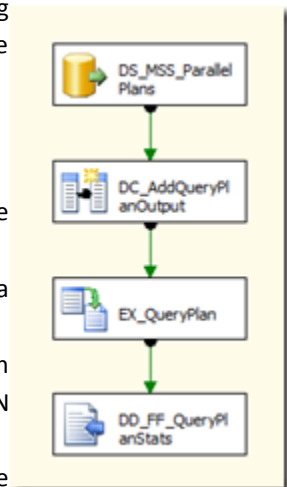
We only have a single data flow. This will do the work of exporting the statistics and files from the SQL Server.

DFT_EXPORTPLANS

As mentioned, this data flow will export the information on parallel plans that is being sought. The data flow task will query the plan cache to identify the plans that should be exported. Once identified, they will be exported with some statistics for offline analysis.

This is done with the following transformations:

- **DS_MSS_ParallelPlans:** OLE DB Data Source that will query the plan cache for plans utilizing parallelism.
- **DC_AddQueryPlanOutput:** Derived Column transformation that adds a column to create the path and file name for the execution plans.
- **EX_QueryPlan:** Export Column transformation that exports each execution plans into a file for each execution plan. These will be saved with the SQLPLAN extension so that SSMS recognizes the files as execution plans.
- **DD_FF_QueryPlanStats:** Flat File destination that provides an index of the execution plans exported along with some basic information on the plans.



DS_MSS_PARALLELPLANS QUERY

The query below is being used to identify plans that utilize parallelism. This query is derived from the post [Can You Dig It? – The Plan Cache](#); where I discuss querying the plan cache in further detail.

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED ;

WITH XMLNAMESPACES
(DEFAULT 'http://schemas.microsoft.com/sqlserver/2004/07/showplan')
SELECT TOP 10
CAST('ParallelPlan'+'_ '
+CAST(ROW_NUMBER() OVER (ORDER BY UseCounts DESC) AS VARCHAR(2))+'_ '
+REPLACE(REPLACE(REPLACE(CONVERT(VARCHAR(20), GETDATE(), 120), ' ', ''),
'-', ''), ':', ''))+'_ '
+CONVERT(VARCHAR(50), cp.plan_handle, 1) AS VARCHAR(128)) QueryPlanName
```

```

,cp.plan_handle
,COALESCE(DB_NAME(p.dbid),
p.query_plan.value('(/RelOp/OutputList/ColumnReference/@Database)[1]',
'nvarchar(128)')) AS DatabaseName --Works in a number of cases, but not perfect.
,DB_NAME(p.dbid)+'.'+OBJECT_SCHEMA_NAME(p.objectid, p.dbid)+'.'
+OBJECT_NAME(p.objectid, p.dbid) AS ObjectName
,cp.objtype
,cp.UseCounts
,p.query_plan
FROM sys.dm_exec_cached_plans cp
CROSS APPLY sys.dm_exec_query_plan(cp.plan_handle) p
CROSS APPLY sys.dm_exec_sql_text(cp.plan_handle) AS q
WHERE cp.cacheobjtype = 'Compiled Plan'
AND p.query_plan.exist('(/RelOp[@Parallel = "1"])') = 1

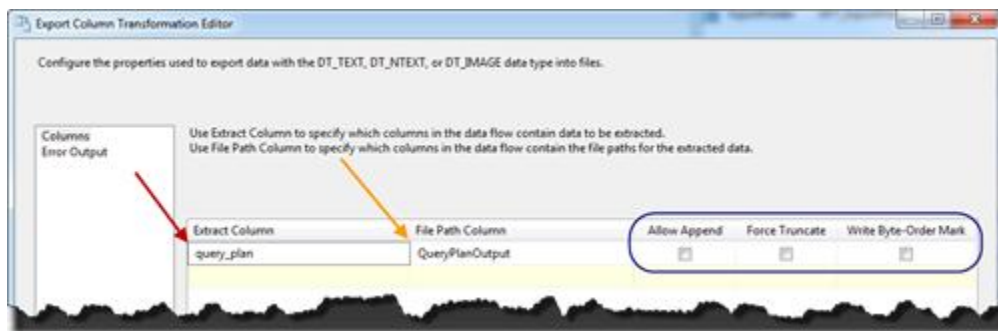
```

EC_QUERYPLAN CONFIGURATION

The key to this package is setting up the Export Column transformation. Fortunately, compared to many other tasks, setting up this transformation is relatively easy.

The first piece to configure is the Extract Column (red arrow below). This will map to a column in the data flow that has a data type that can be exported to a file. The only data types that can be exported are DT_TEXT, DT_NTEXT, or DT_IMAGE. The benefit of this limitation is that the 4,000 character limit for string data (DT_STR and DT_WSTR) doesn't interfere with this transformation.

Next that File Path Column needs to be configured (orange arrow below). This column represents the name of the file that the data will be placed within. In this solution, I've configured this through the use of a Derived Column transformation. This can be create in which ever manner you wish.



With those two items configured, the next step is to configure how the export will occur. There are three options for setting this up (blue bubble above):

- **Allow Append:** Determines whether data is appended to the end of the file if it exists. Otherwise, the data is added to a new file.
- **Force Truncate:** Determines whether data overwrites any data already in the file.
- **Write Byte-Order Mark:** Determines whether a byte-order mark (BOM) is written to the file. For DT_NTEXT data I have found that this value should be false. Otherwise, the file will be corrupt.

WRAP-UP

There are plenty of reasons you may want to export data in this manner from your database. The reason listed above is a prime example that I use as a consultant. Or maybe you want to use [Plan Explorer](#) and need to get a good selection of plans to investigate.

Outside of execution plans, you may need to change how a database stores image and document files. In this case, you can export out each of the files and place them in their new directory. If if you need to delete data that contains image or document information you could export it to archive the data in a manner that doesn't require restoring the SQL Server database.

This transformation can be very useful when you include it in your tool belt. Take some time to play with it, and you may find that a solution or two you've already built could have leveraged this transformation.

IMPORTING EXECUTION PLANS

Almost time to wrap up the [31 Days of SSIS](#). We are down to just two more posts with today's post being number thirty. If you're behind on this series, check out the [introductory post](#).

[This morning's one day late twenty-ninth post](#) went over the use of the Export Column transformation. Since we discussed that transformation, I figured it would be worthwhile to flip the coin and go through the use of the [Import Column transformation](#).

FILE IMPORT REQUIREMENTS

To demonstrate the Import Column, I am going to expand on the scenario that I used previously. In the previous post, I needed to export execution plans for later analysis. The execution plans exported in to files and sent over to me. Now let's put those plans back in a database so that they can be investigated with [XQuery](#) using some of the concepts I discuss in the [plan cache series](#).

FILE EXPORT SOLUTION

To accomplish this we need an SSIS package that can look at a directory and import all of the files in that folder into a table in a database. We'll use the Import Column transformation to populate the information to the table. The trouble is, there isn't a data source that returns directory contents. For this need, we'll use a Script Component transformation and configure it as a data source.

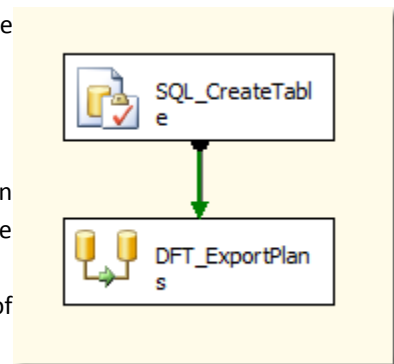
With those transformations, we should be able to load the contents of the folder. Then use that information to populate a table with the contents of all of the files that are discovered.

PACKAGE CONTROL FLOW

In this package, there are two tasks in the Control Flow. These tasks provide the table for the data and the data flow to load the data.

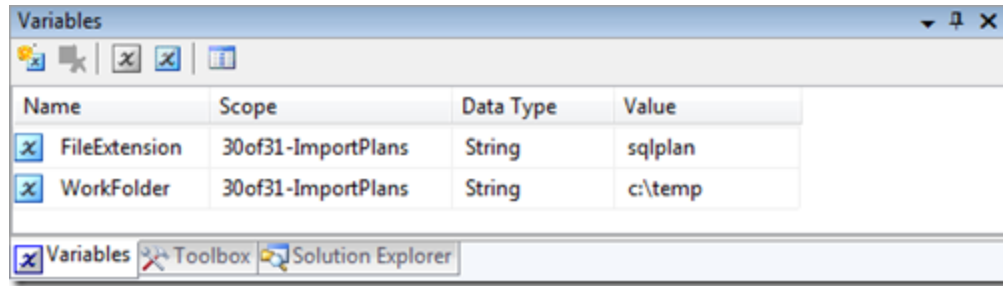
The components are:

- **SQL_CreateTable:** Execute SQL task that creates a table in tempdb. This table will store the files and their contents that are imported.
- **DFT_ExportPlans:** Data Flow task that imports the contents of files in the the table created in the previous task.



PACKAGE VARIABLES

To allow the most flexibility for rewriting this package to your needs, the package has been setup to use a couple package variables.



The variable from above are:

- **FileExtension:** Extension of the files that the SSIS package will be importing. Since we only want to import execution plans, the package will filter based on this extension.
- **WorkFolder:** Folder in which files are stored that are to be imported.

PACKAGE DATA FLOW

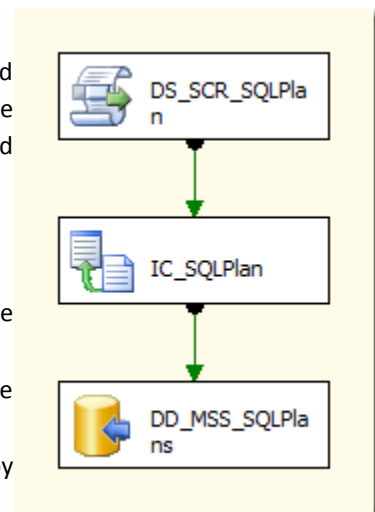
We only have a single data flow. This will list the files from a folder. The contents of these files will be imported into the table created in the Control Flow.

DFT_EXPORTPLANS

The Data Flow task here will be used to import the files from the directory stored in the variable above. These files will be read into the data flow and the contents will be pulled in as well. At completion, all of the data will be inserted into a table.

This is done with the following transformations:

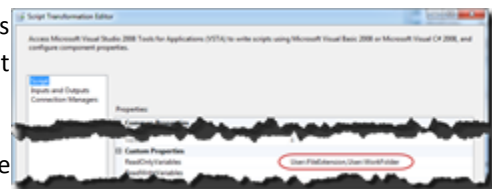
- **DS_SCR_SQLPlan:** Script Component transformation that reads the contents of a directory and returns a data flow output with those files.
- **IC_SQLPlan:** Import Columns transformation that will export the contents of the file names in the data flow path into a new column.
- **DD_MSS_SQLPlans:** OLE DB Destination for the data retrieved by the script component and input column transformations.



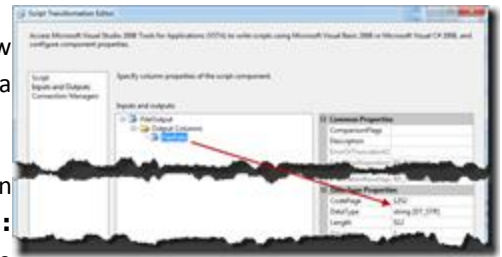
DS_SCR_SQLPLAN CONFIGURATION

Previous posts haven't covered using this transformation in this manner. Since this hasn't been done before, let's take a look a bit at how to configure this in your SSIS package.

To start, the **Script** tab will need to be configured. The transformation will need to have the variables FileExtension and WorkFolder added as **ReadOnlyVariables**. This is shown in the image above.



Next, configure the **Inputs and Outputs** tab. On this tab, add a new column to the Output Columns folder. Name this column **FilePath**. The data type for the column should be **DT_STR** with a width of 512.



With these configured, open the Script window for the transformation. Within the script window replace the contents of **ScriptMain : UserComponent** with the code below. Also, add a reference to **System.IO** at the beginning of the code block.

```
public override void CreateNewOutputRows()
{
    string path = Variables.WorkFolder.ToString();
    string extension = Variables.FileExtension.ToString();

    DirectoryInfo di = new DirectoryInfo(path);

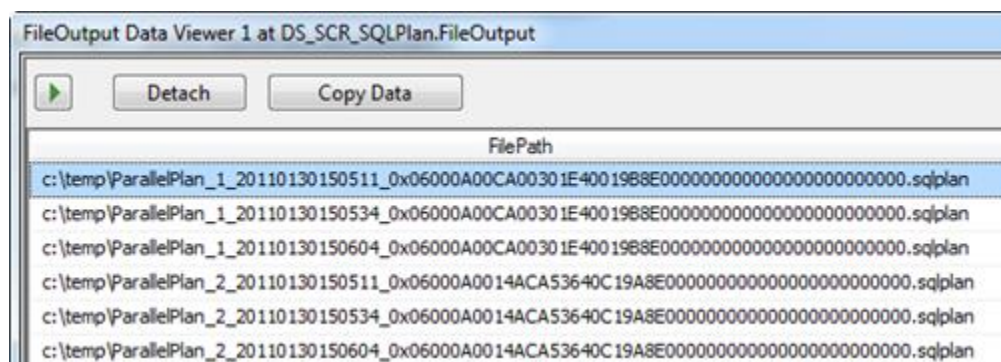
    //[] = di.GetFileSystemInfos(filter);

    foreach (FileInfo fsi in di.GetFileSystemInfos())
    {
        if (fsi.Extension == "." + extension)
        {
            FileOutputBuffer.AddRow();
            FileOutputBuffer.FilePath = fsi.FullName;
        }
    }
}
```

This code will use the **WorkFolder** variable to create a **DirectoryInfo** object. The **DirectoryInfo GetFileSystemInfos()** method is then called to return a collection populated with all of the files in that **WorkFolder**. While looping through all of the files, all of those that match the file extension in the **FileExtension** variable will be added to the data source output.

To add rows to the output of the Script Component, the **AddRow()** method of the output buffer is called. After the row is created, the values for the columns in the row are assigned. In this case, there is only a single column so just set the column **FilePath** to the **FullName** of the **FileInfo** object.

If we run the package now and look at the data through a Data Viewer, it should look similar to the image below:



The Import Column transformation can be used for more than just the above. Consider a situation where you have large images stored in a folder that can be previewed before downloading. You could create some thumbnail images for each and store those in the SQL Server database to reduce bandwidth. Or if you need to migrate some files from a system that stores files outside the database to a new system that stores them within the database.

Maybe it isn't even a migration or new functionality. Consider an admin that has to manually load a few dozen files to a database a day through a web application. How much time and energy would it save if you automated that process with an SSIS package. You might get a cup of coffee out of the deal.

Now, I must really sound like a broken record after a month of mentioning the Script Component transformation in various posts. But this transformation can really bring a lot of flexibility into your packages. Unlike the other packages, in this package the transformation was used as a data source. If you consider non-traditional data sources, like directories, this opens up a lot more ways to get to the data that you need and the way you want to get it.

What are your thoughts? How would you use the Import Column transformation? What data sources will you be looking at to get at with the Script Component transformation?

SSIS NAMING CONVENTIONS

We're at the thirty-first, and last, post for the [31 Days of SSIS](#). The series started with an [introductory post](#). This series has been a collection of the things that I've picked up over the years in my work with SSIS. I figured I'd end this series with a best practice that I've not often been a fan of and am now fully committed to.

The best practice I am talking about is the use of **naming conventions for tasks and transformation in SSIS packages**. Through this practice, the names of tasks and transformations will always start with an acronym or abbreviation followed by a meaningful name for the item. In the place of spaces, underscores are used in the names.

For a list of the naming conventions that I typically use, I will point you to Jamie Thompson's ([Blog](#) | [@JamieT](#)) post [SSIS: Suggested Best Practices and naming conventions](#). About half way down the post, are the naming conventions. The rest of the post is worth reading as well, if you haven't.

PRACTICE IMPACT

As I've worked with SSIS packages, I've found a few areas where this provides benefits. Whether developing new packages or maintaining and troubleshooting existing SSIS package, you will find an appreciation for this best practice.

- **Development Impact:** From a development perspective there is marginal immediate benefit for using this best practice. The only impact is the naming of the task or transformation to specific convention. If you don't typically name the objects in SSIS packages, start naming them and make it meaningful.
- **Maintenance Impact:** When maintaining an SSIS package, the use of a naming convention will help two-fold. First, the prefix will provide an instant indication of the context of the type of work being done. Second, the meaningful name portion of the name provides context for the work itself. When adding in new functionality, this recognition will help reduce the time it takes to understand yours or someone else's code that may have been written weeks or months ago.
- **Troubleshooting Impact:** As I mentioned in the last bullet the naming convention provides an immediate level of understanding of the type of task and the context. When troubleshooting, not having to crack open code and investigate configuration settings for every object is critical to solving issues efficiently. True, you may still end up examining everything. With the use of a naming convention, when you are looking at Billy's or Sally's SSIS packages, you can get a leg up on troubleshooting by having context when they are not around.

CONVINCING ME, CONVINCING YOU

After reading through the impacts, are you convinced? Maybe not, and until a few months ago neither was I. To conclude this best practice, I want to go through a couple of my experiences with naming conventions. This should help convince you that this best practice is worth the effort, as these experiences did for me as well.

LOOKING AT MY PACKAGE

I wrote an SSIS package a while back. This was a fairly simple package. I had a Data Flow task that was named similar to "Insert Rows to SalesOrder". A OLE DB destination within the data flow was named similar to "Insert Rows".

In the context of the developing the SSIS package, these names seemed perfectly reasonable. The package was tested, signed off on, and deployed. Of course, these actions don't always guarantee success when it hits production.

After a few weeks the package did what packages do when you forget about them. The package failed and I had to take a look at the log files. Opening up and digging into the SSIS package revealed that the error occurred on "Insert Rows to SalesOrder Insert Rows".

Since I was no longer working on this package, I had to think some and hunt some to figure out where the error occurred. Was the error in the task "Insert Rows" or "Insert Rows to SalesOrder"? Was the transformation "SalesOrder Insert Rows" or "Insert Rows"? These names may seem simple to figure out when developing the package, but after some time you may forget how you designed the package.

The names of objects in your SSIS packages should help you to immediately identify where to look and start with your troubleshooting. Would you be able to better remember what you were doing if the data flow task was named "DFT_InsertRowsToSalesOrder" and the OLE DB destination was named "OLE_DST_MSS_InsertRows"? Then the error message would say the error occurred at DFT_InsertRowsToSalesOrder OLE_DST_MSS_InsertRows.

LOOKING AT YOUR PACKAGE

You probably won't always be working with your own SSIS packages. Sometimes you will have to troubleshoot or maintain a package that someone else developed. In these cases, using the naming convention best practice will be worth its weight in gold. ~~Probably more since SSIS package aren't that heavy.~~

A while back I was monitoring some SSIS packages that had been deployed to production. Most of these packages had been written by others. Since I was on the administration side of the world, it was under my duties to troubleshoot packages when they failed.

One day, a package did fail. Actually, a lot of packages failed. Some schema changes had occurred and they were causing havoc in the SSIS packages. Due to these failures, I needed to investigate the packages and determine what needed to be done to resolve the issues.

Opening one of the first log files revealed that there was an error at "Data Flow Task 1 1 Derived Column 1 2". Where, oh where did this error go wrong? Of course, the log file does provide the reason for the error. But it doesn't do much to tell me the context for the error. In fact, based on these names, I already expect a mess inside the package. The names provided weren't meaningful, and the purpose could only be discerned by reverse engineering what the component does.

Had the error occurred at "DFT_ProcessHumans DC_AddIsZombieFlag" then I would have been better equipped to troubleshoot the package. I also, probably wouldn't be calling you at 2 AM in the morning so you can tell me what this package is supposed to be doing.

NAMING CONVENTION WRAP-UP

Hopefully, you've taken some time to consider this best practice. I'm a convert for exactly the reasons I've stated above. You should buy into this as well. It's not just about having meaningful names. It is also about leaving behind SSIS packages that others can troubleshoot and maintain.

